

## PROYECTO FIN DE GRADO

**TÍTULO:**

Desarrollo de Modelos Intrínsecamente Interpretables para Estimación de Vida Útil Remanente

**AUTOR:** Miguel Hidalgo Vicente

**TITULACIÓN:** Grado en Ingeniería y Sistemas de Datos

**DIRECTOR:** Mario Refoyo López

**TUTOR:** David Luengo García

**DEPARTAMENTO:** Ingeniería Audiovisual y Comunicaciones

**Miembros del Tribunal Calificador:**

**PRESIDENTE:** Francisco Aznar Ballesta

**TUTOR:** David Luengo García

**SECRETARIO:** Nicolás Sáenz Lechón

**Fecha de lectura:** 22 de Julio de 2024

**Calificación:**

**VºBº TUTOR/A**

LUENGO  
GARCIA  
DAVID -  
20205802A

Firmado  
digitalmente por  
LUENGO GARCIA  
DAVID -  
20205802A  
Fecha: 2024.07.13  
14:32:32 +02'00'

**El Secretario/La Secretaria,**

---

## **Agradecimientos**

Quiero expresar mi más sincero agradecimiento a mis padres y a mi hermana por su incondicional apoyo a lo largo de estos años de carrera. Su amor, comprensión y constante motivación han sido fundamentales para alcanzar este logro. Gracias por estar siempre ahí, brindándome su respaldo y confianza.

A mis amigos de toda la vida y a mi pareja, quiero agradecerles por su compañía y apoyo inquebrantable. Vuestra amistad y cariño han sido una fuente constante de alegría y fuerza. También quiero agradecer a los amigos que he hecho durante esta etapa universitaria, quienes han enriquecido mi vida con su compañía y con quienes he compartido innumerables momentos inolvidables.

Finalmente, quiero agradecer a mis tutores, Mario Refoyo López y David Luengo García, por su guía y dirección en este proyecto. Su experiencia, consejo y apoyo han sido vitales para la realización de este trabajo. Gracias por su paciencia y por creer en mí.



## **Resumen**

Este Proyecto de Fin de Grado (PFG) titulado "Desarrollo de Modelos Intrínsecamente Interpretables para Estimación de Vida Útil Remanente" se centra en el desarrollo de modelos predictivos que no solo sean precisos sino también transparentes y comprensibles. El objetivo principal es predecir la vida útil remanente (RUL) de componentes o sistemas, lo cual es crucial para el mantenimiento predictivo y la optimización de recursos en ingeniería y gestión de activos.

En este proyecto, se abordan las limitaciones de los modelos de caja negra, que, aunque son efectivos, no son fácilmente interpretables. Se propone el uso de modelos intrínsecamente interpretables, que proporcionan tanto precisión como explicaciones claras de sus predicciones. Se desarrollan y comparan dos algoritmos principales: RUL-RVE ("Remaining Useful Life - Recurrent Variational Encoder") y RUL-LAXCAT ("Remaining Useful Life - Latent Explanation-wise Attention for Time-series Classification"). Ambos modelos utilizan técnicas avanzadas de aprendizaje automático, como mecanismos de atención y espacios latentes interpretables.

La metodología del proyecto incluye el desarrollo, implementación y depuración de los modelos en Python, utilizando datos reales para evaluar su desempeño. Los resultados obtenidos demuestran que los modelos propuestos no solo alcanzan altos niveles de precisión en la estimación de RUL, sino que también ofrecen explicaciones comprensibles sobre cómo se llega a dichas predicciones. Esto es esencial para su aceptación y aplicación en entornos industriales, donde la confianza en las decisiones automáticas es crítica.

Finalmente, el proyecto concluye con una discusión sobre los resultados, comparando la precisión y la interpretabilidad de los modelos desarrollados, y proponiendo líneas futuras de investigación para mejorar aún más estas técnicas.



## **Abstract**

This Final Degree Project (PFG) titled “Development of Intrinsically Interpretable Models to Estimate Remaining Useful Life” focuses on the development of predictive models that are not only accurate but also transparent and understandable. The main objective is to predict the remaining useful life (RUL) of components or systems, which is crucial for predictive maintenance and the optimization of resources in engineering and asset management.

This project addresses the limitations of black-box models, which, although effective, are not easily interpretable. It proposes the use of intrinsically interpretable models that provide both accuracy and clear explanations of their predictions. Two main algorithms are developed and compared: RUL-RVE ("Remaining Useful Life - Recurrent Variational Encoder") and RUL-LAXCAT ("Remaining Useful Life - Latent Explanation-wise Attention for Time-series Classification"). Both models utilize advanced machine learning techniques, such as attention mechanisms and interpretable latent spaces.

The project's methodology includes the development, implementation, and debugging of the models in Python, using real data to evaluate their performance. The obtained results demonstrate that the proposed models not only achieve high levels of accuracy in RUL estimation but also offer understandable explanations of how these predictions are made. This is essential for their acceptance and application in industrial environments, where trust in automated decisions is critical.

Finally, the project concludes with a discussion of the results, comparing the accuracy and interpretability of the developed models, and proposing future research lines to further improve these techniques.



## Índice de Figuras

|                                                                                                                                                                                                          |           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <i>Figura 1. 1. Desglose de Tareas.....</i>                                                                                                                                                              | <i>21</i> |
| <i>Figura 2. 1. Ejemplo de agrupación (aprendizaje no supervisado) [8]. ....</i>                                                                                                                         | <i>24</i> |
| <i>Figura 2. 2. Ejemplo de aprendizaje semi-supervisado [9]. ....</i>                                                                                                                                    | <i>24</i> |
| <i>Figura 2. 3. Esquema de aprendizaje por refuerzo [10]. ....</i>                                                                                                                                       | <i>25</i> |
| <i>Figura 2. 4. Diagrama de bloques de aprendizaje automático supervisado, adaptado de [12]. ....</i>                                                                                                    | <i>26</i> |
| <i>Figura 2. 5. Ejemplo de regresión lineal [15]. ....</i>                                                                                                                                               | <i>27</i> |
| <i>Figura 2. 6. Estructura de los árboles de decisión [18]. ....</i>                                                                                                                                     | <i>28</i> |
| <i>Figura 2. 7. Estructura del “Random Forest” [19]. ....</i>                                                                                                                                            | <i>28</i> |
| <i>Figura 2. 8. Ejemplo gráfico de SVM [22]. ....</i>                                                                                                                                                    | <i>29</i> |
| <i>Figura 2. 9. Ejemplo de KNN [29]. ....</i>                                                                                                                                                            | <i>29</i> |
| <i>Figura 2. 10. Estructura de una Red Neuronal [27]. ....</i>                                                                                                                                           | <i>30</i> |
| <i>Figura 2. 11. Estructura de una RNN [24]. ....</i>                                                                                                                                                    | <i>32</i> |
| <i>Figura 2. 12. Estructura de una CNN [34]. ....</i>                                                                                                                                                    | <i>33</i> |
| <i>Figura 2. 13. Arquitectura de una GAN [38]. ....</i>                                                                                                                                                  | <i>33</i> |
| <i>Figura 2. 14. Arquitectura de un autocodificador [40]. ....</i>                                                                                                                                       | <i>34</i> |
| <i>Figura 2. 15. ML, DL e IA [42]. ....</i>                                                                                                                                                              | <i>35</i> |
| <i>Figura 3. 1. Clasificador de huskys y lobos [48]. ....</i>                                                                                                                                            | <i>40</i> |
| <i>Figura 3. 2. Ejemplo de explicaciones visuales [51]. ....</i>                                                                                                                                         | <i>41</i> |
| <i>Figura 3. 3. Diagrama de bloques de explicaciones ante-hoc y post-hoc [61]. ....</i>                                                                                                                  | <i>48</i> |
| <i>Figura 4. 1. Objetivos principales del Mantenimiento Predictivo [66]. ....</i>                                                                                                                        | <i>50</i> |
| <i>Figura 4. 2. GAN para la detección de fallos [67]. ....</i>                                                                                                                                           | <i>51</i> |
| <i>Figura 4. 3. “Reservoir Computing” para el diagnóstico de fallos [70]. ....</i>                                                                                                                       | <i>52</i> |
| <i>Figura 4. 4. Método basado en ESN para el pronóstico de fallos con mediciones basadas en eventos [66]. ....</i>                                                                                       | <i>53</i> |
| <i>Figura 5. 1. Arquitectura del algoritmo RUL-RVE [1]. ....</i>                                                                                                                                         | <i>57</i> |
| <i>Figura 5. 2. Representación del espacio latente de 2 dimensiones del modelo RUL-RVE [1]. Cada punto representa la RUL de una instancia. Cuanto más oscuro, está más cerca del 0 (del fallo). ....</i> | <i>58</i> |
| <i>Figura 5. 3. Proyección de las secuencias de datos de un conjunto de prueba. ....</i>                                                                                                                 | <i>61</i> |
| <i>Figura 5. 4. Implementación en Python del codificador recurrente. ....</i>                                                                                                                            | <i>63</i> |
| <i>Figura 5. 5. Implementación del modelo de regresión en Python. ....</i>                                                                                                                               | <i>64</i> |
| <i>Figura 5. 6. Implementación del modelo envolvente en Python. ....</i>                                                                                                                                 | <i>64</i> |

|                                                                                                                                     |    |
|-------------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 5. 7. Arquitectura de LAXCAT [4].                                                                                            | 65 |
| Figura 5. 8. Representación visual de las variables latentes [4].                                                                   | 66 |
| Figura 5. 9. Arquitectura del modelo RUL-LAXCAT.                                                                                    | 67 |
| Figura 5. 10. Ejemplo de mapa de calor de Atenciones Cruzadas.                                                                      | 70 |
| Figura 5. 11. Implementación en Python de la Atención Variable.                                                                     | 71 |
| Figura 5. 12. Implementación en Python de la Atención Temporal.                                                                     | 72 |
| Figura 5. 13. Implementación en Python del Modelo LaxCat.                                                                           | 72 |
| Figura 6. 1. Cálculo de etiqueta RUL para cada instancia de datos [1].                                                              | 78 |
| Figura 6. 2. Histograma de las atenciones de las variables de FD001 usando RUL-LAXCAT y simulado con todos los sensores.            | 79 |
| Figura 6. 3. Histograma del RMSE del conjunto de entrenamiento de FD001 usando RUL-LAXCAT.                                          | 81 |
| Figura 6. 4. Histograma de las atenciones de las variables de FD001 usando RUL-LAXCAT y simulado con 9 sensores.                    | 82 |
| Figura 6. 5. Mapa de calor de las atenciones cruzadas de FD001 usando RUL-LAXCAT y simulado con 9 sensores.                         | 82 |
| Figura 6. 6. RUL real vs. estimada de todas las trayectorias de validación de FD001 usando RUL-LAXCAT.                              | 83 |
| Figura 6. 7. Simulación 12 para “batch size” 32 y “learning rate” 0.001 (izquierda), y para “batch size” 64 y “learning rate” 0.01. | 84 |
| Figura 6. 8. Histograma de las atenciones de las variables de FD003 usando RUL-LAXCAT y simulado con todos los sensores.            | 85 |
| Figura 6. 9. Histograma del RMSE del conjunto de entrenamiento de FD003 usando RUL-LAXCAT.                                          | 85 |
| Figura 6. 10. Histograma de las atenciones de las variables de FD003 usando RUL-LAXCAT y simulado con 9 sensores.                   | 86 |
| Figura 6. 11. Mapa de calor de las atenciones cruzadas de FD003 usando RUL-LAXCAT y simulado con 9 sensores.                        | 86 |
| Figura 6. 12. RUL real vs. estimada de las trayectorias 12 y 14 de validación de FD003 usando RUL-LAXCAT.                           | 87 |
| Figura 6. 13. Simulación del conjunto de datos FD004 con 5 sensores (izquierda) y con todos los sensores (derecha).                 | 87 |
| Figura 6. 14. Espacio latente de los conjuntos de entrenamiento y prueba de FD001 usando RUL-RVE.                                   | 88 |
| Figura 6. 15. Trayectorias de validación 10 y 20 de FD001 usando RUL-RVE.                                                           | 89 |
| Figura 6. 16. Espacio latente de los conjuntos de entrenamiento y prueba de FD002 usando RUL-RVE.                                   | 90 |
| Figura 6. 17. Trayectorias de validación 2 y 47 de FD002 usando RUL-RVE.                                                            | 90 |
| Figura 6. 18. Espacio latente de los conjuntos de entrenamiento y prueba de FD003 usando RUL-RVE.                                   | 91 |

|                                                                                                                                                            |                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <i>Figura 6. 19. Trayectorias de validación 2 y 17 de FD003 usando RUL-RVE. ....</i>                                                                       | <i>92</i>      |
| <i>Figura 6. 20. Espacio latente de los conjuntos de entrenamiento y prueba de<br/>FD004 usando RUL-RVE. ....</i>                                          | <i>92</i>      |
| <i>Figura 6. 21. Trayectorias de validación 24 y 39 de FD004 usando RUL-RVE. ....</i>                                                                      | <i>93</i>      |
| <br><i>Figura B. 1. ODS 3: Salud y Bienestar, ODS 9: Industria, Innovación e<br/>Infraestructura, ODS 12: Producción y Consumo Responsables [92]. ....</i> | <br><i>102</i> |
| <br><i>Figura D. 1. Simulaciones de validación de FD003 usando RUL-LAXCAT para los 9<br/>mejores sensores. ....</i>                                        | <br><i>107</i> |



## Índice de Tablas

|                                                                                                                                             |            |
|---------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <i>Tabla 1. 1. Desglose de tareas por fechas. ....</i>                                                                                      | <i>21</i>  |
| <i>Tabla 6. 1. Descripción de los sensores por Tipo. ....</i>                                                                               | <i>77</i>  |
| <i>Tabla 6. 2. Tamaño de los sub-conjuntos de datos de C-MAPSS. ....</i>                                                                    | <i>77</i>  |
| <i>Tabla 6. 3. Resultados de las simulaciones con distinto número de sensores del conjunto FD001 usando RUL-LAXCAT. ....</i>                | <i>80</i>  |
| <i>Tabla 6. 4. Simulaciones con distintos valores de "batch size" y "learning rate" del conjunto de datos FD001 usando RUL-LAXCAT. ....</i> | <i>80</i>  |
| <i>Tabla 6. 5. Comparación de las simulaciones de RUL-LAXCAT y RUL-RVE. ....</i>                                                            | <i>93</i>  |
| <i>Tabla A. 1. Coste de materiales. ....</i>                                                                                                | <i>99</i>  |
| <i>Tabla A. 2. Coste de Personal. ....</i>                                                                                                  | <i>99</i>  |
| <i>Tabla A. 3. Coste total del Proyecto. ....</i>                                                                                           | <i>100</i> |



## **Lista de Acrónimos**

**AE** “AutoEncoder”, Autocodificador

**ANN** “Artificial Neural Network”, Red Neuronal Artificial

**ATT** “Average Training Time”, Tiempo de Entrenamiento Promedio

**CAM** “Class Activation Map”, Mapeo de Activación de Clases

**CatAAE** “Categorical Adversarial AutoEncoder”, Autocodificador Categórico Adversarial

**CM** “Condition Monitoring”, Monitorización de la Condición

**CMASS** “Commercial Modular Aero-Propulsion System Simulation”

**CNN** “Convolutional Neural Network”, Red Neuronal Convolucional

**DL** “Deep Learning”, Aprendizaje Profundo

**EDA** “Exploratory Data Analysis”, Análisis Exploratorio de los Datos

**ESN** “Echo State Network”, Red de Estado de Eco

**GAP** “Global Average Pooling”, Agrupación Promedio Global

**GAN** “Generative Adversarial Networks”

**Grad-CAM** “Gradient-weighted Class Activation Mapping”

**IA** Inteligencia Artificial

**ILSVRC** “ImageNet Large Scale Visual Recognition Challenge”

**KL** “Kullback-Leibler”

**K-NN** “K-Nearest Neighbour”, K-Vecinos más Cercanos

**LIME** “Local Interpretable Model-agnostic Explanations”

**LRP** “Layer-wise Relevance Propagation”, Propagación de relevancia por Capas

**LSTM** “Long Short-Term Memory”

**MAVIS** “Microsoft Audio Video Indexing Service”

**ML** “Machine Learning”, Aprendizaje Automático

**MNIST** “Modified National Institute of Standards and Technology”

**MSE** “Mean Square Error”, Error Cuadrático Medio

- NASA** “National Aeronautics and Space Administration”
- ODS** Objetivos de Desarrollo Sostenible
- PCA** “Principal Component Analysis”, Análisis de Componentes Principales
- PdM** “Predictive Maintenance”, Mantenimiento Predictivo
- PFG** Proyecto de Fin de Grado
- PIAE** “Physics Informed AutoEncoder”, Autocodificador Informado por Física
- ReLU** “Rectified Linear Unit”
- RNN** “Recurrent Neural Network”, Red Neuronal Recurrente
- RMSE** “Root Mean Square Error”, Raíz del Error Cuadrático Medio
- RUL** “Remaining Usefull Life”, Vidal Útil Remanente
- RUL-LAXCAT** “Remaining Useful Life - Latent Explanation-wise Attention for Time-series Classification”
- RUL-RVE** “Remaining Useful Life - Recurrent Variational Encoder”
- SARSA** “State-Action-Reward-State-Action”, Estado-Acción Recompensa-Estado-Acción
- SHAP** “SHapley Additive exPlanations”
- SL** “Shallow Learning”, Aprendizaje Superficial
- SSC** “Structures, Systems and Components”, Estructuras, Sistemas y Componentes
- SVM** “Support Vector Machine”, Máquina de Vectores de Soporte
- TOI** “Time Of Interest”, Intervalo de Interés
- XAI** “eXplainable Artificial Intelligence”, Inteligencia Artificial Explicable

# Índice de Contenidos

|                                                                                            |             |
|--------------------------------------------------------------------------------------------|-------------|
| <b>Agradecimientos .....</b>                                                               | <b>i</b>    |
| <b>Resumen .....</b>                                                                       | <b>iii</b>  |
| <b>Abstract .....</b>                                                                      | <b>v</b>    |
| <b>Índice de Figuras .....</b>                                                             | <b>vii</b>  |
| <b>Índice de Tablas.....</b>                                                               | <b>xi</b>   |
| <b>Lista de Acrónimos .....</b>                                                            | <b>xiii</b> |
| <b>Índice de Contenidos .....</b>                                                          | <b>xv</b>   |
| <b>Capítulo 1: Introducción .....</b>                                                      | <b>19</b>   |
| 1.1. <i>Introducción .....</i>                                                             | 19          |
| 1.2. <i>Objetivos.....</i>                                                                 | 19          |
| 1.3. <i>Especificaciones y Restricciones del Diseño .....</i>                              | 20          |
| 1.4. <i>Metodología de Trabajo Propuesta .....</i>                                         | 20          |
| 1.5. <i>Desglose de Tareas y Cronograma .....</i>                                          | 20          |
| 1.6. <i>Organización de la Memoria .....</i>                                               | 21          |
| <b>Capítulo 2: Aprendizaje Automático.....</b>                                             | <b>23</b>   |
| 2.1. <i>Introducción .....</i>                                                             | 23          |
| 2.2. <i>¿Qué es el Aprendizaje Automático?.....</i>                                        | 23          |
| 2.3. <i>Aprendizaje Supervisado: Diagrama de Bloques .....</i>                             | 25          |
| 2.4. <i>Métodos Tradicionales de Aprendizaje Supervisado .....</i>                         | 26          |
| 2.5. <i>¿Qué es el Aprendizaje Profundo?.....</i>                                          | 30          |
| 2.6. <i>Métodos de Aprendizaje Profundo .....</i>                                          | 32          |
| 2.7. <i>Aprendizaje Automático vs. Aprendizaje Profundo.....</i>                           | 34          |
| 2.8. <i>Modelos de Caja Blanca y Modelos de Caja Negra: Rendimiento vs. Explicabilidad</i> | 36          |
| 2.9. <i>Discusión .....</i>                                                                | 36          |
| <b>Capítulo 3: Explicabilidad.....</b>                                                     | <b>39</b>   |
| 3.1. <i>Introducción .....</i>                                                             | 39          |
| 3.2. <i>¿Qué es la explicabilidad?.....</i>                                                | 39          |
| 3.3. <i>Características de la Explicabilidad .....</i>                                     | 42          |
| 3.4. <i>Taxonomía de la Inteligencia Artificial Explicable .....</i>                       | 44          |
| 3.5. <i>Explicabilidad Ante-hoc y Post-hoc.....</i>                                        | 44          |
| 3.5.1. <i>¿Qué es la Explicabilidad Ante-hoc? .....</i>                                    | 44          |
| 3.5.2. <i>¿Qué es la Explicabilidad Post-hoc? .....</i>                                    | 45          |
| 3.5.3. <i>Explicabilidad Ante-hoc vs. Post-hoc .....</i>                                   | 47          |
| 3.6. <i>Discusión.....</i>                                                                 | 48          |
| <b>Capítulo 4: Mantenimiento Predictivo .....</b>                                          | <b>49</b>   |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 4.1. Introducción .....                                   | 49        |
| 4.2. ¿Qué es el Mantenimiento Predictivo? .....           | 49        |
| 4.3. ¿Qué es la Vida Útil Remanente? .....                | 52        |
| 4.4. Explicabilidad en Mantenimiento Predictivo .....     | 54        |
| 4.4.1. Métodos Post-hoc en Mantenimiento Predictivo ..... | 54        |
| 4.4.2. Métodos Ante-hoc en Mantenimiento Predictivo ..... | 55        |
| 4.5. Discusión .....                                      | 56        |
| <b>Capítulo 5: Algoritmos Propuestos.....</b>             | <b>57</b> |
| 5.1. Introducción .....                                   | 57        |
| 5.2. Algoritmo RUL-RVE .....                              | 57        |
| 5.2.1. Arquitectura del Modelo .....                      | 57        |
| 5.2.2. Función de Coste y Regularización .....            | 59        |
| 5.2.3. Explicaciones de RUL-RVE .....                     | 61        |
| 5.2.3. Implementación en Python .....                     | 62        |
| 5.3. Algoritmo RUL-LAXCAT.....                            | 65        |
| 5.3.1. Arquitectura de LAXCAT .....                       | 65        |
| 5.3.2. Arquitectura de RUL-LAXCAT .....                   | 66        |
| 5.3.3. Función de Coste.....                              | 69        |
| 5.3.4. Explicaciones de RUL-LAXCAT.....                   | 69        |
| 5.3.5. Implementación en Python .....                     | 71        |
| 5.4. Discusión .....                                      | 72        |
| <b>Capítulo 6: Resultados .....</b>                       | <b>75</b> |
| 6.1. Introducción .....                                   | 75        |
| 6.2. Conjunto de Datos C-MAPSS.....                       | 75        |
| 6.3. Simulaciones en RUL-LAXCAT .....                     | 78        |
| 6.3.1. FD001 .....                                        | 78        |
| 6.3.2. FD002.....                                         | 84        |
| 6.3.3. FD003.....                                         | 84        |
| 6.3.4. FD004.....                                         | 87        |
| 6.4. Simulaciones en RUL-RVE .....                        | 88        |
| 6.3.1. FD001 .....                                        | 88        |
| 6.3.2. FD002.....                                         | 89        |
| 6.3.3. FD003.....                                         | 91        |
| 6.3.4. FD004.....                                         | 92        |
| 6.5. Comparación de los Modelos.....                      | 93        |
| 6.5.1. Precisión y Tiempo de Entrenamiento.....           | 93        |
| 6.5.2. Calidad de las Explicaciones.....                  | 94        |
| 6.6. Discusión .....                                      | 94        |
| <b>Capítulo 7: Conclusiones y Líneas Futuras .....</b>    | <b>95</b> |
| 7.1. Introducción .....                                   | 95        |
| 7.2. Conclusiones.....                                    | 95        |
| 7.3. Líneas Futuras .....                                 | 96        |
| 7.4. Discusión.....                                       | 97        |

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| <b>Apéndice A: Presupuesto.....</b>                                           | <b>99</b>  |
| <i>A.1. Coste de Materiales .....</i>                                         | 99         |
| <i>A.2. Coste de Personal .....</i>                                           | 99         |
| <i>A.3. Coste Total .....</i>                                                 | 100        |
| <b>Apéndice B: Impacto del Proyecto .....</b>                                 | <b>101</b> |
| <i>B.1. Introducción .....</i>                                                | 101        |
| <i>B.2. Impacto Social.....</i>                                               | 101        |
| <i>B.3. Impacto en la Salud y la Seguridad .....</i>                          | 101        |
| <i>B.4. Impacto Económico .....</i>                                           | 102        |
| <i>B.5. Impacto Tecnológico e Industrial .....</i>                            | 102        |
| <i>B.6. Contribución a los Objetivos de Desarrollo Sostenible (ODS) .....</i> | 102        |
| <i>B.7. Discusión.....</i>                                                    | 103        |
| <b>Apéndice C: Manual de Usuario .....</b>                                    | <b>105</b> |
| <i>C.1. RUL-RVE .....</i>                                                     | 105        |
| <i>C.2. RUL-LAXCAT.....</i>                                                   | 105        |
| <b>Apéndice D: Resultados Adicionales .....</b>                               | <b>107</b> |
| <b>Referencias .....</b>                                                      | <b>109</b> |



# Capítulo 1: Introducción

## 1.1. Introducción

En el ámbito de la ingeniería y la gestión de activos, la estimación de la Vida Útil Remanente (RUL) desempeña un papel fundamental para garantizar la eficiencia operativa y la seguridad de los sistemas. La capacidad de prever con precisión cuándo un componente o sistema alcanzará el final de su vida útil es crucial para planificar mantenimientos preventivos, optimizar recursos y reducir costes asociados a fallos inesperados [1].

Los modelos predictivos, como las redes neuronales o los algoritmos de regresión, han demostrado ser efectivos para estimar la RUL. Sin embargo, su complejidad a menudo los convierte en cajas negras [2]. ¿Cómo podemos confiar en un modelo que no podemos entender completamente? ¿Cómo explicamos a los ingenieros y responsables de la toma de decisiones por qué una turbina específica está cerca de su límite de vida útil?

Aquí es donde entra en juego la explicabilidad. Los modelos intrínsecamente interpretables están diseñados para ser transparentes desde el principio. No solo se trata de predecir correctamente; también se trata de explicar cómo se llega a esa predicción. Estos modelos buscan un equilibrio entre precisión y comprensibilidad [3]. Por un lado, deben proporcionar estimaciones precisas de la RUL. Por otro lado, deben ofrecer explicaciones claras y comprensibles para que los profesionales confíen en las decisiones tomadas por estos sistemas.

El presente Proyecto de Fin de Grado (PFG) se centra en la implementación y evaluación de modelos intrínsecamente interpretables para la estimación de Vida Útil Remanente. Las tendencias actuales utilizan espacios latentes interpretables [1], mecanismos de atención [4] o redes prototípicas [5]. Estos modelos no solo buscan alcanzar altos niveles de precisión en sus predicciones, sino que también se esfuerzan por proporcionar explicaciones claras y comprensibles sobre cómo llegan a dichas predicciones. Esta dualidad entre precisión e interpretabilidad es esencial para garantizar la aceptación y adopción efectiva de estas herramientas en entornos industriales y de ingeniería.

## 1.2. Objetivos

En el presente proyecto se persiguen los siguientes objetivos específicos:

- Revisión del estado del arte en técnicas de explicabilidad y vida útil remanente.
- Desarrollo de modelos intrínsecamente interpretables para estimación de vida útil remanente (RUL).
- Implementación y depuración en Python, a través de Visual Studio Code, de los modelos intrínsecamente interpretables desarrollados.
- Utilización de los modelos intrínsecamente interpretables con señales reales.
- Evaluación de los resultados obtenidos.

### **1.3. Especificaciones y Restricciones del Diseño**

En el proyecto se parte de las siguientes especificaciones y restricciones de diseño:

- El tiempo de evaluación (“Test”) debe ser aceptable.
- Los modelos deben maximizar la fidelidad (“Accuracy”), equilibrar la precisión y la comprensibilidad, y garantizar la portabilidad, esto es, permitir que sean aplicables en distintos entornos [6].
- Los modelos han de ser flexibles en cuanto al número de señales con las que se trabaja y su arquitectura interna.
- Los modelos deben ser validados con señales reales.

### **1.4. Metodología de Trabajo Propuesta**

La metodología propuesta en el presente PFG se basa en un proceso iterativo:

- 1) Desarrollar un modelo.
- 2) Implementarlo en Python.
- 3) Depurar el modelo y testearlo.
- 4) Identificar y corregir los errores.
- 5) Repetir los pasos 3) y 4) hasta que no haya errores.
- 6) Utilizar el modelo con señales reales.
- 7) Repetir los pasos anteriores para todos los modelos desarrollados.
- 8) Comparar las métricas de los distintos modelos.

### **1.5. Desglose de Tareas y Cronograma**

El desarrollo del proyecto se llevará a cabo a lo largo de 4 meses y medio con dedicación a tiempo parcial. Se dedicarán aproximadamente 320 horas para la realización del proyecto. El desglose de tareas es el mostrado en la Figura 1.1.

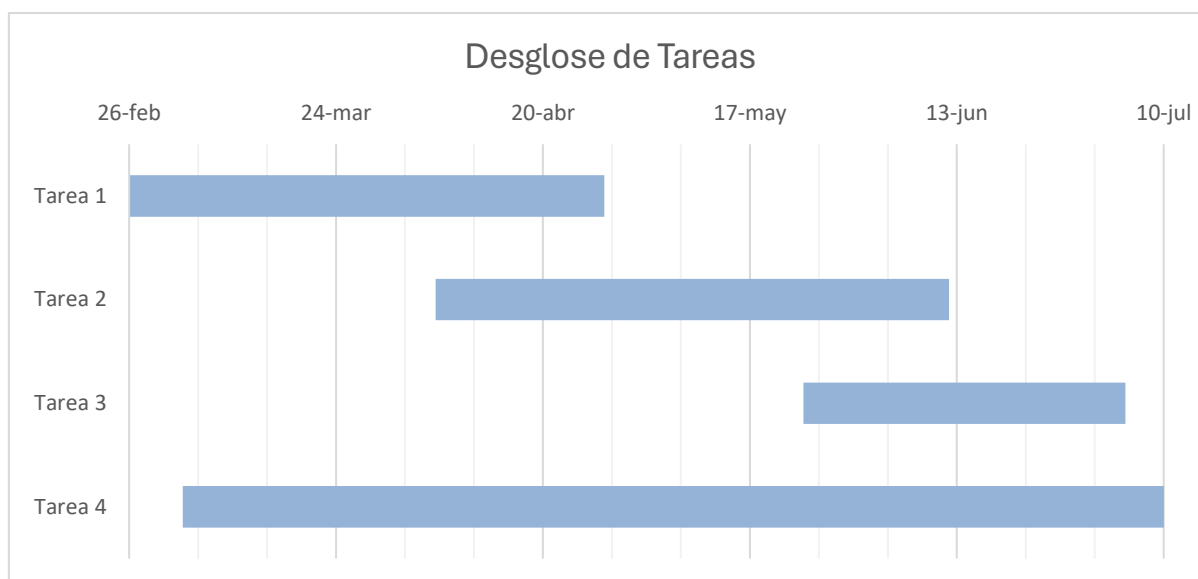


Figura 1. 1. Desglose de Tareas.

- **Tarea 1:** Revisión de modelos intrínsecamente interpretables y técnicas de estimación de vida útil remanente.
- **Tarea 2:** Desarrollo, programación y depuración de los modelos intrínsecamente interpretables para estimación de vida útil remanente en Python.
- **Tarea 3:** Simulación de los modelos usando señales reales, seguida de la evaluación y comparación de su rendimiento, así como de su interpretabilidad.
- **Tarea 4:** Redacción de la memoria del proyecto y preparación de la presentación.

| Nombre de la Tarea | Día inicio | Duración (días) | Día fin | Horas      |
|--------------------|------------|-----------------|---------|------------|
| Tarea 1            | 26-feb     | 62              | 28-abr  | 70         |
| Tarea 2            | 06-abr     | 67              | 12-jun  | 100        |
| Tarea 3            | 24-may     | 42              | 05-jul  | 80         |
| Tarea 4            | 04-mar     | 128             | 10-jul  | 70         |
| <b>Total</b>       |            | <b>135</b>      |         | <b>320</b> |

Tabla 1. 1. Desglose de tareas por fechas.

## 1.6. Organización de la Memoria

La memoria del proyecto se organiza en siete capítulos y tres apéndices.

El primer capítulo se trata del anteproyecto. En él se hace una introducción al tema del proyecto y se explican los objetivos, las especificaciones, la metodología, las tareas y la organización de la memoria.

Los capítulos 2, 3 y 4 corresponden al marco tecnológico del proyecto. En ellos se exploran las tecnologías y conceptos necesarios para comprender el desarrollo de modelos intrínsecamente interpretables para la estimación de la vida útil remanente. Se comienza con una visión global del campo en el que se sitúa el proyecto, para finalizar explicando en detalle el área específica de este. El segundo capítulo explica el concepto clave del

aprendizaje automático, además de revisar sus paradigmas más comunes y distintas aplicaciones. El capítulo 3 se centra en la explicabilidad de los modelos de aprendizaje automático. Por su parte, el capítulo 4 explora los conceptos y trabajos relacionados con el mantenimiento predictivo y los modelos explicables para estimar el RUL (“Remaining Useful Life”).

En el capítulo 5 se explica tanto el algoritmo desarrollado, RUL-LAXCAT, como el algoritmo con el que se compararán los resultados, RUL-RVE. En ambos casos se habla de su arquitectura y de su implementación en Python.

En el capítulo 6 se muestran los resultados obtenidos después de realizar distintas simulaciones. Se comparan los resultados y las explicaciones de ambos algoritmos.

En el capítulo 7 se presentan las conclusiones del trabajo y las líneas futuras donde se puede seguir trabajando.

En cuanto a los apéndices, el primero se centra en los presupuestos del proyecto, el segundo en el impacto del proyecto y la contribución a los ODS (Objetivos de Desarrollo Sostenible), y el tercero es un manual de usuario donde se explica el flujo del código.

## Capítulo 2: Aprendizaje Automático

### 2.1. Introducción

En este capítulo se explorarán los conceptos fundamentales del aprendizaje automático y el aprendizaje profundo, dos pilares esenciales en la inteligencia artificial. Se revisarán los paradigmas principales del aprendizaje automático, incluyendo el aprendizaje supervisado, no supervisado, semi-supervisado y por refuerzo, destacando sus aplicaciones prácticas y relevancia. Además, se enfatizará en el aprendizaje profundo, analizando su capacidad para manejar datos complejos y extraer características relevantes automáticamente, diferenciándose así de los métodos tradicionales.

Asimismo, se analizarán los métodos tradicionales de aprendizaje supervisado y se compararán con los enfoques de aprendizaje profundo. La discusión incluirá la distinción entre modelos de caja blanca y caja negra, abordando las implicaciones en términos de rendimiento y explicabilidad. Con esto, se sientan las bases para desarrollar modelos predictivos robustos y eficientes para la estimación de RUL.

### 2.2. ¿Qué es el Aprendizaje Automático?

El “machine learning”, aprendizaje máquina o aprendizaje automático, es un conjunto de métodos que utilizan las máquinas para realizar predicciones o mejorar comportamientos basados en datos [6]. El objetivo principal es dotar a las máquinas de la capacidad de aprendizaje. Por ejemplo, para predecir el valor de una casa, la máquina aprendería patrones de ventas pasadas de casas.

En el aprendizaje automático se distinguen 4 paradigmas principales [7]:

- **Aprendizaje supervisado.** En este enfoque, el modelo se entrena utilizando un conjunto de datos etiquetado, donde cada ejemplo de entrada está asociado con una etiqueta de salida conocida. El objetivo es aprender una función que relacione las entradas con las salidas. Ejemplos comunes de aplicaciones incluyen la clasificación (por ejemplo, un clasificador de imágenes de perros y gatos) y la regresión (por ejemplo, un modelo para calcular el precio de una casa en función de distintos parámetros). Algunos ejemplos de algoritmos de aprendizaje supervisado son las máquinas de vectores de soporte (SVM), los árboles de decisión, la regresión lineal y las redes neuronales.
- **Aprendizaje no supervisado.** Aquí, el modelo se entrena con un conjunto de datos no etiquetado, lo que significa que no hay salida conocida asociada con las entradas. El objetivo es encontrar patrones o estructuras interesantes dentro de los datos. Algunos ejemplos de tareas de aprendizaje no supervisado incluyen la agrupación (“clustering”), la reducción de dimensionalidad y la asociación entre las características de los datos. Algoritmos comunes en este dominio incluyen el k-means, el análisis de componentes principales (PCA) y los algoritmos de asociación o agrupación. En la Figura 2.1 podemos ver un ejemplo gráfico de cómo funciona un algoritmo de agrupación. Se representan todos los datos, y en función

del resultado obtenido y los parámetros elegidos, el algoritmo encuentra  $X$  núcleos, y agrupa los datos en esos  $X$  tipos [8].

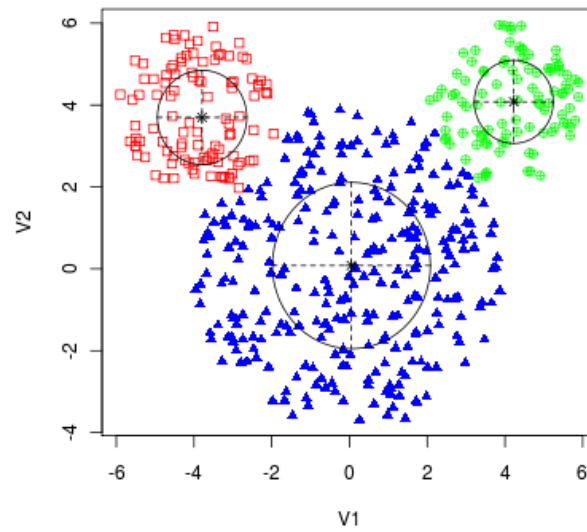


Figura 2. 1. Ejemplo de agrupación (aprendizaje no supervisado) [8].

- Aprendizaje semi-supervisado.** Este tipo de aprendizaje combina elementos de aprendizaje supervisado y no supervisado. El modelo se entrena con un conjunto de datos que contiene tanto ejemplos etiquetados como no etiquetados. Esto puede ser útil cuando los conjuntos de datos etiquetados son escasos o difíciles de obtener. Algunos algoritmos de aprendizaje semi-supervisado son métodos de propagación de etiquetas y técnicas de auto-aprendizaje. En la Figura 2.2 se puede ver un ejemplo de uso de aprendizaje semi-supervisado. El punto blanco y el punto negro son los únicos etiquetados. Como podemos ver en la parte inferior de la figura, el algoritmo hace una agrupación de todos los datos basándose solo en la etiqueta de estos dos, en lugar de la frontera lineal mostrada en la parte superior que sería lo natural con solo dos datos etiquetados.

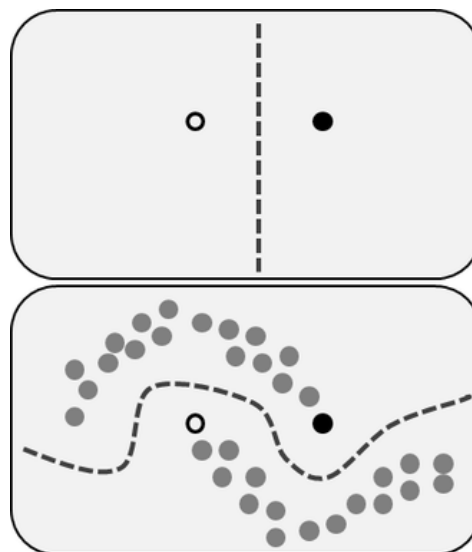


Figura 2. 2. Ejemplo de aprendizaje semi-supervisado [9].

- **Aprendizaje por refuerzo.** En este paradigma, un agente aprende a tomar decisiones secuenciales interactuando con un entorno dinámico. El agente recibe retroalimentación en forma de recompensas o castigos en función de sus acciones, y el objetivo es aprender una política óptima que maximice la recompensa a largo plazo. En la Figura 2.3 podemos ver un esquema de cómo funciona el aprendizaje por refuerzo. Se aplica en juegos, robótica y control de procesos. Los algoritmos más populares de aprendizaje por refuerzo son Q-learning, SARSA (Estado-Acción-Recompensa-Estado-Acción), así como algoritmos basados en políticas.

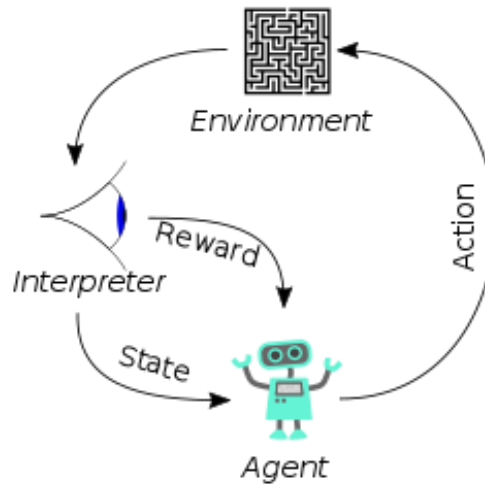


Figura 2. 3. Esquema de aprendizaje por refuerzo [10].

A continuación, se detallará el paradigma de aprendizaje automático supervisado, el cual se utilizará para desarrollar los modelos de estimación de vida útil remanente en este proyecto.

### 2.3. Aprendizaje Supervisado: Diagrama de Bloques

Como se ha mencionado antes, el objetivo del aprendizaje supervisado es aprender un modelo predictivo que asocie características de los datos (por ejemplo, tamaño de la casa, ubicación, tipo de suelo, etc.) a una salida (por ejemplo, precio de la casa). Si la salida es categórica, la tarea se llama clasificación, y si es numérica, se llama regresión. El algoritmo de aprendizaje automático aprende un modelo estimando parámetros (como pesos) o aprendiendo estructuras (como árboles). El algoritmo está guiado por una función de puntuación o pérdida que se minimiza. En el ejemplo del valor de la casa, la máquina minimiza la diferencia entre el precio de la casa estimado y el precio real. Un modelo de aprendizaje automático completamente entrenado se puede usar para hacer predicciones para nuevas instancias. La estimación de precios de casas, recomendaciones de productos, detección de señales de tráfico, predicción de incumplimiento de créditos y detección de fraudes tienen en común que pueden resolverse mediante aprendizaje automático [6]. Las tareas son diferentes, pero el enfoque es el mismo [11]:

1. **Recolección de datos y etiquetado (Preprocesamiento):** Se recopilan datos,  $D=\{(x_1,y_1),(x_2,y_2),\dots,(x_N,y_N)\}$ , donde  $x_i$  representa un vector de características de entrada e  $y_i$  representa la salida asociada (categórica o numérica). Estos datos se preparan adecuadamente para el modelado, lo que incluye el manejo de valores vacíos, la normalización de características y la codificación de salidas.

2. **Entrenamiento del modelo (Ajuste de parámetros):** Se utiliza un algoritmo de aprendizaje automático para ajustar los parámetros de un modelo  $f(x;\theta)$ , donde  $\theta$  representa los parámetros del modelo. Esto se logra minimizando una función de pérdida  $L(\theta)$ , que mide la distancia entre las predicciones del modelo y las salidas reales en el conjunto de entrenamiento. Matemáticamente, el proceso de entrenamiento busca encontrar el conjunto de parámetros óptimo  $\theta^*$  que minimiza  $L(\theta)$ , es decir:

$$\theta^* = \arg_{\theta} \min L(\theta)$$

3. **Validación y ajuste del modelo (Evaluación del rendimiento):** Después del entrenamiento, se evalúa el rendimiento del modelo utilizando datos de validación. Esto puede implicar calcular métricas de evaluación como la precisión, el “recall” o el “F1-score”, entre otras. Si el rendimiento no es satisfactorio, se pueden realizar ajustes en el modelo, como la selección de diferentes algoritmos o la optimización de parámetros ( $\theta$ ).
4. **Implementación del modelo en producción:** Una vez que el modelo ha sido validado y ajustado según sea necesario, se implementa en un entorno de producción. Esto implica integrar el modelo en sistemas existentes y establecer mecanismos para su despliegue y monitorización continua.

En la Figura 2.4 se puede ver el diagrama de bloques de los modelos de aprendizaje automático supervisado. El bloque de la izquierda (marcado con el 1) se corresponde con el apartado de preprocesamiento. Las siglas EDA significan análisis exploratorio de los datos. El bloque de la derecha (marcado con el 2 y el 3) se corresponde con los apartados de entrenamiento y validación (o evaluación). La implementación en producción no se muestra en la figura.

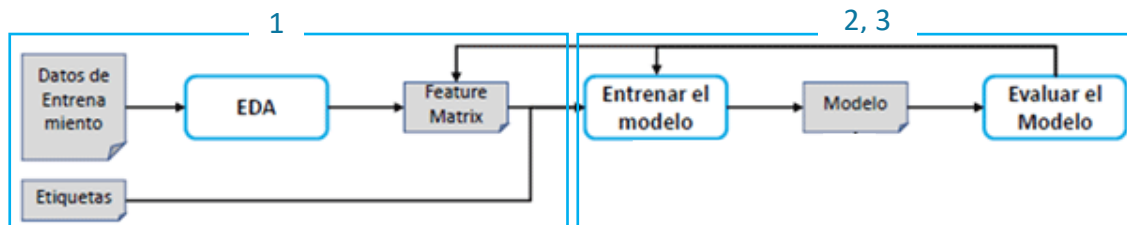


Figura 2. 4. Diagrama de bloques de aprendizaje automático supervisado, adaptado de [12].

Un modelo de aprendizaje automático, una vez implementado, puede completar una tarea mucho más rápido que los humanos, entrega resultados consistentes de manera confiable y se puede copiar infinitas veces. Replicar un modelo de aprendizaje automático en otra máquina es rápido y barato. El entrenamiento de un humano para una tarea puede llevar décadas y es muy caro [6].

## 2.4. Métodos Tradicionales de Aprendizaje Supervisado

Existen muchos tipos de algoritmos tradicionales de aprendizaje automático supervisado. Algunos de los principales son [13]:

- **Regresión lineal.** La regresión lineal es un método estadístico utilizado para modelar la relación entre una o más variables independientes  $X$  y una variable dependiente  $y$  mediante una función lineal. En el contexto del aprendizaje automático supervisado, el objetivo de la regresión lineal es aprender los parámetros de esta función lineal para predecir valores continuos de  $y$  a partir de  $X$ . Matemáticamente, la regresión lineal puede expresarse como:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Donde:

- $y$  es la variable dependiente que se desea predecir.
- $x_1, x_2, \dots, x_n$  son las variables independientes.
- $\beta_0$  es el término independiente del modelo de regresión.
- $\beta_1, \dots, \beta_n$  son los coeficientes que representan la pendiente de la relación entre  $y$  y las variables independientes.
- $\varepsilon$  es el término de error (residuo), que representa la diferencia entre las predicciones del modelo y los valores reales de  $y$ .

La regresión lineal asume que la relación entre las variables es lineal y que los errores siguen una distribución normal. Típicamente se utilizan técnicas de optimización para ajustar los coeficientes del modelo a los datos de entrenamiento, de manera que minimicen la suma de los cuadrados de los residuos [14].

En la Figura 2.5 se puede ver un ejemplo de regresión lineal. La recta roja es la función resultante de ajustar los parámetros de la regresión lineal a los datos del problema (puntos azules). Las predicciones de esta regresión se corresponderán con el valor de  $y$  de la recta, teniendo unos valores  $X$  de entrada. La regresión lineal tiene muchas aplicaciones prácticas en distintas áreas, como por ejemplo en la astronomía [15] o la economía [16], ya que son fácilmente interpretables.

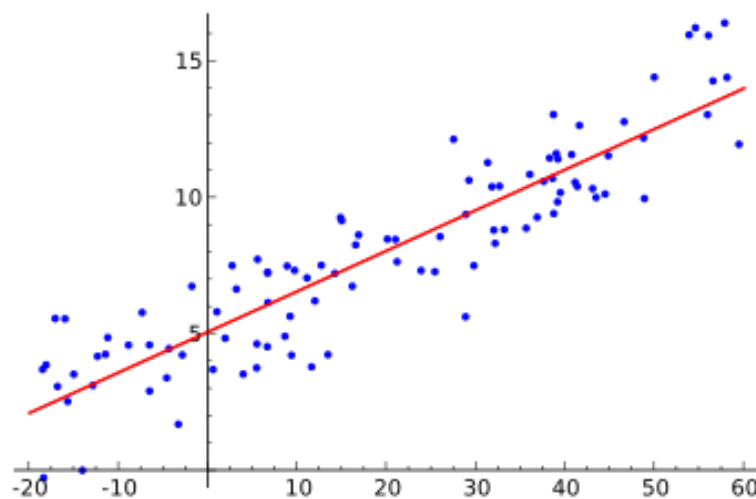


Figura 2. 5. Ejemplo de regresión lineal [17].

- **Árboles de decisión.** Los árboles de decisión son modelos de aprendizaje automático que se utilizan en problemas de clasificación y regresión. En un árbol de decisión, el espacio de características se divide recursivamente en regiones más pequeñas basadas en valores de características, con el objetivo de predecir la variable objetivo. Cada nodo interno del árbol representa una característica, cada rama representa una posible partición de esa característica, y cada hoja representa la predicción para una observación determinada. Los árboles de decisión son capaces de capturar relaciones no lineales entre las características y la variable objetivo, y su estructura basada en reglas es fácilmente interpretable. En la Figura 2.6 podemos ver cómo es la estructura básica de un árbol de decisión.

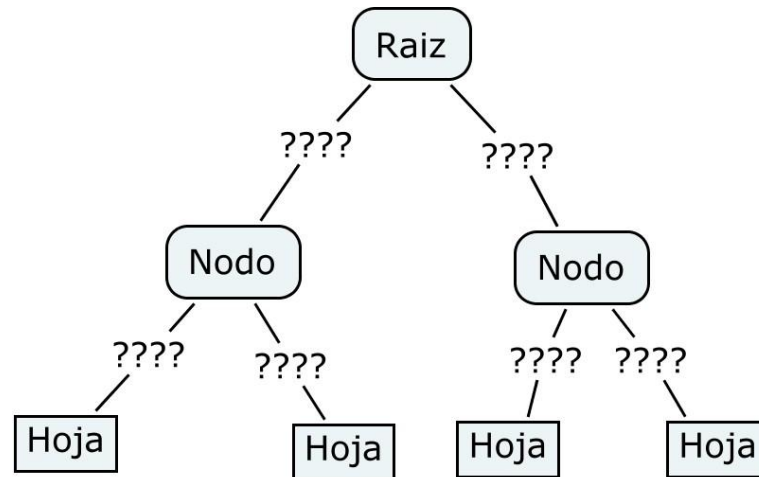


Figura 2. 6. Estructura de los árboles de decisión [18].

“**Random Forest**” se basa en la idea de los árboles de decisión, pero lleva esta técnica un paso más allá mediante la construcción de un conjunto de árboles de decisión. En lugar de depender únicamente de la predicción de un solo árbol, “Random Forest” combina las predicciones de múltiples árboles de decisión entrenándolos con subconjuntos aleatorios de los datos y características para luego promediar sus predicciones. Esta estrategia de conjunto ayuda a mejorar la precisión predictiva y la generalización del modelo. En la Figura 2.7 se puede ver la estructura del “random forest”.

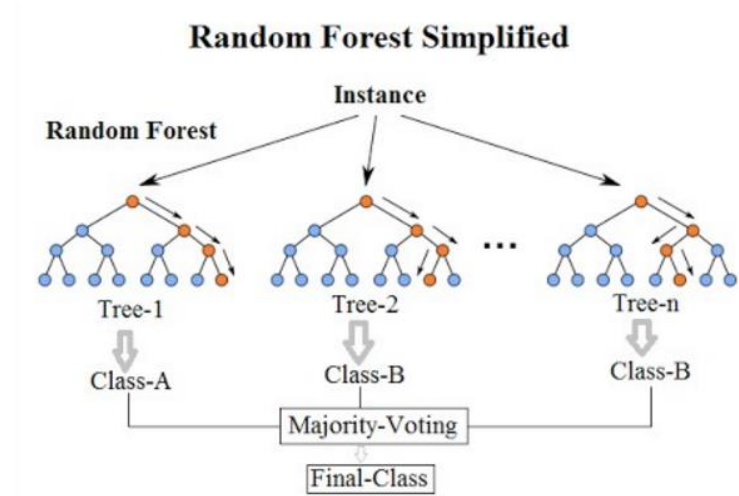


Figura 2. 7. Estructura del “Random Forest” [19].

Los árboles de decisión tienen diversas aplicaciones prácticas, como por ejemplo la medicina [20] o el “marketing” [21].

- **Máquinas de vectores soporte (SVM).** Son algoritmos que buscan encontrar el hiperplano que mejor separa las clases en un espacio dimensional más alto. Pueden ser utilizadas tanto para clasificación como para regresión. En la Figura 2.8 vemos como la SVM genera un hiperplano en el espacio de características mediante una transformación no lineal ( $\phi$ ) que permite discriminar los datos en dos conjuntos distintos a través una frontera no lineal en el espacio de entrada.

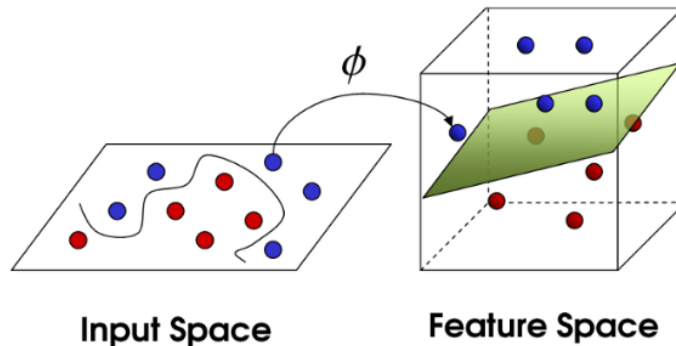


Figura 2. 8. Ejemplo gráfico de SVM [22].

- **K-Vecinos más Cercanos (K-NN).** K-NN clasifica un punto de datos basándose en la clase mayoritaria de sus k vecinos más cercanos en el espacio de características. La clase del punto se determina por votación. En la Figura 2.9 se puede ver un ejemplo de cómo funciona el algoritmo. Se distinguen 3 clases en el conjunto de datos (A, B y C), y para determinar la clase del punto P se observan las clases de sus vecinos más cercanos. Si se usan los  $k = 7$  vecinos más cercanos, el punto P se clasificará como correspondiente a la Clase B, ya que es la clase de 3 de estos 7 vecinos. Un ejemplo de aplicación de este algoritmo es en la medicina [23].

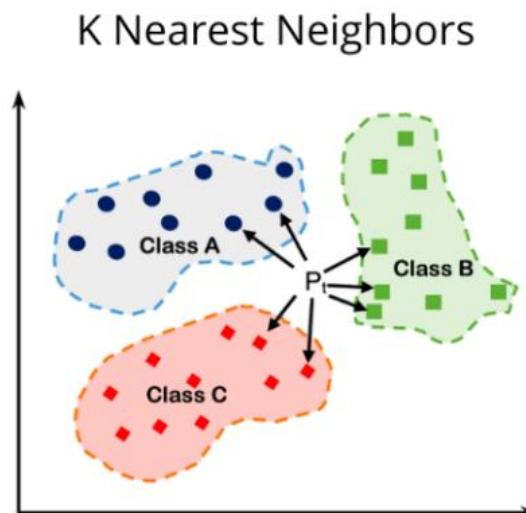


Figura 2. 9. Ejemplo de KNN [29].

- **“Naive Bayes”**. Se basa en el teorema de Bayes, asumiendo que las características son independientes entre sí. Calcula la probabilidad de que una instancia pertenezca a una clase en función de la probabilidad condicional de cada característica dada su clase.

Dado un conjunto de datos de entrenamiento  $D=\{(x_1,y_1),(x_2,y_2),\dots,(x_N,y_N)\}$ , donde  $x_i$  representa un vector de características e  $y_i$  representa la clase a la que pertenece  $x_i$ , “Naive Bayes” calcula la probabilidad de que la clase sea  $C_k$  dado un conjunto de características  $x$  utilizando el Teorema de Bayes:

$$P(C_k|x) = \frac{P(x|C_k) \times P(C_k)}{P(x)}$$

Donde:

- $P(C_k|x)$  es la probabilidad *a posteriori* de que la clase sea  $C_k$  dado un conjunto de características  $x$ .
- $P(x|C_k)$  es la probabilidad condicional de las características  $x$  dadas la clase  $C_k$ . “Naive Bayes” asume que estas características son independientes entre sí, por lo que esta probabilidad se puede calcular como el producto de las probabilidades condicionales individuales de cada característica.
- $P(C_k)$  es la probabilidad *a priori* de la clase  $C_k$ , que se estima a partir de la frecuencia de las clases en el conjunto de entrenamiento.
- $P(x)$  es la probabilidad marginal de las características  $x$ , que se puede calcular como la suma de  $P(x|C_k) \times P(C_k)$  sobre todas las clases  $C_k$  [25].

## 2.5. ¿Qué es el Aprendizaje Profundo?

El “deep learning”, o aprendizaje profundo, es una subdisciplina del aprendizaje automático que se enfoca en el entrenamiento de algoritmos llamados redes neuronales artificiales para realizar tareas complejas de reconocimiento de patrones y toma de decisiones. Lo que distingue al aprendizaje profundo es la arquitectura de estas redes neuronales, que están compuestas por múltiples capas de nodos interconectados, imitando la estructura del córtex cerebral [26].

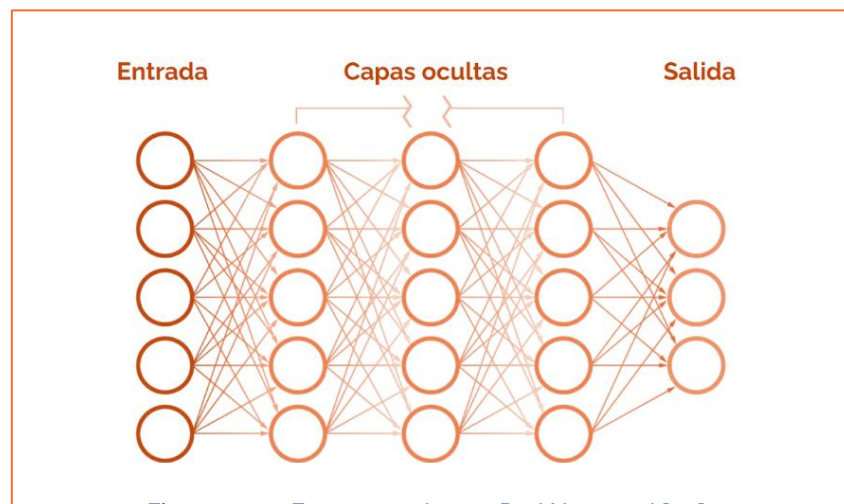


Figura 2. 10. Estructura de una Red Neuronal [27].

En las redes neuronales se distinguen distintos tipos de capas, como se puede ver representado en la Figura 2.10:

- **Capa de Entrada (“Input Layer”).** Esta capa recibe los datos de entrada y transmite estos datos a las capas ocultas para su procesamiento. El número de neuronas en esta capa coincide con la dimensionalidad de los datos de entrada.
- **Capas Ocultas (“Hidden Layers”).** Las capas ocultas realizan transformaciones no lineales de los datos de entrada mediante el cálculo de combinaciones lineales y la aplicación de funciones de activación. Estas capas son responsables de extraer características relevantes de los datos.
- **Capa de Salida (“Output Layer”).** La capa de salida produce las predicciones o resultados finales de la red neuronal. El número de neuronas en esta capa depende del tipo de problema que se esté abordando (clasificación, regresión, etc.). La función de activación utilizada en esta capa varía según el tipo de problema (por ejemplo, sigmoide para clasificación binaria o “softmax” para clasificación multiclase).

A diferencia de los métodos tradicionales de aprendizaje automático, donde se utilizan características específicas diseñadas por humanos, el aprendizaje profundo es capaz de aprender automáticamente características relevantes de los datos a partir de las instancias de entrada. Esto hace que sea particularmente efectivo en problemas donde los datos son complejos y de alta dimensionalidad, como el procesamiento de imágenes, el reconocimiento de voz, la traducción automática y la conducción autónoma.

Los métodos de aprendizaje profundo funcionan de la siguiente manera:

1. **Inicialización de pesos.** Se comienza inicializando los pesos de la red neuronal de manera aleatoria. Los pesos son parámetros ajustables que determinan la importancia de cada entrada en la predicción final del modelo.
2. **Propagación hacia adelante (“Forward Propagation”).** Durante esta fase, los datos de entrada se propagan a través de la red neuronal, capa por capa, aplicando transformaciones y combinaciones lineales y no lineales a medida que pasan por las diferentes capas.
3. **Cálculo de la pérdida (“Loss Calculation”).** Se calcula la discrepancia entre la salida predicha por la red neuronal y la salida deseada, utilizando una función de pérdida.
4. **Retropropagación del error (“Backpropagation”).** Se propaga el error desde la capa de salida hacia las capas ocultas, ajustando los pesos de la red neuronal mediante el cálculo de gradientes y la aplicación de un algoritmo de optimización para minimizar la función de pérdida.
5. **Actualización de pesos (“Weight Update”).** Se actualizan los pesos de la red neuronal utilizando el algoritmo de optimización seleccionado (como el descenso del gradiente estocástico) [26].
6. **Repetición del proceso.** Estos pasos se repiten iterativamente hasta que el modelo converge a una solución adecuada o se alcanza un criterio de finalización predefinido.

El aprendizaje profundo ha demostrado ser muy exitoso en una gran variedad de aplicaciones y ha impulsado avances significativos en diversos campos. Algunas de estas aplicaciones son el Reconocimiento de voz de Microsoft (MAVIS) o el buscador de imágenes de Google [28].

## 2.6. Métodos de Aprendizaje Profundo

Estos son algunos de los modelos de aprendizaje profundo que existen:

- **Redes neuronales artificiales (ANN).** Las redes neuronales artificiales son modelos inspirados en la estructura y funcionamiento del cerebro humano. Están formadas por múltiples capas de nodos interconectados y son capaces de aprender representaciones complejas de datos. Se trata de una red genérica que actualmente no suele utilizarse. A partir de esta surgen otras redes neuronales, cada una diseñada para tareas más específicas.
- **Redes neuronales recurrentes (RNN).** Las RNN son un tipo especializado de redes neuronales que tienen conexiones de retroalimentación, lo que les permite procesar secuencias de datos. Son especialmente útiles para datos secuenciales o temporales. En la Figura 2.11 se puede ver la estructura básica de estas redes. Se puede apreciar como cada dato de entrada depende del instante anterior. Se aplican para traducción automática [30] o pronóstico del tiempo [31], por ejemplo.

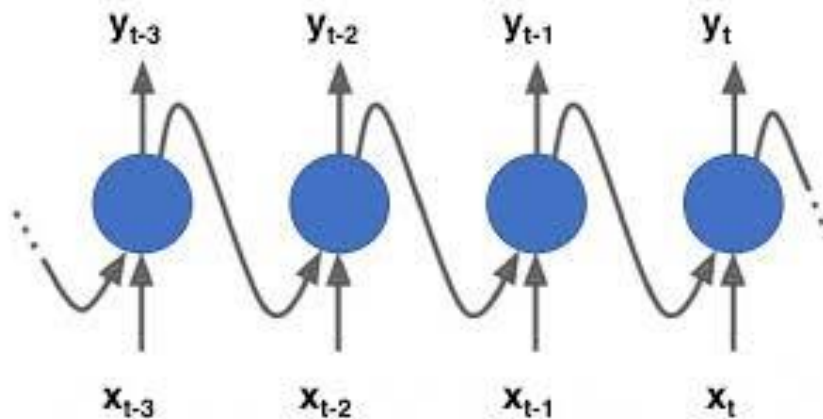


Figura 2. 11. Estructura de una RNN [24].

- **Redes neuronales convolucionales (CNN).** Las CNN son otro tipo de redes neuronales diseñadas específicamente para procesar datos estructurados en forma de matrices, como imágenes. Utilizan capas de convolución para detectar patrones locales en los datos. En la Figura 2.12 se aprecia la estructura de este tipo de redes neuronales. Algunas aplicaciones son la detección de objetos en coches inteligentes [32] o el reconocimiento facial [33].

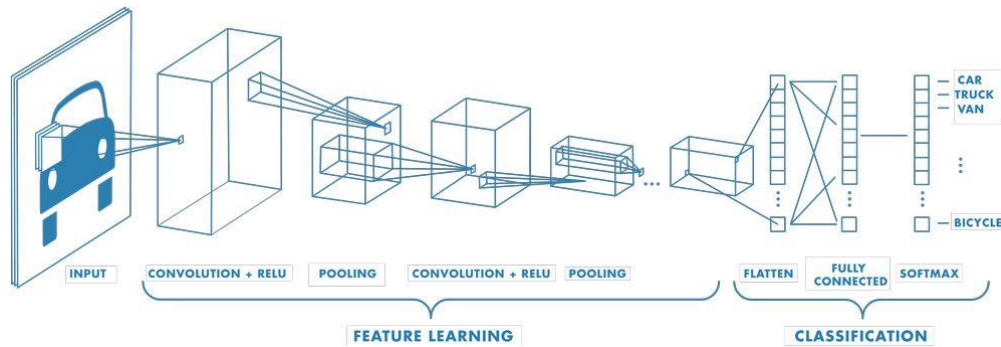


Figura 2. 12. Estructura de una CNN [34].

Dos de las redes neuronales convolucionales más famosas son:

- **LeNet:** Desarrollada por Yann LeCun [35] en los años 90, fue una de las primeras redes neuronales convolucionales exitosas. Fue diseñada para el reconocimiento de caracteres escritos a mano en el conjunto de datos MNIST [36]. Su arquitectura incluye capas convolucionales para detectar características locales, capas de submuestreo (“pooling”) para reducir la dimensionalidad y capas completamente conectadas para la clasificación final.
- **AlexNet:** Desarrollada por Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton [35] en 2012, revolucionó el campo al ganar la competencia “ImageNet Large Scale Visual Recognition Challenge” (ILSVRC). Su arquitectura consta de cinco capas convolucionales profundas. AlexNet mostró el potencial de las CNN con grandes conjuntos de datos, popularizando su uso en diversas aplicaciones de visión artificial.
- **“Generative Adversarial Networks” (GANs).** Las GANs son una arquitectura de redes neuronales compuesta por dos redes en competencia: una generadora y una discriminadora. La red generadora crea datos falsos similares a los reales, mientras que la discriminadora intenta distinguir entre datos reales y generados. A través de este proceso adversarial, ambas redes mejoran continuamente [37]. En la Figura 2.13 se puede ver la arquitectura de una GAN.

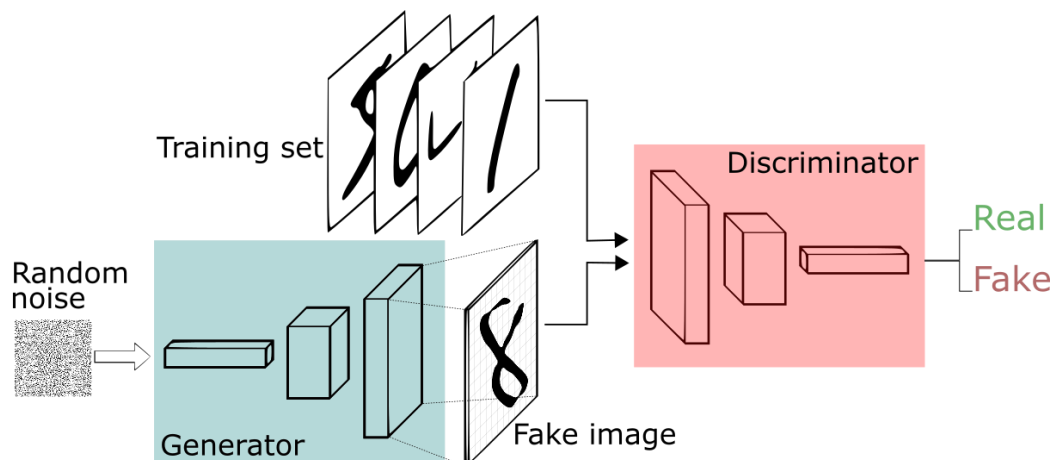


Figura 2. 13. Arquitectura de una GAN [38].

- **Autocodificadores.** Los “autoencoders” (AEs), o autocodificadores, son un tipo de red neuronal utilizada para aprender representaciones eficientes de los datos, típicamente para la reducción de dimensionalidad o la compresión de datos. Consisten en dos partes: un codificador y un decodificador. El codificador transforma los datos de entrada en una representación de menor dimensión, conocida como “espacio latente”, mientras que el decodificador reconstruye los datos originales a partir de este espacio latente. El objetivo es que la salida reconstruida sea lo más similar posible a la entrada original. Los autocodificadores se entrenan mediante un proceso de aprendizaje no supervisado y se utilizan en diversas aplicaciones, como la eliminación de ruido en imágenes, la detección de anomalías y la generación de datos sintéticos [39]. En la Figura 2.14 se ve representada la arquitectura de un autocodificador.

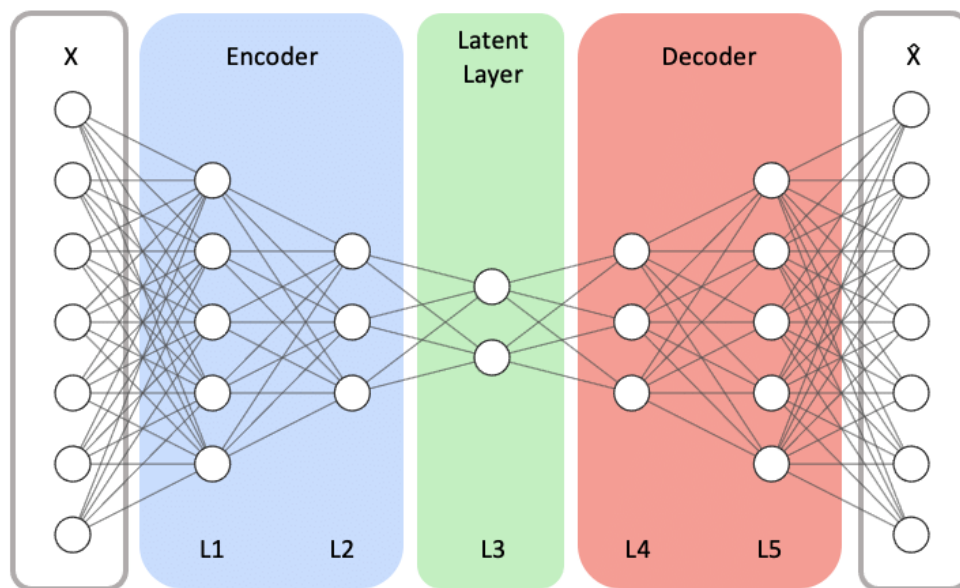


Figura 2. 14. Arquitectura de un autocodificador [40].

- **Transformadores.** Los “Transformers”, o transformadores, son una arquitectura de red neuronal diseñada para procesar secuencias de datos, como texto o series temporales. A diferencia de las redes neuronales convolucionales (CNN) y las redes neuronales recurrentes (RNN), los transformadores no dependen de la estructura secuencial de los datos y son capaces de capturar relaciones de largo alcance entre elementos de la secuencia. En lugar de procesar los datos secuencialmente, los transformadores pueden analizar todas las posiciones de la secuencia simultáneamente, lo que les permite capturar relaciones entre elementos distantes. Sin embargo, este enfoque también tiene un mayor coste computacional. Una de las aplicaciones más extendidas es la generación de texto [41].

## 2.7. Aprendizaje Automático vs. Aprendizaje Profundo

Tanto el “machine learning” (ML) como el “deep learning” (DL) son ramas de la inteligencia artificial (IA), siendo el “deep learning” una especialización dentro del “machine learning”. El ML utiliza algoritmos para analizar datos, aprender de ellos y tomar decisiones o realizar

predicciones basadas en la información. Por otro lado, el DL es un conjunto de algoritmos de ML que modelan abstracciones de alto nivel en datos utilizando arquitecturas compuestas de múltiples transformaciones no lineales. Para referirse al ML excluyendo al DL se va a utilizar el término “Shallow Learning” (SL) o aprendizaje superficial.

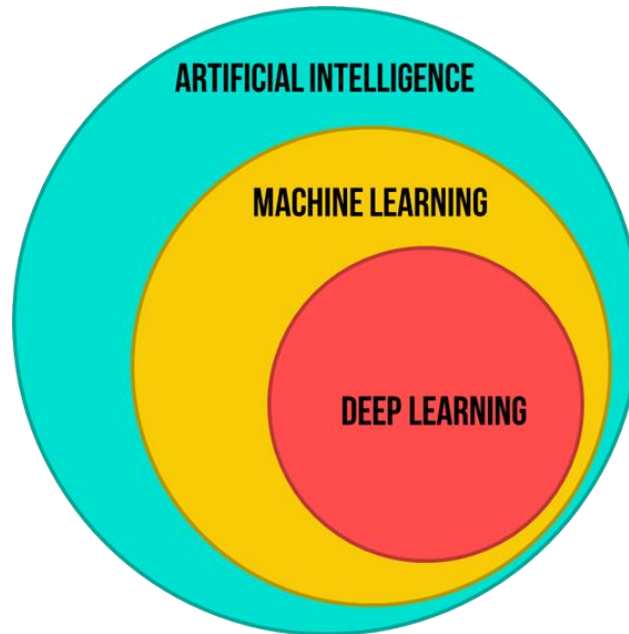


Figura 2. 15. ML, DL e IA [42].

Los modelos de SL no requieren tanta cantidad de datos como los de DL, que necesitan grandes conjuntos de datos para entrenarse eficazmente y son más adecuados para tareas que involucran el procesamiento de grandes cantidades de datos complejos, como las imágenes y el lenguaje natural. Además, los modelos de DL requieren una mayor potencia computacional y, a menudo, hardware especializado para manejar la complejidad de sus cálculos.

Mientras que el SL puede ser más accesible y rápido de implementar para problemas más simples y donde la extracción de características puede ser realizada por expertos humanos, el DL es preferible para problemas más complejos que requieren la identificación automática de características intrincadas, aunque esto conlleva un tiempo de entrenamiento más prolongado y una menor explicabilidad (véase el Capítulo 3) de los resultados.

La explicabilidad de los algoritmos de SL suele ser más directa, especialmente en modelos como los árboles de decisión, donde las decisiones se pueden rastrear a través de un camino claro de características y umbrales. Sin embargo, los modelos de DL, como las redes neuronales profundas, son a menudo considerados como “cajas negras” debido a su complejidad y la gran cantidad de parámetros involucrados, lo que hace que la interpretación de sus decisiones sea más complicada.

El SL es adecuado para problemas donde la extracción manual de características es factible, mientras que el DL destaca en escenarios donde se pueden aprender características automáticamente de grandes cantidades de datos complejos, como imágenes y texto. A pesar de su complejidad y menor explicabilidad, el DL ha logrado avances significativos en campos como el reconocimiento de voz y la visión por

computadora, superando a menudo el rendimiento de los métodos tradicionales de ML [43].

## **2.8. Modelos de Caja Blanca y Modelos de Caja Negra: Rendimiento vs. Explicabilidad**

Los modelos de caja blanca, o “White Box Models”, son aquellos cuyo proceso de toma de decisiones es fácilmente comprensible y explicable mediante reglas lógicas, fórmulas matemáticas o relaciones causales claras. Los modelos de caja blanca son, entre otros, los modelos de “Shallow Learning” mencionados en la Sección 2.4.

Los modelos de caja negra, o “Black Box Models”, son aquellos cuyo proceso de toma de decisiones no es transparente y no puede ser fácilmente explicado por reglas lógicas o relaciones causales directas. Estos modelos tienden a ser más complejos y se basan en algoritmos sofisticados que pueden capturar patrones complejos en los datos. Los modelos de aprendizaje profundo (DL) mencionados en la Sección 2.6 son modelos de caja negra.

Los modelos de caja blanca y caja negra varían considerablemente en cuanto a su rendimiento y explicabilidad. Los modelos de caja blanca, tales como árboles de decisión o regresiones lineales, se caracterizan por su alta interpretabilidad y transparencia. Estos modelos pueden explicar las decisiones tomadas a través de reglas lógicas claras o ecuaciones matemáticas simples. En contraste, los modelos de caja negra, como las redes neuronales profundas, son menos transparentes en su proceso de toma de decisiones. Estos últimos utilizan algoritmos complejos para capturar patrones en los datos, lo que puede dificultar la comprensión de cómo se llega a una predicción. Aunque estos modelos pueden ofrecer alta precisión en conjuntos de datos complejos, su falta de explicabilidad puede limitar su utilidad en situaciones donde se requiere una comprensión clara de las decisiones del modelo.

La elección entre modelos de caja blanca y caja negra para las distintas aplicaciones depende del contexto particular de la aplicación. Por ejemplo, en un entorno industrial donde se busca comprender en detalle las variables que afectan a la degradación de equipos, los modelos de caja blanca, como regresiones lineales, permiten una explicación clara del proceso de toma de decisiones, facilitando decisiones informadas sobre mantenimiento o reemplazo. Por otro lado, si se prioriza la precisión predictiva sin necesidad de entender completamente las decisiones del modelo, los modelos de caja negra, como redes neuronales profundas, pueden ser más adecuados, especialmente cuando se dispone de muchos datos. La selección del tipo de modelo debe basarse en las necesidades específicas del proyecto, ya sea comprender detalladamente el proceso de toma de decisiones o maximizar la precisión predictiva [44, 45].

## **2.9. Discusión**

En este capítulo se han abordado los conceptos fundamentales y metodologías esenciales del aprendizaje automático. El capítulo comienza presentando el aprendizaje automático y profundo como subdisciplinas de la inteligencia artificial (IA). Se destaca que el aprendizaje automático se centra en métodos que permiten a las máquinas realizar predicciones o mejorar comportamientos basados en datos, mientras que el aprendizaje

profundo se enfoca en entrenar redes neuronales para tareas complejas de reconocimiento de patrones. Se discute la diferencia entre los métodos tradicionales de aprendizaje automático, como la regresión lineal y los árboles de decisión, y el aprendizaje profundo. Se resalta que el aprendizaje profundo es capaz de aprender automáticamente características relevantes de los datos, lo que lo hace efectivo en problemas con datos de alta dimensionalidad.

Posteriormente, se introduce la distinción entre modelos de caja blanca y caja negra, explicando cómo difieren en términos de transparencia y explicabilidad. Se discute cómo modelos como los árboles de decisión son intrínsecamente interpretables, mientras que modelos como las redes neuronales profundas son considerados cajas negras.



## Capítulo 3: Explicabilidad

### 3.1. Introducción

En este capítulo se abordará el tema de la explicabilidad, resaltando la importancia de comprender cómo los modelos de inteligencia artificial toman decisiones y generan predicciones. La capacidad de explicar y entender el comportamiento de los modelos es esencial para generar confianza en sus resultados, detectar errores y sesgos, y revelar patrones y relaciones ocultas en los datos.

Se explorará la taxonomía y características de la explicabilidad, así como los distintos tipos de explicabilidad y explicaciones que existen. Además, se discutirá la adaptación de estas explicaciones para diferentes audiencias, asegurando que cada grupo reciba la información necesaria de manera comprensible y útil.

### 3.2. ¿Qué es la explicabilidad?

La explicabilidad se refiere a la capacidad de entender y explicar cómo funciona un modelo y cómo llega a sus predicciones. Es esencial para que los usuarios, que pueden no ser expertos en inteligencia artificial o aprendizaje automático, comprendan la lógica y el proceso detrás de las decisiones del modelo. En este sentido, la explicabilidad busca desglosar el comportamiento del modelo y proporcionar una visión clara de cómo se llega a una determinada estimación [46].

La inteligencia artificial explicable, o “explainable artificial intelligence” (XAI), tiene varias aplicaciones dependiendo del grado de madurez del modelo [3]:

- **Detectar errores en el modelo:** La XAI permite identificar y comprender los errores o sesgos en los modelos de inteligencia artificial. Al analizar las decisiones tomadas por el modelo, los investigadores y desarrolladores pueden corregir problemas y mejorar la precisión. Uno de los experimentos más famosos para demostrar la importancia de la explicabilidad para detectar errores en el modelo es el clasificador de huskies y lobos, cuyos resultados se muestran en la Figura 3.1 [47]. El objetivo era construir un modelo de IA capaz de distinguir entre perros husky y lobos. Los investigadores introdujeron un sesgo a propósito: todas las fotos de lobos utilizadas para entrenar el modelo se mostraban en un entorno nevado. En la Figura 3.1 se puede ver cuáles son los píxeles que el algoritmo consideraba más importante a la hora de clasificar una imagen. Lo que en realidad había aprendido el modelo es que, si hay nieve en la imagen, entonces tiene que ser un lobo. El modelo para distinguir huskies de lobos se había convertido, básicamente, en un detector de nieve. Este fallo sería imposible de detectar si el algoritmo se comporta como una caja negra [47].

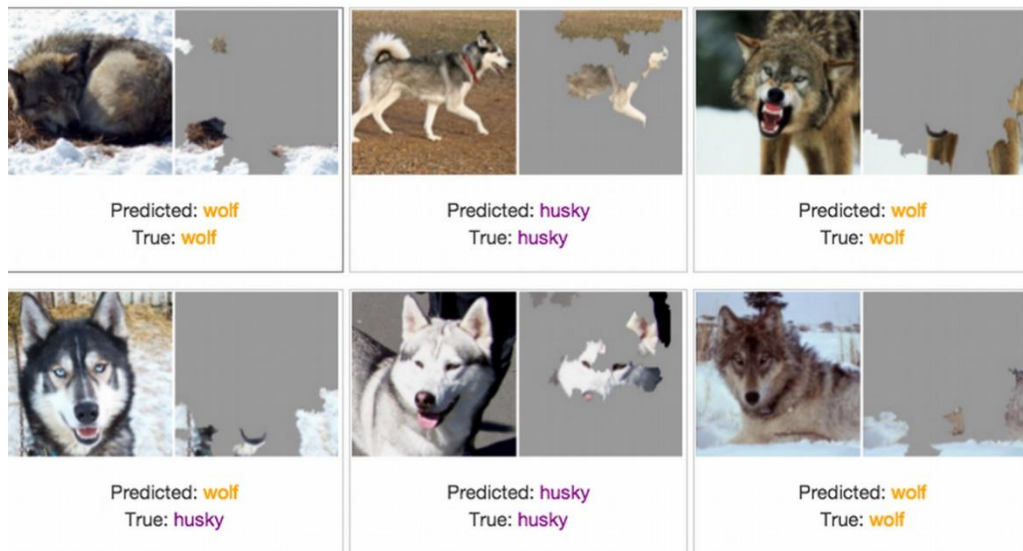


Figura 3. 1. Clasificador de huskys y lobos [48].

- **Aumentar la confianza:** La transparencia es fundamental para generar confianza en los sistemas de IA. La XAI proporciona explicaciones claras sobre cómo se toman las decisiones, lo que ayuda a los usuarios a comprender y confiar en los resultados. Por ejemplo, en aplicaciones médicas la XAI puede explicar por qué un modelo sugiere un diagnóstico específico, lo que permite a los médicos tomar decisiones informadas.
- **Generar nuevo conocimiento:** La XAI no solo explica las decisiones existentes, sino que también puede revelar patrones y relaciones ocultas en los datos. Un ejemplo muy famoso de la capacidad de generar nuevo conocimiento es AlphaZero. Se trata de una IA que aprendió a jugar al ajedrez solamente jugando contra sí misma. Esto le permitió adquirir conocimientos y estrategias que los jugadores de ajedrez no conocían. AlphaZero ganó a los mejores ajedrecistas del mundo en 2017 [49].

Las explicaciones pueden representarse de diferentes maneras para hacer comprensibles los modelos de aprendizaje automático. Algunos ejemplos son [6, 50]:

- **Explicaciones visuales:** Estas explicaciones utilizan representaciones gráficas, como mapas de calor, diagramas de árbol, etc., para visualizar cómo el modelo toma decisiones y qué características son más influyentes en esas decisiones. En la Figura 3.2 se muestra un ejemplo de explicaciones visuales usando mapas de calor.

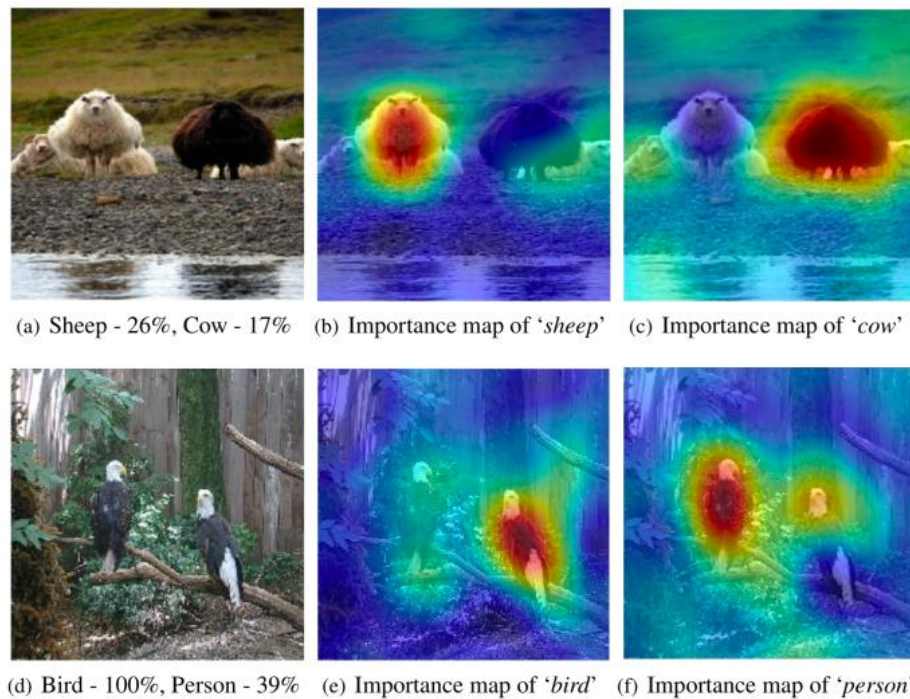


Figura 3. 2. Ejemplo de explicaciones visuales [51].

- **Explicaciones textuales:** Se presentan en forma de texto para describir en palabras simples y comprensibles cómo funciona el modelo y qué factores influyen en sus predicciones. Esto puede incluir la explicación de características importantes, reglas de decisión, etc.
- **Contraejemplos:** Los contraejemplos son instancias de datos que muestran cómo el modelo podría haber llegado a una predicción diferente. Ayudan a entender por qué el modelo cometió ciertos errores o tomó ciertas decisiones.
- **Explicaciones globales y locales:** Las explicaciones globales aclaran el comportamiento general del modelo en todo el conjunto de datos, mientras que las explicaciones locales se centran en explicar una predicción específica para una instancia de datos individual.
- **Valores de influencia de características:** Estos valores asignan una contribución a cada característica en una predicción específica, mostrando cómo cada característica afecta la salida del modelo.
- **Perturbaciones de características:** Se basan en alterar deliberadamente ciertas características y observar cómo afecta a la predicción del modelo. Puede proporcionar información sobre qué características son más importantes para el modelo.

También podemos distinguir distintos niveles de explicabilidad en función de la audiencia del modelo [39]:

- **Explicaciones para desarrolladores:** El desarrollador es el que hace el modelo. Por lo general, los desarrolladores buscan conocimientos técnicos que puedan explicar todo el modelo en lugar de explicaciones de una predicción. Por este

motivo los desarrolladores generalmente están más interesados en las explicaciones globales que en las explicaciones locales.

- **Explicaciones para los usuarios finales:** Un usuario final es una persona que consume la salida del modelo. Generalmente, los usuarios buscan explicaciones de una predicción o un grupo de predicciones que pueden afectarles. Normalmente no pueden aprovechar todas las explicaciones de un modelo, porque pueden ser difíciles de interpretar y no necesariamente se ajustan a su situación. Es por eso por lo que los usuarios están más interesados en las explicaciones locales que en las explicaciones globales.
- **Explicaciones para los responsables de la toma de decisiones:** El responsable de la toma de decisiones se caracteriza por una falta de experiencia técnica, además de la responsabilidad en caso de un problema con el sistema de IA. No son expertos en aprendizaje automático, pero necesitan garantías de que el sistema funcionará correctamente. Es posible que no necesiten una explicación de cómo funciona el sistema, pero sí tener una medida de confianza en sus decisiones. De esta manera, podrían saber si es razonable o demasiado arriesgado utilizar el sistema.

### **3.3. Características de la Explicabilidad**

La explicabilidad se basa en varias características fundamentales que son esenciales para garantizar la comprensión y confianza en el modelo [6].

Características de los modelos:

- **Poder expresivo:** Hace referencia a la estructura de las explicaciones que el método es capaz de generar. Un método de explicación puede generar árboles de decisión, una suma ponderada, una descripción en lenguaje natural, etc.
- **Transparencia:** Describe cuánto depende el método de explicación de sus parámetros internos. Por ejemplo, los modelos intrínsecamente interpretables, como el modelo de regresión lineal, son altamente transparentes. Los métodos que solo dependen de la manipulación de entradas y la observación de las predicciones no tienen transparencia.
- **Portabilidad:** Este aspecto describe la variedad de modelos de aprendizaje automático con los que el método de explicación puede ser utilizado. Los métodos con baja transparencia poseen una mayor portabilidad, al tratar al modelo de aprendizaje automático como una caja negra.
- **Complejidad algorítmica:** Describe la complejidad computacional del método utilizado para generar la explicación. Esta característica es crucial, al considerar el tiempo de computación como un factor limitante en la generación de explicaciones.

Características de las explicaciones:

- **Precisión:** Cómo de bien predice una explicación datos no etiquetados. La alta precisión es importante si la explicación se utiliza para predicciones en lugar del modelo de aprendizaje automático. La baja precisión está bien si la precisión del

modelo también es baja y si el objetivo es explicar qué hace el modelo de caja negra. En este caso sólo importa la fidelidad.

- **Fidelidad:** Cómo de bien se aproxima la explicación a la predicción del modelo de caja negra. La alta fidelidad es una de las propiedades más importantes de una explicación. La precisión y la fidelidad están estrechamente relacionadas. Si el modelo de caja negra tiene alta precisión y la explicación tiene alta fidelidad, la explicación también tiene alta precisión.
- **Consistencia:** En qué medida difiere una explicación entre modelos que han sido entrenados en la misma tarea y que producen predicciones similares. Por ejemplo, se entrenan una máquina de vectores de soporte y un modelo de regresión lineal en la misma tarea y ambos producen predicciones muy similares. Se calculan las explicaciones y se analiza cuán diferentes son. Si las explicaciones son muy similares, estas son muy consistentes.
- **Estabilidad:** Cómo de similares son las explicaciones para casos similares. Mientras que la consistencia compara explicaciones entre modelos, la estabilidad compara explicaciones entre instancias similares para un modelo fijo. Una alta estabilidad significa que ligeras variaciones en las características de una instancia no cambian sustancialmente la explicación (a menos que estas ligeras variaciones también cambien fuertemente la predicción). La falta de estabilidad puede ser el resultado de una gran variación del método de explicación.
- **Comprensibilidad:** Cómo de bien entienden los humanos las explicaciones. Difícil de definir y medir, pero extremadamente importante. Para medir la comprensibilidad se suele medir el tamaño de la explicación o probar cómo de bien las personas pueden predecir el comportamiento del modelo de aprendizaje automático a partir de las explicaciones. También se debe considerar la comprensibilidad de las características utilizadas en la explicación.
- **Certeza:** La capacidad de la explicación de reflejar la certeza del modelo de aprendizaje automático. Muchos modelos de aprendizaje automático solo dan predicciones sin dar información sobre la confianza del modelo en que la predicción es correcta. Si el modelo predice una probabilidad del 4% de cáncer para un paciente, ¿es igual de segura esa estimación como la de que otro paciente con otras características tenga también un 4%?
- **Grado de importancia:** Cómo de bien refleja la explicación la importancia de las características. Por ejemplo, si se genera una regla de decisión como explicación para una predicción individual, ¿está clara cuál de las condiciones de la regla fue la más importante?
- **Novedad:** Este punto se refiere a la capacidad de la explicación para discernir si una instancia de datos que se está analizando es significativamente diferente de las instancias de datos utilizadas para entrenar el modelo. Cuando una instancia de datos está en una región "nueva" o muy distante de la distribución de los datos de entrenamiento, puede indicar que el modelo no tiene suficiente información para hacer una predicción precisa. La novedad está vinculada al grado de certeza del modelo: cuanto más "nueva" sea la instancia, más probable es que el modelo tenga menos certeza en su predicción debido a la falta de datos relevantes.

- **Representatividad:** Se refiere a la cantidad de instancias de datos que una explicación abarca. Algunas explicaciones pueden cubrir todo el modelo, lo que significa que proporcionan una comprensión global de cómo funciona el modelo en su conjunto. Por otro lado, algunas explicaciones pueden representar solo una predicción individual, lo que significa que explican específicamente por qué el modelo hizo una determinada predicción para una única instancia de datos.

### 3.4. Taxonomía de la Inteligencia Artificial Explicable

La taxonomía de la XAI es la forma en la que se pueden clasificar las diferentes técnicas según varios criterios. Estas son las tres categorías clave de la taxonomía de XAI [50]:

- **Ante-hoc y Post-hoc:** La distinción entre *ante-hoc* y *post-hoc* se refiere al punto de la cadena de tratamiento de datos en que se implementan las técnicas de explicabilidad. Las técnicas *ante-hoc* se incorporan durante el proceso de diseño y entrenamiento del modelo, con el objetivo de garantizar la explicabilidad desde el principio. Los modelos intrínsecamente interpretables pertenecen a esta categoría. En contraste, las técnicas *post-hoc* se aplican después de que el modelo ha sido entrenado para comprender sus decisiones retrospectivamente. En la Sección 3.5 se entrará más en detalle de esta distinción.
- **Local y Global:** La dimensión local y global se centra en el alcance de la explicabilidad proporcionada por las técnicas. Las técnicas de explicabilidad local se centran en entender las decisiones o predicciones individuales del modelo, dando una visión detallada de cómo se tomaron esas decisiones para una instancia específica. Por otro lado, las técnicas de explicabilidad global buscan comprender el modelo en su conjunto, ofreciendo una visión general de su comportamiento y funcionamiento.
- **Agnóstico y Específico:** La diferenciación entre agnóstico y específico se relaciona con la generalidad de las técnicas de explicabilidad. Las técnicas agnósticas son independientes del tipo de modelo o algoritmo de IA utilizado y se aplican de manera general a una gran variedad de modelos. En contraste, las técnicas específicas están diseñadas para un tipo de modelo o algoritmo en particular, ofreciendo explicaciones más detalladas y precisas adaptadas a las características específicas de ese modelo.

### 3.5. Explicabilidad Ante-hoc y Post-hoc

#### 3.5.1. ¿Qué es la Explicabilidad Ante-hoc?

La explicabilidad *ante-hoc* se refiere a la incorporación de técnicas de explicabilidad durante el proceso de diseño y entrenamiento de un modelo de inteligencia artificial o aprendizaje automático. En lugar de aplicar estas técnicas después de que el modelo esté entrenado, se integran desde el principio para garantizar que el modelo sea interpretable y transparente desde su concepción. El objetivo es comprender cómo el modelo toma decisiones antes de que se realicen predicciones en datos reales.

Al adoptar un enfoque *ante-hoc*, los diseñadores del modelo pueden tomar decisiones informadas sobre qué características utilizar y cómo configurar el modelo. Por ejemplo, al elegir atributos interpretables o limitar la complejidad del modelo. Esto ayuda a evitar “cajas negras” y permite a los expertos comprender las razones detrás de las predicciones del modelo desde el principio.

Estos modelos suelen tener una estructura simple y transparente que permite identificar fácilmente cómo cada característica de entrada influye en la salida del modelo. Ejemplos comunes de modelos de explicación *ante-hoc* son la regresión lineal, los árboles de decisión y las reglas de asociación [6].

Sin embargo, la explicación *ante-hoc* no solo incluye modelos de caja blanca como los anteriormente mencionados. También se pueden aplicar mecanismos *ante-hoc* a modelos de caja negra, como los mecanismos de atención. Estos mecanismos permiten al modelo centrarse en partes específicas de la entrada durante el procesamiento. Los “Transformers” utilizan mecanismos de atención para procesar secuencias de entrada, como texto o imágenes. Durante el entrenamiento, el modelo aprende a asignar pesos a diferentes partes de la entrada. Estos pesos se utilizan para calcular una representación ponderada de la entrada, lo que permite al modelo fijarse en las partes más importantes. Para explicar las decisiones del modelo, podemos visualizar los pesos de atención. Por ejemplo, al traducir una oración de inglés a francés, podemos mostrar qué palabras en inglés contribuyeron más a la predicción de cada palabra en francés [52].

### 3.5.2. ¿Qué es la Explicabilidad Post-hoc?

La explicabilidad *post-hoc* se refiere a los métodos y técnicas utilizados para explicar el comportamiento y las decisiones de modelos después de que han sido entrenados. Estos métodos son aplicables a modelos de caja negra. La explicabilidad *post-hoc* se centra en generar explicaciones comprensibles y claras sobre cómo el modelo ha llegado a una determinada predicción o decisión. Estas explicaciones pueden proporcionar información sobre qué características de entrada fueron más influyentes en la salida del modelo, cómo se relacionan estas características entre sí, y qué patrones o correlaciones fueron detectados por el modelo durante el entrenamiento [53].

Podemos identificar 3 categorías de explicabilidad *post-hoc*:

- **Agnósticos al modelo:** estos métodos se centran en técnicas que pueden aplicarse de manera general a cualquier modelo predictivo. Estos son dos de los más conocidos:
  - LIME (“Local Interpretable Model-agnostic Explanations”): Su objetivo es explicar las predicciones de un modelo de manera local, es decir, para una instancia específica. LIME crea un modelo interpretable (como una regresión lineal) alrededor de la instancia de interés y evalúa cómo las características afectan a la predicción en esa área local. Luego, utiliza esta información para explicar la predicción del modelo original [54].
  - SHAP (“SHapley Additive exPlanations”): SHAP asigna una contribución a cada característica (o atributo) en función de su impacto en la predicción del modelo. Utiliza la teoría de juegos y los valores de Shapley para calcular estas contribuciones. SHAP proporciona una explicación local para cada

instancia de datos, mostrando cómo cada característica afecta a la salida del modelo [55].

- **Específicos del modelo:** métodos explicativos que basan las explicaciones en las propiedades de la arquitectura que pretenden explicar. Son [2]:
  - Mapeo de activación de clase (CAM): CAM crea un mapa de calor que indica las regiones de la imagen utilizadas por una CNN para hacer la predicción. CAM logra esto mediante una capa de agrupación promedio global (GAP) en la arquitectura de la CNN, que calcula el promedio espacial del mapa de características de la última capa. Los píxeles con valores más altos se asocian con los píxeles en la imagen relacionados con la predicción [56].
  - Grad-CAM (“Gradient-weighted Class Activation Mapping”): mejora el proceso de CAM utilizando los gradientes de cualquier clase objetivo que fluyen hacia la última capa convolucional, permitiendo así identificar las áreas más importantes de la imagen para la predicción [57].
  - Grad-CAM++: es una versión avanzada de Grad-CAM que proporciona mapas de calor más precisos al ponderar los gradientes de forma más detallada, permitiendo identificar con mayor exactitud las regiones relevantes, incluso cuando múltiples objetos están presentes en la imagen [58].
  - “LionForests”: Un método de explicación local específicamente diseñado para bosques aleatorios [59]. Sigue estos pasos: estimar el número mínimo de caminos para la respuesta correcta; reducir los caminos mediante reglas de asociación, agrupamiento, selección aleatoria o distribución; extraer los rangos de características; manejar categóricamente las características; componer la interpretación y visualizar los rangos de características. Los resultados del método son interpretaciones en forma de reglas IF-THEN y visualizaciones de las características.
  - Propagación de relevancia por capas (LRP): Un método de explicabilidad específico de redes neuronales que parte del supuesto de que un clasificador puede dividirse en varias capas de cálculo. LRP trabaja con el concepto de una clasificación de relevancia que indica cómo de importante es una característica para una salida. Funciona extrapolando esta relevancia hasta la capa de entrada, retrocediendo a través de la arquitectura desde la salida. La importancia de una característica de entrada se calcula como la suma de la relevancia de las características largo de la arquitectura.
- **Agnósticos y específicos:** Existen métodos que son a la vez o por separado agnósticos y específicos dependiendo del contexto.
  - No combinados: dependiendo del problema son agnósticos o específicos.
    - Contrafactuales: Funcionan proporcionando una explicación para la salida del modelo, ofreciendo explicaciones sobre qué cambios habría que hacer para obtener el resultado deseado. Esta

capacidad hace que los contrafactuales sean únicos en el ámbito de los métodos de XAI.

- **Combinación de métodos:** El uso de múltiples métodos de explicabilidad puede hacerse de manera escalonada o simultánea. La manera escalonada implica usar métodos explicables de forma secuencial. Sin embargo, lo más común es hacerlo de forma simultánea, obteniendo múltiples explicaciones de diferentes métodos [46].

Una de las formas de visualización más populares son los mapas de calor. No son un método por sí mismos, sino que son una forma visual de representar la importancia de las características en un conjunto de datos. Los mapas de calor se utilizan tanto para explicabilidad *ante-hoc* como *post-hoc*. Por ejemplo, en el contexto de SHAP, podemos usar un mapa de calor para mostrar cómo las características afectan a las predicciones [60]. La Figura 2.15 es un ejemplo de mapa de calor.

### 3.5.3. Explicabilidad Ante-hoc vs. Post-hoc

La elección entre modelos de explicabilidad *ante-hoc* y *post-hoc* depende del contexto específico del problema y las necesidades del usuario, ya que cada uno cuenta con sus propias ventajas y limitaciones.

La explicabilidad *ante-hoc*, como los árboles de decisión, es ideal cuando se necesita una comprensión directa de cómo se toman las decisiones. Por ejemplo, en un sistema de recomendación de películas donde se utilizan características simples como género, año de lanzamiento y calificación de usuarios para predecir las preferencias de los usuarios, un árbol de decisión sería fácil de interpretar y proporcionaría explicaciones claras sobre por qué se hacen ciertas recomendaciones [6].

Por otro lado, la explicabilidad *post-hoc* podría ser más útil en casos donde se utilizan modelos de inteligencia artificial más complejos, como redes neuronales profundas. Por ejemplo, en un sistema de diagnóstico médico basado en imágenes de resonancia magnética para detectar enfermedades, una red neuronal convolucional (CNN) podría lograr un alto rendimiento predictivo, pero sería difícil entender cómo llega a sus diagnósticos. En este caso, técnicas de explicabilidad *post-hoc* como la visualización de mapas de activación de características podrían proporcionar información sobre qué características de la imagen son más importantes para el modelo y ayudar a comprender su proceso de toma de decisiones [53].

En la Figura 3.3 se puede ver el diagrama de bloques de los modelos de explicabilidad *ante-hoc* y *post-hoc*. Queda reflejado como la explicabilidad *ante-hoc* da una explicación del modelo durante el entrenamiento de los datos, mientras que los modelos de explicación *post-hoc* siguen siendo cajas negras tras el entrenamiento.

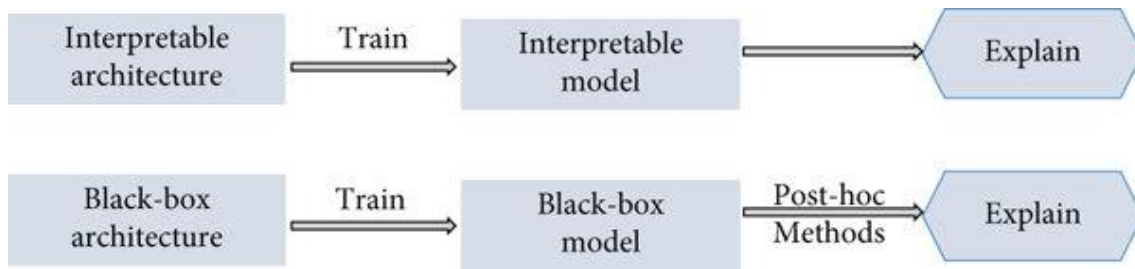


Figura 3. 3. Diagrama de bloques de explicaciones ante-hoc y post-hoc [61].

### 3.6. Discusión

En este capítulo se ha destacado la importancia de la explicabilidad en el contexto del aprendizaje automático, subrayando su papel crucial para que los usuarios comprendan y confíen en los modelos. Se ha definido qué significa la explicabilidad y se ha discutido por qué es fundamental, no solo para los desarrolladores y expertos en IA, sino también para los usuarios finales y responsables de la toma de decisiones.

Se ha explorado la taxonomía de la explicabilidad, categorizando los diferentes tipos de explicaciones, como visuales, textuales, contraejemplos, globales y locales. Además, se han discutido las características esenciales de una buena explicación, incluyendo precisión, fidelidad, estabilidad y comprensibilidad.

Asimismo, se ha comparado la explicabilidad *ante-hoc* y *post-hoc*, destacando sus diferencias y aplicaciones. La explicabilidad *ante-hoc* se integra durante el diseño y entrenamiento del modelo, garantizando transparencia desde el principio, mientras que la explicabilidad *post-hoc* se aplica retrospectivamente, proporcionando información sobre las decisiones del modelo ya entrenado. Esta distinción es crucial para seleccionar la estrategia de explicabilidad más adecuada según el contexto y las necesidades específicas del proyecto.

## Capítulo 4: Mantenimiento Predictivo

### 4.1. Introducción

El mantenimiento predictivo se ha convertido en una herramienta esencial para optimizar la gestión de activos en diversas industrias, permitiendo anticipar y prevenir fallos mediante el uso de tecnologías avanzadas. Este capítulo ofrece una exploración detallada del concepto de mantenimiento predictivo, abordando sus definiciones, técnicas y aplicaciones prácticas. Se analiza cómo el uso de inteligencia artificial y aprendizaje automático facilita la predicción de fallos, adaptándose a las condiciones reales del equipo en lugar de seguir intervalos de tiempo fijos. Además, se describen las principales técnicas de análisis, como el análisis frecuencial y el análisis de series temporales. Al comprender estas técnicas y su implementación, se destaca la importancia del mantenimiento predictivo para mejorar la eficiencia operativa y reducir los costes asociados a fallos imprevistos. También se explorarán los distintos métodos de explicabilidad aplicados al mantenimiento predictivo, dividiéndolos en dos categorías principales: métodos ante-hoc y post-hoc.

### 4.2. ¿Qué es el Mantenimiento Predictivo?

El mantenimiento predictivo, o “predictive maintenance” (PdM), es una colección de técnicas que tienen como objetivo prolongar la vida útil de un sistema, mediante el uso de inteligencia artificial y aprendizaje automático, para predecir el momento óptimo para realizar el mantenimiento. A diferencia del mantenimiento preventivo, que se basa en intervalos de tiempo fijos, el mantenimiento predictivo se adapta a las condiciones reales del equipo [62].

El PdM requiere la recopilación constante de datos del equipo en tiempo real. Esto puede incluir sensores, registros de operación y otros indicadores. El objetivo del PdM es buscar patrones anómalos o desviaciones en los datos recopilados, que pueden indicar posibles problemas o fallos inminentes. Por ello se utilizan algoritmos y modelos matemáticos para predecir el tiempo restante hasta el próximo fallo o el estado de salud del equipo [2].

Una forma de clasificar el mantenimiento predictivo es en función del dominio (temporal o frecuencial) donde se realiza el análisis:

- **Análisis Frecuencial:** Se monitorizan las frecuencias de las señales del equipo para detectar desgaste, desequilibrios o problemas mecánicos. Este enfoque permite identificar anomalías en las frecuencias características de operación, proporcionando información crucial para el mantenimiento predictivo.
- **Análisis de Series Temporales:** Se predicen futuros valores basados en datos históricos. Este análisis permite identificar tendencias y patrones a lo largo del tiempo, facilitando la detección temprana de posibles fallos y la planificación de intervenciones de mantenimiento.

El mantenimiento predictivo se aplica en diversas industrias como:

- **Manufactura:** En líneas de producción, se utiliza para predecir fallos en maquinaria y evitar paradas no planificadas [63].

- **Aeronáutica:** En aviones y helicópteros, se monitorizan componentes críticos para garantizar la seguridad y la disponibilidad [64].
- **Energía:** En plantas de energía, se previenen fallos en turbinas y generadores [65].

Como se puede ver en la Figura 4.1, el mantenimiento predictivo tiene tres objetivos principales: detección de fallos, diagnóstico de fallos y pronóstico de fallos [66].

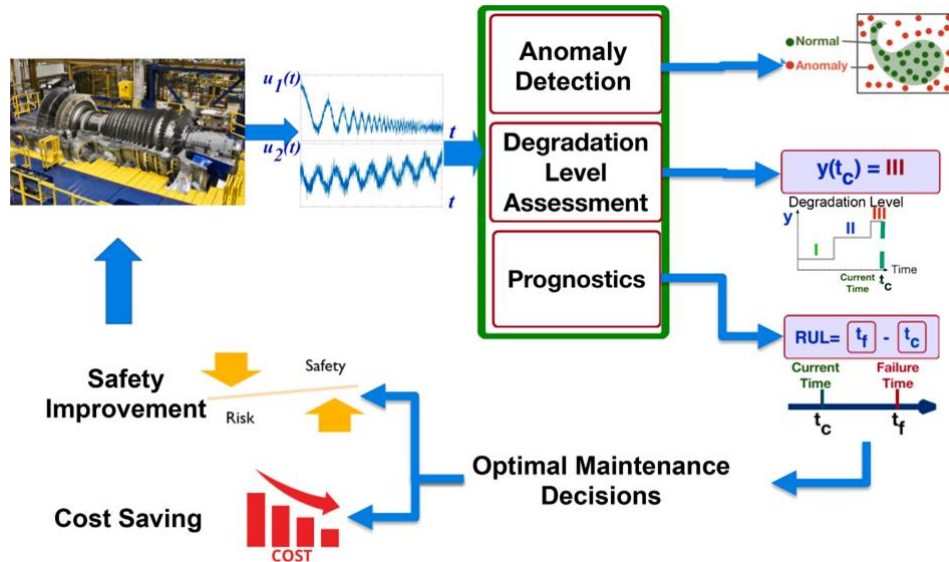


Figura 4. 1. Objetivos principales del Mantenimiento Predictivo [66].

- **Detección de fallos:** Esta tarea de PdM se concentra en identificar anomalías durante la operación de una estructura, sistema o componente (SSC). Estas anomalías, también conocidas como daños, deben ser detectadas a partir de señales de variables físicas medidas durante el funcionamiento del SSC, lo cual se complica debido a diversas fuentes de incertidumbre o ruido. Algunas técnicas de detección de fallos incluyen modelos entrenados con datos de condiciones normales y fallidas, así como la comparación de las señales reales con aquellas reconstruidas bajo condiciones normales.

En la Figura 4.2 se presenta un método basado en “Generative Adversarial Networks” (GAN) para la detección de fallos que aborda el desafío de la falta de datos. La GAN realiza una reconstrucción de la distribución de datos bajo condiciones normales a través del Generador, mientras que un Autocodificador (AE) se encarga de calcular puntuaciones de anomalía (errores de reconstrucción). Estas puntuaciones permiten distinguir si las muestras son anómalas o no [67].

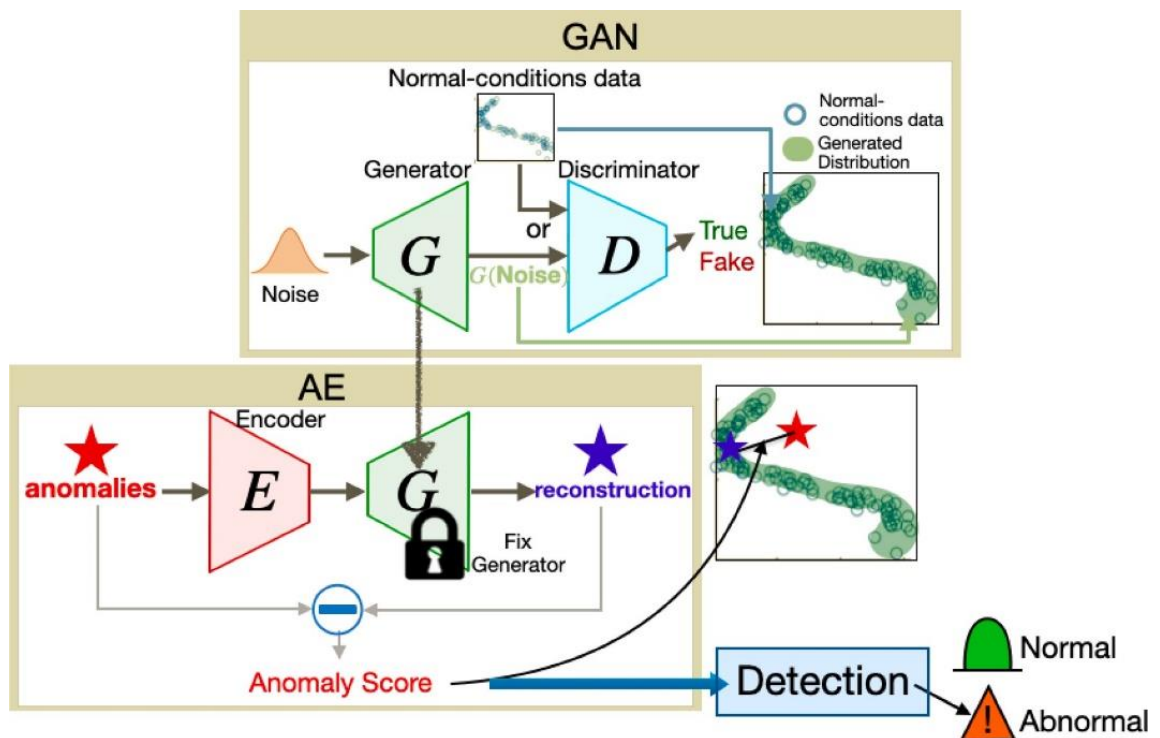


Figura 4. 2. GAN para la detección de fallos [67].

- Diagnóstico de fallos:** Se trata de identificar el estado de los equipos, recopilando datos tales como fallos, modos de fallo y su ubicación en sistemas, a través de señales monitorizadas como temperatura, presión o aceleración. En este proceso, se emplean técnicas de aprendizaje supervisado [68], como redes neuronales, para construir clasificadores capaces de distinguir entre distintas clases de datos y así realizar diagnósticos de fallos. La escasez de datos etiquetados en la práctica impulsa la necesidad de métodos de aprendizaje no supervisado. A su vez, los métodos no supervisados se enfrentan con dificultades para manejar dependencias temporales a largo plazo. En este contexto, se han propuesto enfoques innovadores, como un Autocodificador Categórico Adversarial (CatAAE) [69] dirigido al diagnóstico de fallos de los rodamientos.

En la Figura 4.3 se muestra un método basado en “Reservoir Computing” para el diagnóstico de fallos a pesar de la falta de etiquetas claras del estado de degradación. Utiliza un Generador de Conceptor para convertir señales de longitud variable en matrices de Conceptores que reducen el ruido y las condiciones operativas no relevantes. Luego, emplea una combinación de estas matrices y un agrupamiento espectral para generar pseudoetiquetas en las trayectorias de fallos. Finalmente, entrena una red neuronal convolucional con estas pseudoetiquetas y matrices de Conceptores para el diagnóstico de fallos.

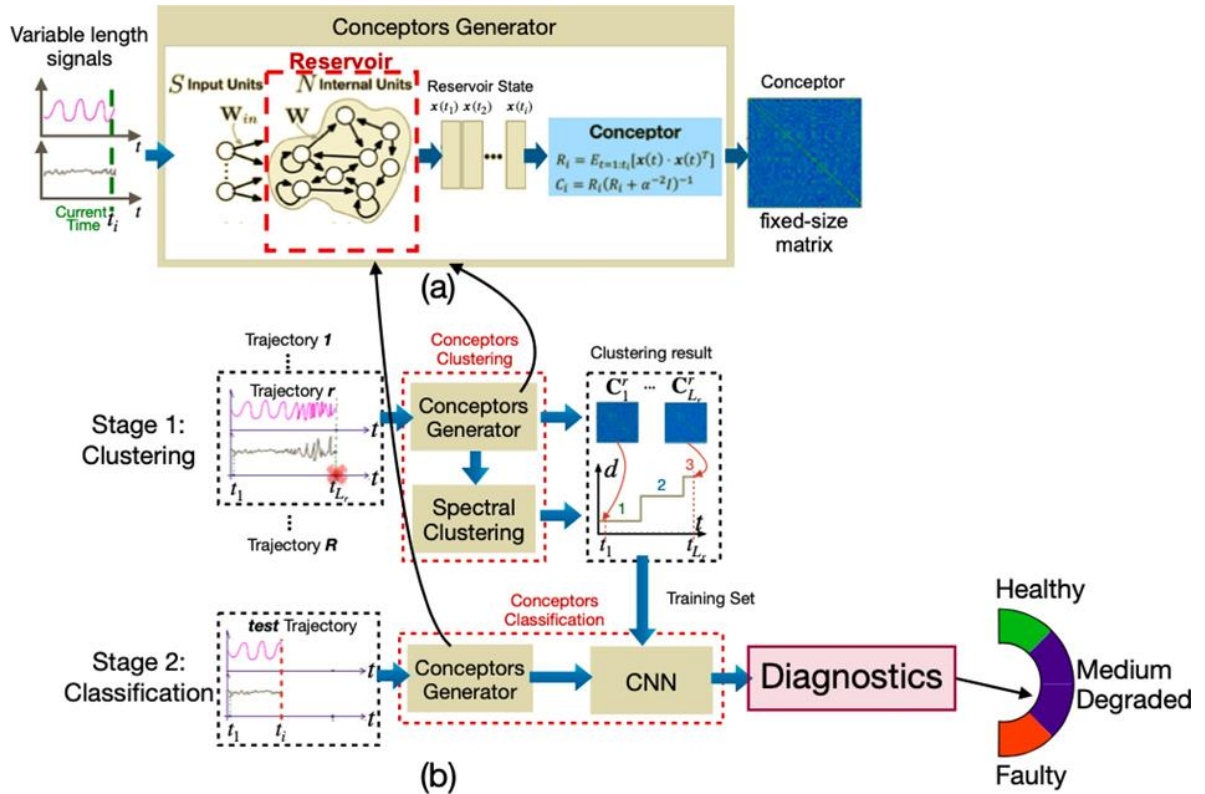


Figura 4. 3. “Reservoir Computing” para el diagnóstico de fallos [70].

- Pronóstico de fallos:** Está enfocado a prever la progresión hacia el fallo de un SSC, implicando el análisis de datos para anticipar tanto la degradación de sus componentes estructurales y funcionales como su vida útil remanente (véase la Sección 4.3) o la probabilidad de fallo del propio SSC. Estas predicciones son fundamentales para la gestión de la salud del SSC, lo que implica la ejecución de acciones operativas e intervenciones de mantenimiento. Existen tres categorías principales de enfoques para el pronóstico de fallos:
  - Basados en modelos (“Model-based”): Utilizan modelos físicos para prever la degradación. Su precisión es limitada cuando se enfrentan a SSC complejos con múltiples mecanismos de degradación.
  - Basados en datos (“Data-driven”): Extraen las leyes de degradación directamente de los datos usando técnicas de inteligencia artificial y aprendizaje automático. Su rendimiento depende de la cantidad y calidad de los datos disponibles.
  - Enfoques híbridos: combinan diversas fuentes de conocimiento, información y datos para aprovechar tanto las ventajas de los métodos basados en modelos como en datos.

### 4.3. ¿Qué es la Vida Útil Remanente?

La vida útil remanente, o “Remaining Useful Life” (RUL), es un concepto fundamental en el mantenimiento predictivo. Se refiere al tiempo estimado que le queda a un componente, máquina o sistema antes de que ocurra un fallo o se alcance un umbral crítico. En otras

palabras, el RUL es la predicción del tiempo restante hasta que el equipo deje de funcionar de manera confiable [2, 71].

La precisión del RUL depende de la calidad de los datos, la elección del modelo y la disponibilidad de información relevante. La aplicación exitosa del RUL permite programar el mantenimiento de manera más eficiente, evitar paradas no planificadas y prolongar la vida útil de los equipos [66].

Existen dos tipos principales de datos utilizados en la estimación de RUL: datos de eventos y datos de monitorización de la condición (CM). Los datos de eventos incluyen información sobre fallos pasados registrados, mientras que los datos de CM abarcan una amplia gama de información relacionada con el rendimiento y la salud del activo, como datos de vibración, análisis de aceite, temperatura, presión, humedad, etc.

Los métodos tradicionales de pronóstico se enfrentan con el desafío de manejar datos incompletos y ruidosos, a menudo recopilados en intervalos de tiempo no uniformes. Actualmente hay escasa investigación sobre pronósticos de fallos con datos faltantes [72]. Se requieren avances y nuevos métodos para predecir el RUL de un SSC basándose únicamente en mediciones recopiladas cuando ocurren eventos desencadenantes, como fallos del SSC o condiciones operativas extremas.

En la Figura 4.4 se presenta un método basado en ESN ("Echo State Network" o Red de Estado de Eco) para el pronóstico de fallos con mediciones basadas en eventos [73]. Las neuronas de entrada del ESN se activan para actualizar el estado del reservorio (una red de neuronas recurrentes que transforma las entradas en representaciones de alta dimensión) cuando las mediciones están disponibles (ocurren eventos). En cambio, si faltan datos (no ocurren eventos), las neuronas de entrada se desactivan y el estado se actualiza solo con el estado anterior y la señal objetivo, lo que obliga a la red a aprender del patrón de degradación histórico y la evolución de la señal objetivo incluso en situaciones de datos faltantes.

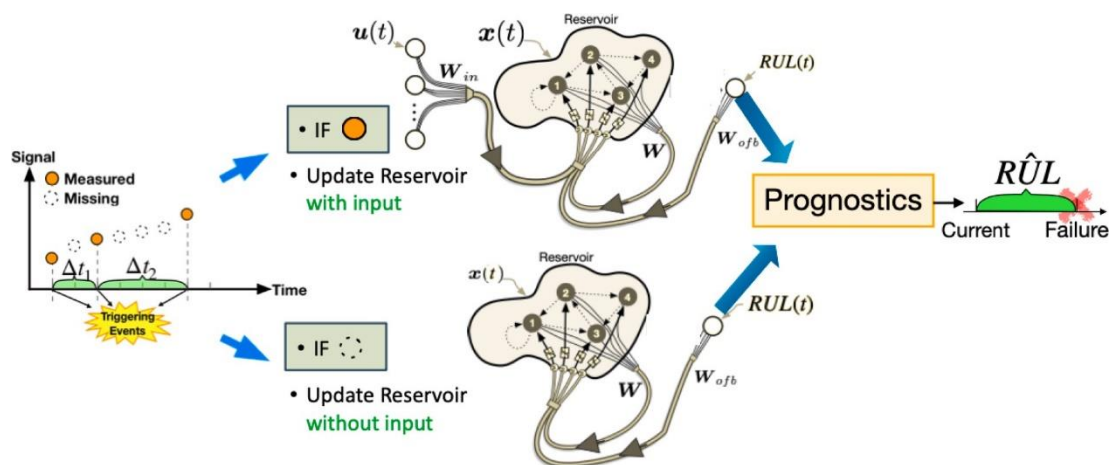


Figura 4. 4. Método basado en ESN para el pronóstico de fallos con mediciones basadas en eventos [66].

El concepto de RUL es fundamental para la planificación de actividades de mantenimiento, la provisión de repuestos, la evaluación del rendimiento operativo y la rentabilidad del propietario de un activo. Además, la estimación de la RUL tiene implicaciones estratégicas

en la gestión de la reutilización y el reciclaje de productos, con impactos significativos en el consumo de energía, el uso de materias primas, la contaminación y la gestión de residuos [2].

#### 4.4. Explicabilidad en Mantenimiento Predictivo

El XAI en el mantenimiento predictivo abarca una amplia gama de métodos que pueden categorizarse de varias maneras. Podemos separar estos métodos en las siguientes categorías: *ante-hoc* y *post-hoc*, tal y como se vio en la Sección 3.5. Se hace esta clasificación porque el objetivo de este proyecto es desarrollar modelos de estimación de RUL intrínsecamente interpretables (son métodos *ante-hoc*).

##### 4.4.1. Métodos *Post-hoc* en Mantenimiento Predictivo

Tal y como se explicó en la Sección 2.4.2, podemos identificar tres categorías de métodos *post-hoc*: agnósticos al modelo, específicos del modelo y combinaciones [2]. A continuación, se exponen ejemplos reales de los métodos vistos en la Sección 2.4.2 en mantenimiento predictivo.

- **SHAP:** es el método más utilizado en mantenimiento predictivo. Esto se debe a su versatilidad como método independiente del modelo y que puede proporcionar explicaciones globales. Se ha aplicado a problemas como la detección de anomalías [74], el diagnóstico de fallos [75] o el pronóstico de fallos [76].
- **LIME:** Protopapadakis et al. [77] calcularon la RUL aplicada al conjunto de datos CMAPSS de turbofán [78]. Inicialmente intentaron predecir la RUL con dos modelos: un “Random Forest” y una red neuronal profunda. Encontraron que el “Random Forest” tenía un rendimiento deficiente, lo que resultaba en explicaciones poco útiles. Su red neuronal profunda logró un alto rendimiento, por lo que aplicaron LIME. Compararon dos explicaciones de LIME, la primera en una etapa temprana del ciclo de vida del turbofan, cuando no había fallos presentes, y otra en una etapa posterior con fallos específicos. Descubrieron que LIME era capaz de identificar las características importantes para los fallos que reflejaban los fallos físicos. Además, demostraron que LIME tiene más dificultades para identificar las características importantes cuando se aplica a segmentos sin fallos, ya que cualquier cosa podría ocurrir en el futuro.
- **Propagación de relevancia por capas (LRP):** Han et al. [79] realizaron un diagnóstico de fallos en motores utilizando el modelo LeNet [80]. A través del uso de LRP, pudieron aportar explicabilidad a una arquitectura notable.
- **Contrafactuales:** Jakubowski et al. [81] desarrollaron una solución de mantenimiento predictivo para una operación industrial de laminado en frío. Utilizaron un algoritmo semi-supervisado basado en un Autocodificador Informado por Física (PIAE) [82]. Esta arquitectura se basó en la física al aplicar una lista de ecuaciones al comienzo de sus datos de entrada. El resultado de las ecuaciones se añadió a los datos de entrada de su autocodificador. Su modelo demostró ser más preciso que un autocodificador base. Aunque el PIAE ya tiene algunos aspectos interpretables, aplicaron contrafactuales como método de explicabilidad para mostrar las características importantes en las decisiones de su algoritmo.

- **Mapeo de activación de clase (CAM):** Mey et al. [83] está enfocado a la viabilidad de la XAI para explicar una CNN. Investigaron Grad-CAM, LRP y LIME como métodos para explicar una CNN en la detección de anomalías. Encontraron que LRP destacaba características indistinguibles y que LIME resaltaba características poco importantes. Grad-CAM pudo resaltar las características importantes que ellos habían identificado antes del entrenamiento de la CNN. Esto podría indicar que los métodos específicos del modelo superan a los métodos independientes del modelo cuando son aplicables.
- **“LionForests”:** Mylonas et al. [84] buscaron mejorar la explicabilidad de los “random forests” aplicando una versión expandida de “LionForests” para el diagnóstico de fallos. Ampliaron “LionForests” al ámbito de la clasificación multietiqueta mediante tres estrategias: etiqueta única, conjunto de etiquetas predicho y subconjuntos de etiquetas. La etiqueta única explica cada predicción individual (local), el conjunto de etiquetas predicho explica todas las predicciones (global) y los subconjuntos de etiquetas explican el modelo basándose en subconjuntos frecuentes de predicciones. Utilizaron métricas de precisión y proporcionaron métricas para sus explicaciones, como la longitud de las explicaciones y la cobertura de datos.

#### 4.4.2. Métodos Ante-hoc en Mantenimiento Predictivo

Como se vio en la Sección 2.4.1, no existen apenas métodos de explicabilidad *ante-hoc*. De igual forma, tampoco existen numerosos métodos *ante-hoc* aplicados al mantenimiento predictivo. En esta sección se presentarán los métodos desarrollados en el Capítulo 5.

- **RUL-RVE:** El método RUL-RVE (“Remaining Useful Life - Recurrent Variational Encoder”) es una técnica de predicción de la vida útil remanente, diseñada específicamente para gestionar y mantener la salud de sistemas complejos como los motores de aeronaves [1]. Este enfoque combina el poder de las redes neuronales recurrentes con la inferencia variacional para crear un modelo robusto y altamente interpretable. El algoritmo procesa datos de sensores en tiempo real, proyecta estos datos en un espacio latente continuo y utiliza estas proyecciones para realizar predicciones precisas de la vida útil remanente. Al integrar técnicas avanzadas de aprendizaje profundo, el RUL-RVE no solo proporciona estimaciones numéricas precisas, sino que también ofrece una representación visual de la degradación del motor, facilitando así la toma de decisiones informadas para el mantenimiento predictivo y la gestión de la salud del motor.
- **RUL-LAXCAT:** el algoritmo RUL-LAXCAT (“Remaining Useful Life - Latent Explanation-wise Attention for Time-series Classification”) combina técnicas avanzadas de aprendizaje profundo y mecanismos de atención para proporcionar predicciones precisas y explicables de la vida útil remanente de sistemas complejos, como turbinas de turbofan. Este modelo, basado en la arquitectura LAXCAT [4], utiliza redes LSTM bidireccionales y capas convolucionales para capturar tanto las dependencias temporales como las características espaciales de los datos de series temporales. La clave del modelo es el mecanismo de atención, que asigna pesos a diferentes características y momentos en el tiempo,

permitiendo identificar cuáles son los factores más influyentes en la predicción del RUL. Esta capacidad de atención no solo mejora la precisión del modelo, sino que también proporciona una interpretación clara y comprensible de las predicciones, facilitando la toma de decisiones informadas en el mantenimiento predictivo de equipos.

#### **4.5. Discusión**

El capítulo ha presentado una visión integral del mantenimiento predictivo, destacando su relevancia en la mejora de la gestión de activos industriales. A través del análisis frecuencial y de series temporales, se pueden detectar y diagnosticar fallos con mayor precisión, permitiendo planificar intervenciones de mantenimiento de manera más eficiente y reduciendo los costes asociados a paradas no planificadas. La capacidad de estos análisis para adaptarse a las condiciones reales del equipo ofrece una ventaja significativa sobre el mantenimiento preventivo tradicional basado en intervalos de tiempo fijos.

Asimismo, se ha resaltado la importancia de la explicabilidad en los modelos predictivos, un aspecto crucial para su integración y aceptación en entornos industriales. Los métodos ante-hoc, como el RUL-RVE y el RUL-LAXCAT, proporcionan predicciones precisas y comprensibles sobre la vida útil remanente (RUL), permitiendo una gestión más efectiva y transparente de los equipos. Por otro lado, los métodos post-hoc, como SHAP y LIME, se aplican a modelos ya entrenados para ofrecer explicaciones claras y detalladas sobre su funcionamiento, ayudando a identificar las variables y momentos más relevantes para las predicciones.

La combinación de precisión y transparencia en los modelos predictivos es esencial para maximizar los beneficios y garantizar un enfoque proactivo en la gestión del mantenimiento. Estas técnicas avanzadas no solo mejoran la eficiencia operativa y reducen los costes, sino que también aseguran la confianza y adopción de las soluciones de mantenimiento predictivo en diversas industrias.

## Capítulo 5: Algoritmos Propuestos

### 5.1. Introducción

En este capítulo se explicarán los dos algoritmos avanzados utilizados para la predicción de la vida útil remanente (RUL) de sistemas complejos: RUL-RVE y RUL-LAXCAT. Ambos algoritmos están diseñados para gestionar y analizar grandes cantidades de datos de sensores, aprendiendo patrones de degradación y proporcionando predicciones precisas y explicables sobre el estado de los sistemas.

A lo largo de este capítulo, se describirá en detalle la arquitectura de cada modelo, las técnicas utilizadas para la regularización y la función de coste, así como las metodologías implementadas en Python para desarrollar estos algoritmos.

### 5.2. Algoritmo RUL-RVE

#### 5.2.1. Arquitectura del Modelo

El algoritmo RUL-RVE (“Remaining Useful Life - Recurrent Variational Encoder”) se compone de varias partes interconectadas que trabajan juntas para procesar datos de sensores, aprender patrones de degradación, y predecir la vida útil remanente de sistemas [1]. Como se puede ver en la Figura 5.1, la arquitectura del modelo incluye un codificador recurrente (“Encoder”), que transforma las secuencias de datos en un espacio latente comprimido y significativo, y un modelo de regresión (“Regressor”), que utiliza estas representaciones latentes para realizar predicciones precisas de RUL. Además, el modelo incorpora técnicas de inferencia variacional para regularizar y estructurar el espacio latente, asegurando que las proyecciones de datos similares estén agrupadas de manera coherente. Esta combinación de componentes permite al algoritmo proporcionar no solo predicciones numéricas, sino también una visualización interpretable de la evolución de la degradación del sistema.

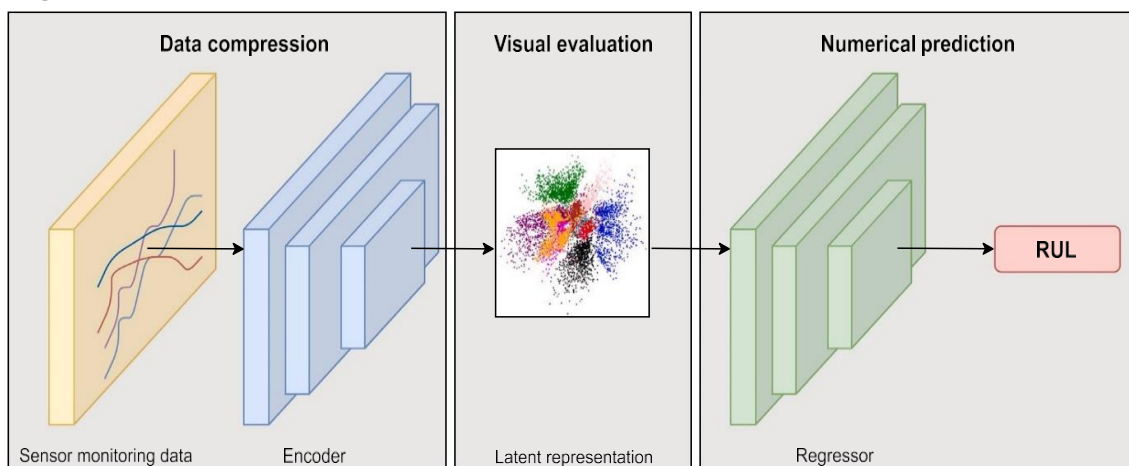


Figura 5. 1. Arquitectura del algoritmo RUL-RVE [1].

#### Codificador Recurrente

El codificador recurrente es una parte fundamental del algoritmo RUL-RVE, diseñado para procesar series temporales de datos y extraer características relevantes que puedan ser utilizadas para la predicción de la vida útil remanente (RUL).

El codificador recurrente se encarga de recibir las secuencias de datos de entrada, que pueden provenir de múltiples sensores instalados en el sistema, y transformar estas secuencias en una representación compacta y significativa en el espacio latente. Este proceso es crucial para capturar la dinámica temporal y las dependencias a largo plazo presentes en los datos de deterioro del sistema.

La principal función del codificador recurrente es aprender una representación interna de los datos que encapsule la información necesaria para la predicción precisa del RUL. Esto se logra mediante la compresión de los datos de entrada en un vector de menor dimensionalidad, que retiene las características esenciales del proceso de degradación del sistema. El propósito del codificador es, por tanto, servir como una herramienta de reducción de dimensionalidad y extracción de características que facilite la tarea de predicción.

### Espacio Latente

El espacio latente es un espacio de dimensionalidad reducida en el cual los datos de entrada, después de ser procesados por el codificador recurrente, son representados como vectores. Cada vector en este espacio latente encapsula la información relevante de una secuencia de datos de entrada, permitiendo una representación más manejable y analizable.

El codificador recurrente toma las secuencias de datos de entrada y las procesa a través de capas recurrentes (véase su implementación en la Sección 5.2.3, generando una salida que se comprime en el espacio latente. Esta proyección se realiza mediante la aplicación de una función de activación en las capas finales del codificador, produciendo un vector latente que contiene la información condensada de la secuencia original.

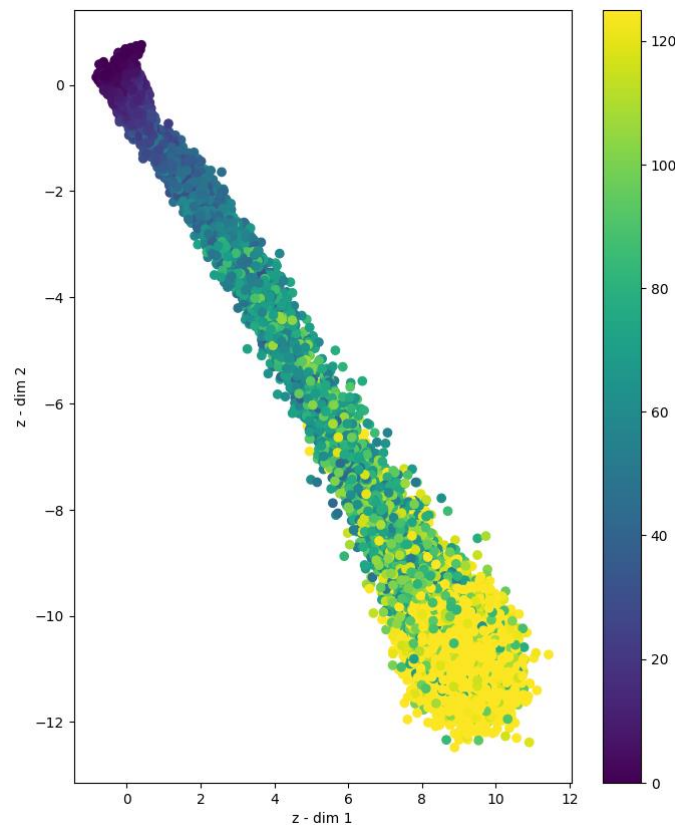


Figura 5. 2. Representación del espacio latente de 2 dimensiones del modelo RUL-RVE [1]. Cada punto representa la RUL de una instancia. Cuanto más oscuro, está más cerca del 0 (del fallo).

El uso de un espacio latente ofrece varias ventajas, incluyendo la capacidad de visualizar la progresión de la degradación del sistema (como el de la Figura 5. 2) y la detección de patrones anómalos. Además, facilita la aplicación de técnicas de inferencia variacional para regularizar y estructurar el espacio latente, asegurando que las proyecciones de datos similares estén cercanas entre sí, lo cual es crucial para la interpretabilidad y la precisión de las predicciones de RUL.

### **Modelo de Regresión**

El modelo de regresión es el componente del algoritmo que utiliza las características extraídas por el codificador recurrente para realizar predicciones numéricas de la vida útil remanente.

El modelo de regresión toma los vectores latentes generados por el codificador y los utiliza para predecir el RUL. Este modelo suele ser una red neuronal “feedforward” que convierte las características latentes en valores de RUL (véase la Sección 5.2.3 para su implementación).

El modelo de regresión está directamente integrado con el codificador recurrente, recibiendo como entrada los vectores latentes producidos por éste. La combinación del codificador y el modelo de regresión permite que el sistema aprenda a predecir la RUL a partir de las representaciones latentes de los datos de entrada.

La función del modelo de regresión es proporcionar estimaciones precisas de RUL basadas en las características latentes. Al minimizar los errores de predicción durante el entrenamiento, el modelo de regresión ajusta sus parámetros para mejorar continuamente la precisión de las predicciones de vida útil remanente, contribuyendo así a un mantenimiento predictivo más eficaz y eficiente.

### **5.2.2. Función de Coste y Regularización**

El algoritmo RUL-RVE utiliza una función de coste sofisticada que combina inferencia variacional con penalización de errores de predicción para regularizar el espacio latente y mejorar la precisión de las predicciones. Este enfoque asegura que las proyecciones de datos similares estén agrupadas de manera coherente y que el modelo pueda proporcionar predicciones precisas y fiables de la vida útil remanente (RUL).

#### **Inferencia Variacional**

La inferencia variacional es una técnica clave en el algoritmo RUL-RVE, utilizada para estructurar el espacio latente y asegurar que los datos proyectados en este espacio mantengan una coherencia interna que facilita tanto la interpretación como la predicción.

La inferencia variacional es un método de aproximación que permite estimar distribuciones complejas de probabilidad de una manera manejable [85]. En el contexto del RUL-RVE, se utiliza para aproximar la distribución posterior de los vectores latentes, lo que permite al modelo aprender representaciones significativas de los datos de entrada.

En el RUL-RVE, la inferencia variacional se implementa mediante el uso de una red neuronal que estima los parámetros de una distribución Gaussiana (media y varianza) para todos los vectores latentes. Durante el entrenamiento, el modelo optimiza una función de coste que incluye un término de divergencia de Kullback-Leibler (KL) que mide la diferencia

entre la distribución aproximada por el modelo y una distribución seleccionada *a priori*, típicamente como una distribución normal estándar.

El uso de inferencia variacional ofrece varios beneficios clave:

- **Regularización del espacio latente:** Garantiza que los vectores latentes estén organizados de manera que datos similares se proyecten cerca unos de otros, facilitando la identificación de patrones de degradación.
- **Evita el sobreajuste:** Al imponer una estructura sobre el espacio latente, la inferencia variacional ayuda a prevenir el sobreajuste a los datos de entrenamiento, mejorando la generalización del modelo.
- **Mejora la interpretabilidad:** La organización coherente de los vectores latentes facilita la visualización y el análisis de las trayectorias de degradación de los sistemas, proporcionando información valiosa para la toma de decisiones de mantenimiento.

### Penalización de errores de predicción

Para asegurar que el modelo no solo organice bien los datos en el espacio latente, sino que también proporcione predicciones precisas de RUL, el algoritmo RUL-RVE incluye un término de penalización de errores de predicción en su función de coste.

El término de penalización de errores de predicción se calcula como el error cuadrático medio (RMSE) entre los valores de RUL predichos por el modelo y los valores reales de RUL en el conjunto de datos de entrenamiento. Este término se agrega a la función de coste para ajustar los parámetros del modelo de manera que se minimicen los errores de predicción.

La inclusión de este término en la función de coste obliga al modelo no solo a aprender una buena representación de los datos en el espacio latente, sino también a realizar predicciones precisas. Al penalizar los errores de predicción, el modelo se entrena para ajustar mejor los vectores latentes a los valores de RUL correspondientes, mejorando la precisión y la fiabilidad de las predicciones.

### Función de Coste Total

El equilibrio entre la inferencia variacional y la penalización de errores de predicción se logra mediante la combinación de ambos términos en la función de coste total del modelo. La función de coste total  $L_x$ , se define como:

$$L_x = -D_{KL}(q_\phi(z | x) \parallel p_\theta(z)) + RMSE$$

Donde:

- $D_{KL}(q_\phi(z | x) \parallel p_\theta(z))$  es el término de divergencia KL que regula la estructura del espacio latente. Este término mide la diferencia entre la distribución posterior aproximada  $q_\phi(z | x)$  y la distribución *prior*  $p_\theta(z)$ . La *prior* seleccionada es una distribución normal estándar  $N(0, 1)$ , que asume que los vectores latentes siguen una distribución gaussiana con media cero y varianza uno.
- $RMSE$  es el error cuadrático medio que penaliza las predicciones incorrectas del RUL, definido como:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Al optimizar esta función de coste combinada, el modelo aprende a proyectar los datos de manera que sean útiles tanto para la organización coherente en el espacio latente como para la predicción precisa de RUL. Este enfoque asegura que el modelo pueda proporcionar diagnósticos interpretables y predicciones fiables, mejorando la eficiencia del mantenimiento predictivo en sistemas.

### 5.2.3. Explicaciones de RUL-RVE

El método RUL-RVE no solo se destaca por su precisión en la predicción de la vida útil remanente (RUL), sino también por su capacidad de proporcionar explicaciones interpretables sobre los datos y las predicciones. La explicabilidad es un aspecto crucial en aplicaciones de mantenimiento predictivo y gestión de la salud de sistemas complejos, como los motores de aeronaves, donde las decisiones deben ser justificables y comprensibles para los ingenieros y operadores [1].

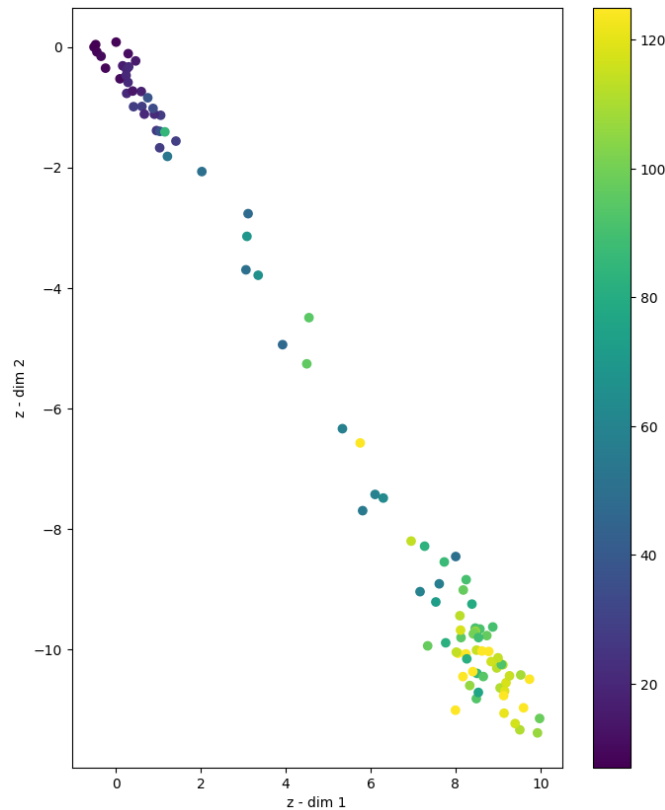


Figura 5. 3. Proyección de las secuencias de datos de un conjunto de prueba.

RUL-RVE proyecta las secuencias de datos en un espacio latente continuo y de baja dimensionalidad, donde cada punto representa una condición específica en un momento dado. En la Figura 5.3 podemos ver un ejemplo de estas proyecciones en 2D para el conjunto de prueba. Esta proyección facilita la visualización de la evolución de la salud del sistema a lo largo del tiempo. Estos son los beneficios de estas explicaciones:

1. **Identificación de Patrones de Degradación:** Al visualizar las proyecciones en el espacio latente, los ingenieros pueden identificar patrones claros de degradación y compararlos con condiciones conocidas de otros sistemas. Esto permite detectar anomalías o aceleraciones en el deterioro antes de que se produzcan fallos críticos.
2. **Diagnóstico Comparativo:** Los sistemas con condiciones operativas similares se agrupan cerca en el espacio latente, permitiendo comparaciones rápidas y efectivas entre diferentes unidades. Esto es útil para identificar sistemas que están experimentando un desgaste inusual.

El RUL-RVE ofrece una explicación visual de sus predicciones mediante la integración de técnicas de inferencia variacional. Las proyecciones en el espacio latente están regularizadas para asegurar que las representaciones de datos similares estén agrupadas, facilitando así la interpretación. Logrando así:

- **Transparencia:** La representación visual del estado del sistema proporciona transparencia en el proceso de predicción, permitiendo a los ingenieros entender por qué el modelo está haciendo una predicción específica.
- **Confianza en las Decisiones:** La capacidad de visualizar y entender el razonamiento detrás de las predicciones aumenta la confianza en las decisiones de mantenimiento basadas en las recomendaciones del modelo.
- **Facilidad de Comunicación:** Las visualizaciones generadas por el RUL-RVE son intuitivas y pueden ser fácilmente comunicadas a todos los niveles de la organización, desde técnicos hasta directivos, facilitando la toma de decisiones colaborativa.

### 5.2.3. Implementación en Python

La implementación del modelo RUL-RVE en Python fue llevada a cabo por los propios creadores de la arquitectura [86]. En este proyecto se ha usado esa misma implementación.

La implementación utiliza las librerías TensorFlow y Keras [87], aprovechando la flexibilidad y potencia de estas bibliotecas para construir redes neuronales profundas. A continuación, se presenta un resumen de los componentes clave involucrados en la creación del modelo: el codificador recurrente, el modelo de regresión y la integración de estos componentes en un modelo envolvente.

#### Codificador Recurrente

El codificador recurrente es responsable de procesar las secuencias de datos de entrada y comprimirlas en un espacio latente de menor dimensionalidad. Este proceso se realiza utilizando capas LSTM bidireccionales y una capa de muestreo personalizada para la inferencia variacional.

- **Capas LSTM Bidireccionales:** El codificador cuenta con una capa bidireccional de LSTM (“Long Short-Term Memory”) que permite capturar la dinámica temporal en ambas direcciones (hacia adelante y hacia atrás en el tiempo). Esta capacidad es crucial para entender las dependencias a largo plazo presentes en los datos de deterioro del sistema. En esta implementación, se utiliza una sola capa de LSTM bidireccional con una dimensión intermedia específica que debe ajustarse según los datos y el problema.
- **Capa de Muestreo Personalizada:** Después de la capa LSTM bidireccional, se aplican dos capas densas para generar los parámetros de la distribución gaussiana (media y varianza). Una capa de muestreo personalizada luego utiliza estos parámetros para generar los vectores latentes  $z$  mediante la reparametrización, permitiendo que el gradiente se propague a través de las operaciones de muestreo.

```
class Sampling(keras.layers.Layer):
    """Uses (z_mean, sigma) to sample z, the vector encoding an engine trajectory."""
    def call(self, inputs):
        mu, sigma = inputs
        batch = tf.shape(mu)[0]
        dim = tf.shape(mu)[1]
        epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
        return mu + tf.exp(0.5 * sigma) * epsilon

# ----- Encoder -----
inputs = Input(shape=(timesteps, input_dim), name='encoder_input')

mask = Masking(mask_value=masking_value)(inputs)

# LSTM encoding
h = Bidirectional(LSTM(intermediate_dim))(mask)

# VAE Z layer
mu = Dense(latent_dim)(h)
sigma = Dense(latent_dim)(h)

z = Sampling()([mu, sigma])

# Instantiate the encoder model:
encoder = keras.Model(inputs, [z, mu, sigma], name='encoder')
print(encoder.summary())
# -----
```

Figura 5. 4. Implementación en Python del codificador recurrente.

## Modelo de Regresión

El modelo de regresión utiliza las representaciones latentes generadas por el codificador para predecir la vida útil remanente (RUL). Este componente toma los vectores latentes como entrada y produce una predicción numérica del RUL. La arquitectura del regresor es una red neuronal “feedforward” simple, que incluye una capa intermedia seguida de una capa de salida.

- **Capa Intermedia:** Esta capa densa aplica una función de activación ('tanh') para transformar el vector latente en una representación intermedia.

- **Capa de Salida:** La capa final del regresor es una capa densa con una sola neurona y sin función de activación, ya que el objetivo es predecir un valor continuo de RUL.

```
# ----- Regressor -----
reg_latent_inputs = Input(shape=(latent_dim,), name='z_sampling_reg')
reg_intermediate = Dense(200, activation='tanh')(reg_latent_inputs)
reg_outputs = Dense(1, name='reg_output')(reg_intermediate)
# Instantiate the classifier model:
regressor = keras.Model(reg_latent_inputs, reg_outputs, name='regressor')
print(regressor.summary())
# -----
```

Figura 5. 5. Implementación del modelo de regresión en Python.

## Modelo Envolvente

El modelo envolvente integra el codificador y el modelo de regresión, y maneja la función de pérdida combinada que incluye tanto la pérdida de regresión como la divergencia KL para la regularización del espacio latente. Este modelo se encarga de la lógica de entrenamiento y evaluación.

```
class RVE(keras.Model):
    def __init__(self, encoder, regressor, decoder=None, **kwargs):
        super(RVE, self).__init__(**kwargs)
        self.encoder = encoder
        self.regressor = regressor
        self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")
        self.reg_loss_tracker = keras.metrics.Mean(name="reg_loss")
        self.decoder = decoder

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.kl_loss_tracker,
            self.reg_loss_tracker,
        ]

    def call(self, inputs):
        z, mu, sigma = self.encoder(inputs)
        reg_prediction = self.regressor(z)
        return reg_prediction, mu, sigma

    def train_step(self, data):
        x, target_x = data
        with tf.GradientTape() as tape:
            z, mu, sigma = self.encoder(x)
            kl_loss = -0.5 * tf.reduce_mean(tf.reduce_sum(1 + sigma - tf.square(mu) - tf.exp(sigma), axis=1))
            reg_prediction = self.regressor(z)
            reg_loss = tf.reduce_mean(keras.losses.mse(target_x, reg_prediction))
            total_loss = kl_loss + reg_loss
        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
        self.total_loss_tracker.update_state(total_loss)
        self.kl_loss_tracker.update_state(kl_loss)
        self.reg_loss_tracker.update_state(reg_loss)
        return {
            "loss": self.total_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result(),
            "reg_loss": self.reg_loss_tracker.result(),
        }

rve = RVE(encoder, regressor)
rve.compile(optimizer=keras.optimizers.Adam(learning_rate=0.0001))
```

Figura 5. 6. Implementación del modelo envolvente en Python.

### 5.3. Algoritmo RUL-LAXCAT

#### 5.3.1. Arquitectura de LAXCAT

El modelo LAXCAT (“Latent Axes for Categorical Data Analysis and Transformation”) está diseñado para manejar datos categóricos de manera eficiente, proporcionando además explicaciones claras y comprensibles sobre las relaciones y patrones presentes en los datos [4]. La arquitectura de LAXCAT se basa en la creación de ejes latentes que capturan las relaciones intrínsecas entre las variables categóricas, permitiendo así una transformación y análisis más efectivos de estos datos. Estos ejes latentes no solo facilitan la transformación y el análisis de los datos, sino que también proporcionan una base para generar explicaciones interpretables, ayudando a los usuarios a entender mejor los patrones y las interacciones dentro de sus datos.

En la Figura 5.7 se puede ver la arquitectura del modelo LAXCAT, que está compuesto por tres grandes bloques:

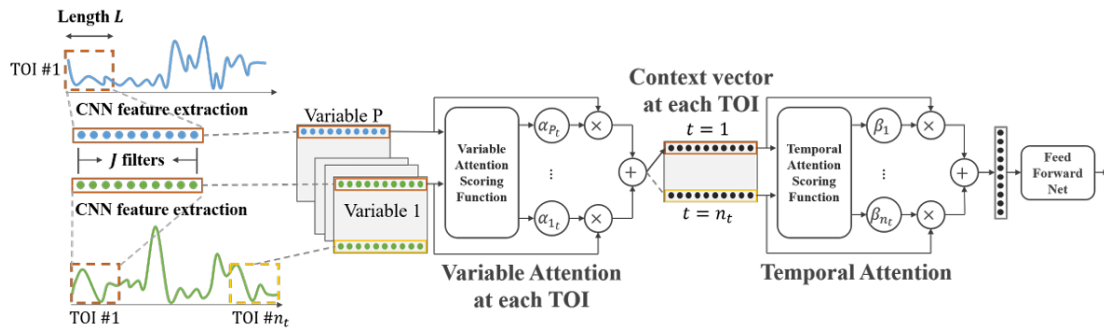


Figura 5. 7. Arquitectura de LAXCAT [4].

#### 1. Extracción de Características con CNN:

La red neuronal convolucional (CNN) procesa los datos de cada Intervalo de Interés (TOI), que son segmentos específicos de la serie temporal seleccionados para el análisis. Utilizando múltiples filtros convolucionales, la CNN genera representaciones específicas para cada variable, capturando diferentes aspectos de la señal temporal. Este proceso transforma los datos en características más manejables y relevantes, facilitando un análisis posterior más efectivo.

#### 2. Atención de Variables:

El módulo de atención de variables asigna pesos a cada variable dentro de un TOI, destacando las más relevantes para la clasificación. Este proceso se realiza mediante una función de puntuación de atención, que evalúa la importancia relativa de cada variable. Las características ponderadas permiten al modelo

enfocarse en las variables más significativas, mejorando la precisión de la clasificación.

### 3. Atención Temporal:

El mecanismo de atención temporal evalúa la relevancia de cada TOI en el contexto temporal global. A través de puntuaciones de atención, este módulo asigna pesos a los TOI, determinando su importancia relativa. Los TOI más críticos reciben mayores pesos, asegurando que su influencia en el análisis final sea proporcional a su relevancia, lo que mejora la precisión del modelo.

### 4. Vector de Contexto en cada TOI:

El vector de contexto se forma combinando las salidas ponderadas de los mecanismos de atención de variables y temporal. Este vector integra la información más relevante extraída de los diferentes TOI, proporcionando una representación compacta y significativa de los datos. Esta integración facilita la posterior clasificación, ofreciendo una visión general y contextualizada de los datos. La Figura 5.8 es un ejemplo de este vector de contexto. El eje X son los instantes temporales, y el Y son las variables. Cuanto más oscuro es el color, más atención recibe esa variable en ese instante de tiempo.

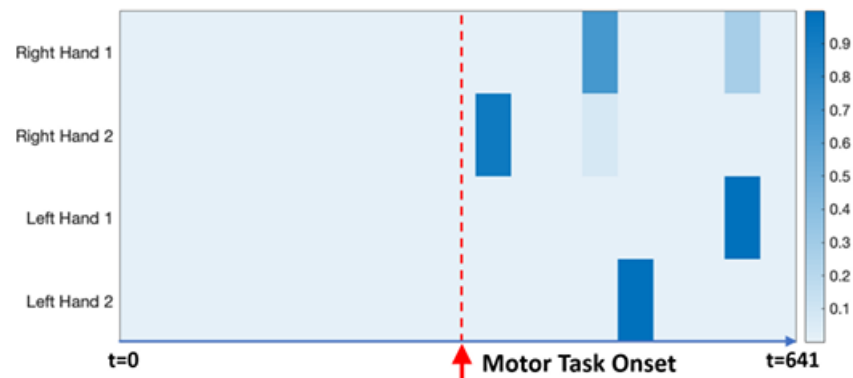


Figura 5. 8. Representación visual de las variables latentes [4].

### 5. Red “Feedforward”:

La red “feedforward” procesa el vector de contexto final, utilizando la información integrada de los TOI para realizar la clasificación final. Esta red neuronal completamente conectada genera las predicciones basadas en la información contextual proporcionada, consolidando todo el procesamiento previo y permitiendo al modelo ofrecer una salida precisa y fundamentada en datos relevantes.

#### 5.3.2. Arquitectura de RUL-LAXCAT

La adaptación del modelo LAXCAT para el análisis de la vida útil remanente (RUL) ha resultado en la creación del modelo RUL-LAXCAT, creado en su totalidad en este proyecto.

Este modelo está diseñado para trabajar con series temporales y hacer estimaciones precisas de una variable continua, que en este caso es la vida útil remanente de los sistemas analizados. La arquitectura de RUL-LAXCAT combina técnicas de atención variable y temporal para capturar tanto las dependencias entre diferentes variables como las dependencias a lo largo del tiempo, lo cual es crucial para la predicción precisa del RUL.

En la Figura 5. 9 se puede ver la arquitectura de este modelo, cuyos módulos principales se describen a continuación:

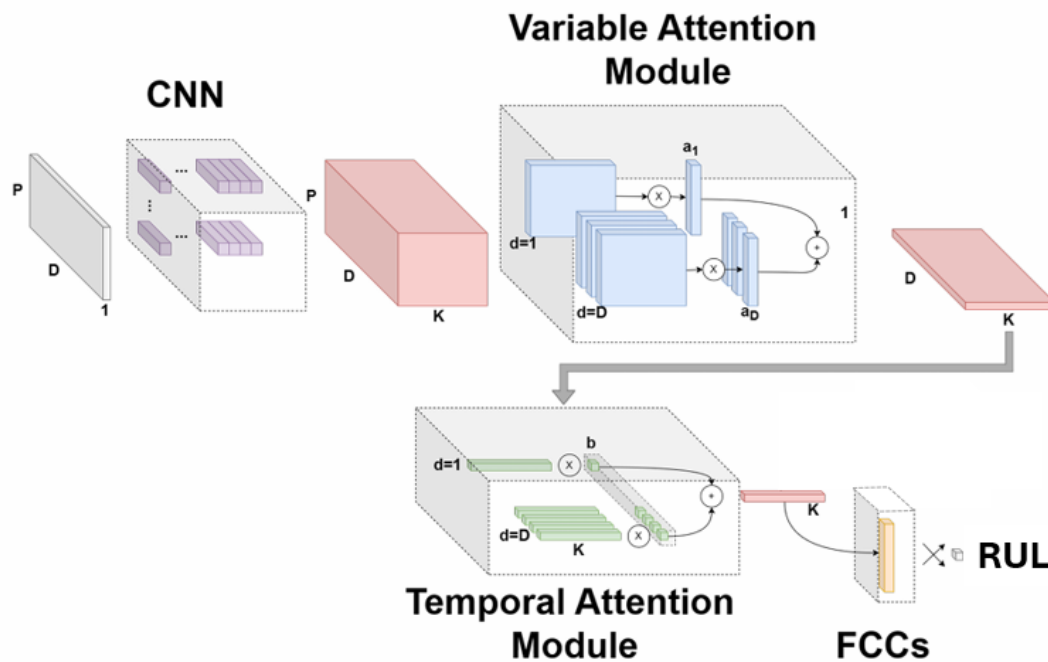


Figura 5. 9. Arquitectura del modelo RUL-LAXCAT.

#### 1. Extracción de Características con CNN:

- **Entrada de Datos:** El modelo recibe series temporales multivariantes como entrada. Cada variable se representa como una serie temporal independiente.
- **Convoluciones:** Se aplican convoluciones a cada variable individualmente para extraer características locales significativas. Este proceso se realiza utilizando múltiples filtros que capturan diferentes patrones dentro de cada serie temporal.

#### 2. Módulo de Atención de Variables:

- **Atención a Variables:** Las características extraídas de cada variable se procesan mediante un módulo de atención que asigna diferentes pesos a las variables en función de su importancia relativa. Este módulo utiliza dos

capas completamente conectadas y funciones de activación ReLU para calcular las puntuaciones de atención.

- **Cálculo del Contexto:** Las características ponderadas se combinan para formar un vector de contexto que captura la información más relevante de todas las variables en cada punto de la serie temporal.

### 3. Módulo de Atención Temporal:

- **Atención a lo Largo del Tiempo:** El vector de contexto obtenido del módulo de atención variable se procesa mediante un módulo de atención temporal que asigna diferentes pesos a cada instante de tiempo. Esto permite al modelo concentrarse en los momentos más importantes para la predicción de RUL.
- **Cálculo del Contexto Temporal:** Las características ponderadas en el tiempo se combinan para formar un vector de contexto temporal que encapsula la información más relevante a lo largo de toda la serie temporal.

### 4. Red de Alimentación Directa (“Feed Forward Net”):

- **Predicción del RUL:** El vector de contexto temporal se pasa a través de una red completamente conectada (“fully connected”) que realiza la predicción final de la vida útil remanente. Esta red utiliza una capa completamente conectada con una sola neurona de salida para proporcionar una estimación continua del RUL.

## Beneficios de la Arquitectura RUL-LAXCAT

- **Captura de Dependencias Complejas:** La combinación de atención variable y temporal permite al modelo capturar de manera efectiva las dependencias tanto entre diferentes variables como a lo largo del tiempo, mejorando la precisión de las predicciones.
- **Centrarse en la Información Relevante:** Los módulos de atención permiten al modelo centrarse en la información más relevante, descartando ruido y detalles irrelevantes, lo que resulta en predicciones más robustas y fiables.
- **Interpretabilidad:** La arquitectura de atención proporciona explicaciones interpretables sobre las predicciones, ayudando a los usuarios a entender qué variables y momentos temporales son más importantes para la estimación de la vida útil remanente.
- **Adaptabilidad:** La arquitectura modular de RUL-LAXCAT facilita su adaptación a diferentes tipos de datos y escenarios de predicción, haciéndolo versátil y aplicable a una amplia variedad de casos en el mantenimiento predictivo.

### 5.3.3. Función de Coste

La función de coste es un componente fundamental en el entrenamiento de cualquier modelo de aprendizaje automático, ya que guía el proceso de optimización ajustando los parámetros del modelo para minimizar los errores de predicción. En el modelo RUL-LAXCAT, se ha utilizado el error cuadrático medio (MSE, por sus siglas en inglés) como función de coste. Esta elección es particularmente adecuada para tareas de regresión, como la predicción de la vida útil remanente (RUL), donde el objetivo es minimizar la diferencia entre las predicciones del modelo y los valores reales.

El error cuadrático medio (MSE) mide la media de los cuadrados de las diferencias entre las predicciones y los valores reales. Matemáticamente, se expresa de la siguiente manera:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Donde:

- $n$  es el número de muestras.
- $y_i$  es el valor real de la  $i$ -ésima muestra.
- $\hat{y}_i$  es el valor predicho por el modelo para la  $i$ -ésima muestra.

El objetivo del entrenamiento es minimizar esta función de coste, lo que implica reducir la discrepancia entre las predicciones y los valores reales.

Al elevar las diferencias al cuadrado, el MSE penaliza más severamente los errores grandes. Esto significa que predicciones que están muy lejos de los valores reales tendrán un mayor impacto en la función de coste, incentivando al modelo a ajustarse mejor y reducir estos grandes errores.

### 5.3.4. Explicaciones de RUL-LAXCAT

El modelo RUL-LAXCAT utiliza técnicas avanzadas de atención para proporcionar explicaciones interpretables. Dos componentes clave en este aspecto son la atención a las variables y la atención temporal:

- **Atención a las Variables:**
  - **Descripción:** El modelo asigna pesos a las diferentes variables de entrada en cada paso de tiempo, determinando su importancia relativa para la predicción del RUL.
  - **Visualización:** Las atenciones a las variables pueden visualizarse en forma de mapas de calor que muestran cómo varía la importancia de cada variable a lo largo del tiempo.

- **Atención Temporal:**
  - **Descripción:** El modelo también asigna pesos a los diferentes instantes de tiempo dentro de la serie temporal, destacando los momentos más críticos para la predicción del RUL.
  - **Visualización:** Las atenciones temporales pueden representarse como mapas de calor para mostrar cómo la importancia de los distintos momentos temporales afecta la predicción final.

En la Figura 5. 10 se muestra un ejemplo de mapa de calor de las atenciones cruzadas de tiempo y variables para un ejemplo del conjunto de datos CMAPSS (véase la Sección 6.2), que ilustra cómo el modelo RUL-LAXCAT asigna atención a diferentes sensores y momentos temporales.

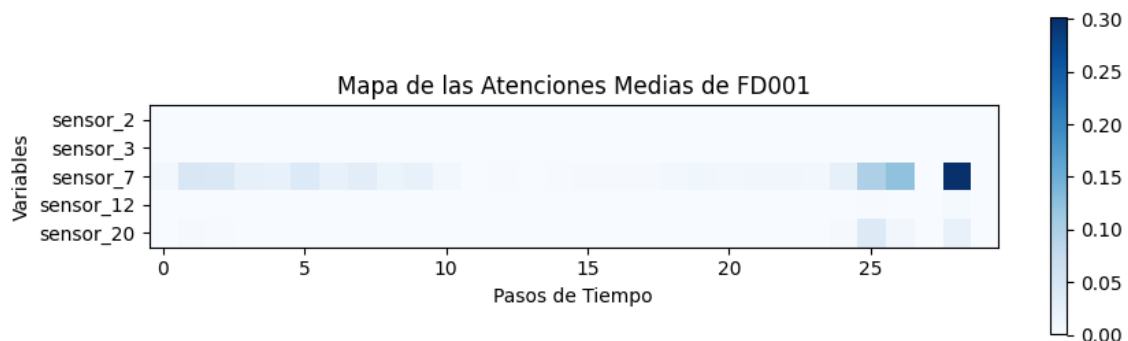


Figura 5. 10. Ejemplo de mapa de calor de Atenciones Cruzadas.

### Interpretación del Mapa de Calor

- **Ejes del Mapa:** El eje vertical representa las variables (en este caso, sensores) utilizadas en el modelo, mientras que el eje horizontal representa los pasos de tiempo.
- **Colores del Mapa:** Los colores en el mapa de calor indican la magnitud de la atención asignada. Colores más claros (hacia el blanco) indican mayor atención, mientras que colores más oscuros (hacia el azul oscuro) indican menor atención. Estas atenciones toman valores entre 0 y 1.
- **Patrones Observados:** En este ejemplo, podemos observar que ciertos sensores reciben más atención en momentos específicos, lo que sugiere que son más relevantes para la predicción del RUL en esos instantes. Esto proporciona una visión clara de qué variables y momentos temporales son considerados más importantes por el modelo.

### Beneficios de la Explicabilidad en RUL-LAXCAT

- **Diagnóstico y Mantenimiento:** La capacidad de identificar qué sensores y periodos de tiempo son más importantes para la predicción permite a los ingenieros centrarse en las áreas críticas y realizar diagnósticos más precisos.

- **Mejora Continua:** Las explicaciones proporcionadas por el modelo pueden utilizarse para mejorar continuamente el proceso de mantenimiento y ajustar las estrategias basadas en los patrones identificados.

### 5.3.5. Implementación en Python

La implementación del modelo RUL-LAXCAT en Python se lleva a cabo utilizando la biblioteca PyTorch [88], que es ampliamente utilizada para el desarrollo de modelos de aprendizaje profundo. A continuación, se describen los componentes clave del modelo y cómo se integran para realizar la predicción de la vida útil remanente (RUL).

#### Capa de Atención de las Variables (“input\_attention\_layer”)

Esta capa se encarga de asignar pesos a las diferentes variables de entrada (sensores) en cada paso de tiempo, determinando su importancia relativa para la predicción del RUL. La capa de atención a las variables permite al modelo centrarse en las variables más relevantes, mejorando la precisión de la predicción y proporcionando una base para generar explicaciones interpretables. La capa de atención a las variables incluye dos capas densas con funciones de activación ReLU (“Rectified Linear Unit”). La primera capa tiene una dimensión de salida igual al número de características ( $p$ ), y la segunda capa tiene una dimensión de salida igual al número de filtros convolucionales ( $j$ ).

```
class input_attention_layer(nn.Module):
    def __init__(self, p, j):
        super(input_attention_layer, self).__init__()
        self.weight1 = nn.Parameter(torch.ones(p, 1), requires_grad=True)
        self.bias1 = nn.Parameter(torch.zeros(j), requires_grad=True)
        self.weight2 = nn.Parameter(torch.ones(j, p), requires_grad=True)
        self.bias2 = nn.Parameter(torch.zeros(p), requires_grad=True)

    def forward(self, x):
        l = x.size(3)
        h = [0 for i in range(1)]
        x = x.transpose(1, 3)
        variable_attn = []
        for i in range(1):
            tmp = F.relu(torch.matmul(x[:, i, :, :], self.weight1).squeeze(-1) + self.bias1)
            tmp = F.relu(torch.matmul(tmp, self.weight2) + self.bias2)
            attn = F.softmax(tmp, -1).unsqueeze(1)
            variable_attn.append(attn)
            h[i] = torch.sum(attn * x[:, i, :, :], -1)
            h[i] = h[i].unsqueeze(-1)
        # B * j
        return torch.cat(h, -1), torch.stack(variable_attn, dim=2)
```

Figura 5. 11. Implementación en Python de la Atención Variable.

#### Capa de Atención Temporal (“temporal\_attention\_layer”)

Esta capa asigna pesos a diferentes instantes de tiempo dentro de la serie temporal, permitiendo al modelo enfocarse en los momentos más críticos para la predicción del RUL. La atención temporal ayuda a capturar las dependencias a lo largo del tiempo, asegurando que el modelo considere adecuadamente los patrones temporales importantes para la predicción del RUL. Al igual que en la capa de atención temporal, esta capa incluye dos capas densas con función de activación ReLU. La primera capa tiene una dimensión de salida igual al número de características ( $j$ ), y la segunda capa tiene una dimensión de salida igual al número de puntos temporales ( $l$ ).

```

class temporal_attention_layer(nn.Module):
    def __init__(self, j, l):
        super(temporal_attention_layer, self).__init__()
        self.weight1 = nn.Parameter(torch.ones(max(1, l), 1), requires_grad=True)
        self.bias1 = nn.Parameter(torch.zeros(j), requires_grad=True)
        self.weight2 = nn.Parameter(torch.ones(j, max(1, l)), requires_grad=True)
        self.bias2 = nn.Parameter(torch.zeros(max(1, l)), requires_grad=True)

    def forward(self, x):
        tmp = F.relu(torch.matmul(x, self.weight1).squeeze(-1) + self.bias1)
        tmp = F.relu(torch.matmul(tmp, self.weight2) + self.bias2)
        attn = F.softmax(tmp, -1).unsqueeze(1)
        x = torch.sum(attn * x, -1)
        return x, attn.squeeze()

```

Figura 5. 12. Implementación en Python de la Atención Temporal.

### Modelo LaxCat

El modelo LaxCat combina las capas de atención de las variables y temporal con una red convolucional (CNN) para extraer características locales de las series temporales de cada variable. Finalmente, una capa completamente conectada se utiliza para realizar la predicción del RUL. En la entrada, se utiliza una capa convolucional (CNN) para cada variable de entrada para extraer características locales. Para la predicción final, se utiliza una capa completamente conectada sin función de activación, con el objetivo de predecir un valor continuo de RUL.

```

class LaxCat(nn.Module):
    def __init__(self, input_size, input_channel, hidden_dim=32, kernel_size=3, stride=1):
        super(LaxCat, self).__init__()
        self.input_size = input_size
        self.Conv1 = nn.ModuleList([
            nn.Conv1d(1, hidden_dim, kernel_size=kernel_size, stride=stride, padding='same') for _ in range(input_channel)
        ])

        self.variable_attn = input_attention_layer(p=input_channel, j=hidden_dim)
        self.temporal_attn = temporal_attention_layer(j=hidden_dim, l=input_size)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        x = x.transpose(1, 2)
        x = [self.Conv1[i](x[:, i:i+1, :]).unsqueeze(-1) for i in range(len(self.Conv1))]
        x = torch.cat(x, -1).permute(0, 3, 1, 2)
        x, variable_attn = self.variable_attn(x)
        x, temporal_attn = self.temporal_attn(x)
        output = self.fc(x).squeeze()
        return output, variable_attn, temporal_attn

```

Figura 5. 13. Implementación en Python del Modelo LaxCat.

## 5.4. Discusión

En este capítulo hemos explorado dos algoritmos avanzados para la predicción de la vida útil remanente (RUL) de sistemas complejos: RUL-RVE y RUL-LAXCAT.

El algoritmo RUL-RVE combina un codificador recurrente con inferencia variacional para transformar secuencias de datos en vectores un espacio latente comprimido y significativo, permitiendo predicciones precisas del RUL y visualizaciones claras de la degradación del sistema. La función de coste, que incluye la penalización de errores de

predicción y la regularización del espacio latente, asegura que el modelo no solo sea preciso sino también interpretable, proporcionando explicaciones claras que mejoran la transparencia y la confianza en las predicciones.

Por otro lado, el modelo RUL-LAXCAT adapta la arquitectura de LAXCAT para el análisis de series temporales, utilizando técnicas de atención de variables y temporal para capturar dependencias complejas en los datos. La combinación de redes convolucionales y de atención permite al modelo centrarse en la información más relevante, mejorando la precisión de las predicciones y proporcionando visualizaciones interpretables. La función de coste basada en el error cuadrático medio (MSE) ayuda a minimizar los errores de predicción, incentivando ajustes precisos del modelo. En conjunto, ambos modelos ofrecen herramientas robustas y efectivas para el mantenimiento predictivo, destacándose por su capacidad de generar predicciones precisas y explicables.

Por último, se ha visto la implementación de ambos algoritmos en Python utilizando bibliotecas como Pytorch o TensorFlow. En el caso de RUL-RVE se ha utilizado el código desarrollado por los propios creadores de la arquitectura, por otro lado, en el caso de RUL-LAXCAT ha sido necesario crear el algoritmo desde el principio.



## Capítulo 6: Resultados

### 6.1. Introducción

En este capítulo se presentan los resultados obtenidos en el análisis del conjunto de datos C-MAPSS y en las simulaciones realizadas con los modelos de predicción RUL-LAXCAT y RUL-RVE. A través de este análisis, se evaluará la precisión y eficiencia de los modelos desarrollados, así como su capacidad para predecir la vida útil remanente (RUL) de los motores aeroespaciales de C-MAPSS.

Primero, se describirá en detalle el conjunto de datos C-MAPSS, desarrollado por la NASA, el cual se ha convertido en un estándar en la investigación de pronóstico y estimación de RUL en sistemas aeroespaciales. Se explicará su estructura, incluyendo las variables de entrada, operativas y de salida, así como los diferentes sub-conjuntos que lo componen, cada uno diseñado para evaluar distintas condiciones operativas y de fallo.

A continuación, se presentarán los resultados de las simulaciones realizadas con los modelos RUL-LAXCAT y RUL-RVE. Para cada modelo, se analizarán los resultados obtenidos en los diferentes sub-conjuntos de datos, evaluando la precisión de las predicciones mediante la métrica RMSE y el tiempo medio de entrenamiento por época (ATT). Finalmente, se realizará una comparación entre ambos modelos, destacando sus ventajas y limitaciones en términos de precisión y tiempo de entrenamiento, así como la calidad de las explicaciones proporcionadas por cada uno.

### 6.2. Conjunto de Datos C-MAPSS

El conjunto de datos CMAPSS (“Commercial Modular Aero-Propulsion System Simulation”) es ampliamente utilizado en la investigación de pronóstico y estimación de RUL de sistemas aeroespaciales [78]. Este conjunto de datos fue desarrollado por la NASA y se ha convertido en un estándar para el desarrollo y validación de algoritmos de predicción de fallos, tal como se ha indicado anteriormente.

#### Estructura del Conjunto de Datos

El conjunto de datos CMAPSS está compuesto por varios sub-conjuntos que simulan diferentes condiciones operativas y de fallo de los motores. Cada sub-conjunto incluye información detallada sobre los ciclos de operación del motor, proporcionando una base robusta para la investigación y el desarrollo de algoritmos de pronóstico.

- **Variables de Entrada:**
  - **ID:** Identificador único del motor.
  - **Ciclo:** Ciclo de operación del motor (equivalente a un registro temporal).
  - **Sensores:** El conjunto de datos incluye lecturas de 21 sensores diferentes que miden diversas variables operativas del motor, como la temperatura, presión y velocidad del ventilador, entre otras.

- **Variables Operativas:**

- **Condiciones de Vuelo:** Incluye tres condiciones operativas (altitud, velocidad del avión y temperatura ambiente) que afectan el comportamiento del motor.
- **Modos de Operación:** Diferentes modos de operación que simulan diversas condiciones de vuelo y carga del motor.

- **Variables de Salida:**

- **RUL:** La vida útil remanente en ciclos para cada motor. Esta es la variable objetivo principal para los algoritmos de pronóstico.

#### Descripción de los Sensores por Tipo

| Sensor    | Descripción                                                       | Tipo        |
|-----------|-------------------------------------------------------------------|-------------|
| Sensor 1  | Temperatura del aire de entrada (T2)                              | Temperatura |
| Sensor 3  | Temperatura de salida del compresor (T24)                         | Temperatura |
| Sensor 4  | Temperatura de entrada de la turbina (T30)                        | Temperatura |
| Sensor 5  | Temperatura de salida de la turbina de baja presión (T50)         | Temperatura |
| Sensor 17 | Temperatura de la pala de la turbina de alta presión (T48)        | Temperatura |
| Sensor 18 | Temperatura de salida de la turbina de alta presión (T49)         | Temperatura |
| Sensor 2  | Presión del compresor (P2)                                        | Presión     |
| Sensor 8  | Relación de presión del motor (EPR)                               | Presión     |
| Sensor 9  | Presión estática a la salida del compresor de alta presión (Ps30) | Presión     |
| Sensor 6  | Velocidad física del ventilador (fan Nf)                          | Velocidad   |
| Sensor 7  | Velocidad física del núcleo (core Nc)                             | Velocidad   |
| Sensor 11 | Velocidad corregida del ventilador (fan Nf_dmd)                   | Velocidad   |
| Sensor 12 | Velocidad corregida del núcleo (core Nc_dmd)                      | Velocidad   |
| Sensor 19 | Velocidad del eje de baja presión                                 | Velocidad   |
| Sensor 20 | Velocidad del eje de alta presión                                 | Velocidad   |

|           |                                                        |                     |
|-----------|--------------------------------------------------------|---------------------|
| Sensor 10 | Relación de flujo de combustible a Ps30 ( $\phi$ )     | Relaciones y Ratios |
| Sensor 13 | Relación de bypass                                     | Relaciones y Ratios |
| Sensor 14 | Relación combustible-aire del quemador (BPR)           | Relaciones y Ratios |
| Sensor 21 | Relación combustible-aire integrada del quemador (IRB) | Relaciones y Ratios |
| Sensor 15 | Aire de sangrado (W31)                                 | Flujo               |
| Sensor 16 | Demanda de potencia (W32)                              | Flujo               |

Tabla 6. 1. Descripción de los sensores por Tipo.

### Detalles de los Sub-Conjuntos

El conjunto de datos CMAPSS está dividido en cuatro sub-conjuntos principales, cada uno diseñado para evaluar diferentes aspectos y condiciones de fallo:

1. **FD001:** Simula un único tipo de fallo: desgaste en la sección del compresor de baja presión, en condiciones operativas constantes, es decir, no varían las condiciones de vuelo.
2. **FD002:** Incluye el mismo tipo de fallo que FD001, pero bajo condiciones operativas variables, es decir, varían las condiciones de vuelo (altitud, velocidad del avión y temperatura ambiente).
3. **FD003:** Contiene múltiples tipos de fallo: desgaste en la sección del compresor de baja presión y fallo en la sección de la turbina de alta presión, en condiciones operativas constantes.
4. **FD004:** Integra múltiples tipos de fallo como FD003, pero esta vez en condiciones operativas variables.

Cada uno de estos sub-conjuntos está diseñado para desafiar diferentes aspectos de los algoritmos de pronóstico, proporcionando una plataforma completa para el desarrollo y evaluación de métodos de estimación de RUL.

En la tabla 6.2 se puede ver el número de trayectorias disponibles para cada uno de los sub-conjuntos de datos, lo que define el tamaño de estos.

| Sub-conjunto | Entrenamiento | Prueba |
|--------------|---------------|--------|
| FD001        | 100           | 100    |
| FD002        | 260           | 259    |
| FD003        | 100           | 100    |
| FD004        | 249           | 248    |

Tabla 6. 2. Tamaño de los sub-conjuntos de datos de C-MAPSS.

Al tratarse de datos de secuencias temporales, es necesario determinar el tamaño de dichas secuencias. Para todas las simulaciones, tanto de RUL-LAXCAT como de RUL-RVE,

se trabaja con secuencias de longitud 30, es decir, para cada instancia  $X$  de cada trayectoria, se toman las 30 instancias anteriores. La longitud de 30 se elige porque proporciona un equilibrio adecuado entre la cantidad de información histórica y la complejidad computacional, permitiendo capturar patrones relevantes sin sobrecargar el modelo. Además, si se utilizaran secuencias más largas, se perdería una cantidad significativa de datos, ya que las primeras 29 instancias de cada trayectoria (longitud de la secuencia menos uno) no pueden ser utilizadas para el entrenamiento o la validación del modelo.

Además, en ambos modelos, la etiqueta RUL se calcula como el ciclo máximo de la trayectoria menos el ciclo en el instante actual. Sin embargo, se determina un RUL máximo de 125 ciclos, es decir, durante las primeras instancias se mantiene fijo en 125 y después comienza a descender hasta 0 tal y como se muestra en la Figura 6.1.

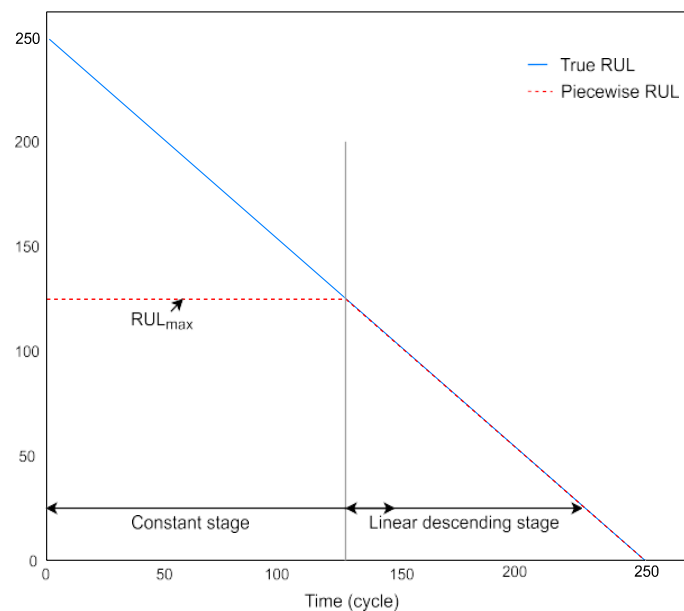


Figura 6. 1. Cálculo de etiqueta RUL para cada instancia de datos [1].

### 6.3. Simulaciones en RUL-LAXCAT

Para todos los conjuntos de datos, se entrenó el modelo con las trayectorias de entrenamiento, las cuales se dividieron en subconjuntos de validación y entrenamiento (20% validación y 80% entrenamiento). Para calcular el RMSE de las simulaciones, se utilizaron las trayectorias de prueba. Sin embargo, debido a que los datos de prueba son instancias individuales independientes, las atenciones se calcularon usando los datos de validación.

#### 6.3.1. FD001

Para ajustar el modelo y los hiperparámetros, se utilizó el conjunto de datos FD001, considerado *a priori* el más manejable por tener solo un tipo de fallo y operaciones operativas constantes.

### Simulación Número de Sensores

En primer lugar, se determinó la cantidad de sensores a utilizar en las simulaciones de manera que se minimizara el RMSE, manteniendo un tiempo de entrenamiento razonable. En estas simulaciones el “batch size” tenía un valor de 32 y el “learning rate” un valor de 0.001 (se explican más adelante en la sección).

Se comenzó simulando el conjunto de datos FD001 con todos los sensores disponibles (21) para identificar, a través de las explicaciones, cuáles eran los sensores más relevantes. Se obtuvo un RMSE de 14.38 y un tiempo de simulación por época de 63.19 segundos. Además, se obtuvo el siguiente gráfico de atención de las variables:

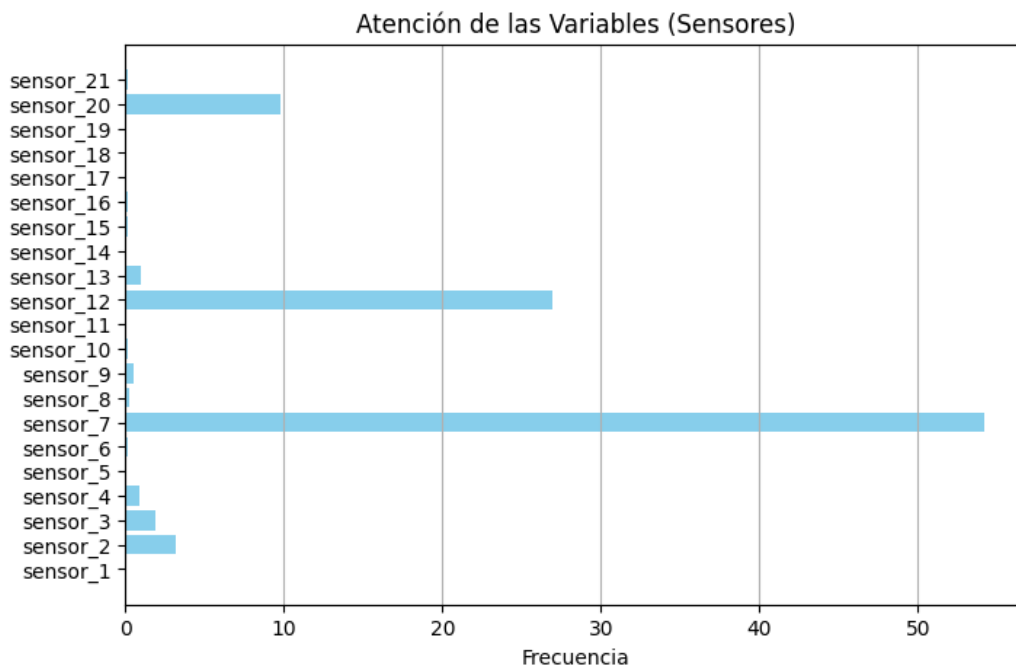


Figura 6. 2. Histograma de las atenciones de las variables de FD001 usando RUL-LAXCAT y simulado con todos los sensores.

Observando la Figura 6.2 se puede determinar cuáles son los sensores más relevantes a la hora de hacer la estimación de RUL. Se procede entonces a la simulación del conjunto FD001 con distintas cantidades de sensores. Primero se simula con 14 sensores, los que muestran algo de atención. Después se va reduciendo la cantidad:

- Los **5 mejores**: sensor 7, sensor 12, sensor 20, sensor 2 y sensor 3.
- Los **9 mejores**: sensor 7, sensor 12, sensor 20, sensor 2, sensor 3, sensor 13, sensor 4, sensor 21 y sensor 9.
- Los **14 mejores**: sensor 7, sensor 12, sensor 20, sensor 2, sensor 3, sensor 13, sensor 4, sensor 21, sensor 9, sensor 8, sensor 15, sensor 10, sensor 6 y sensor 16.

Después de hacer estas simulaciones se obtienen los resultados mostrados en la Tabla 6.3, siendo el RMSE la métrica usada para determinar la precisión de la simulación y el ATT el tiempo medio de entrenamiento por época (para 100 épocas).

|         | Número Sensores |            |           |              |
|---------|-----------------|------------|-----------|--------------|
|         | Todos           | 14 mejores | 9 mejores | 5 mejores    |
| RMSE    | <b>14.38</b>    | 15.96      | 15.86     | 18.81        |
| ATT (s) | 63.19           | 46.45      | 30.63     | <b>11.45</b> |

Tabla 6. 3. Resultados de las simulaciones con distinto número de sensores del conjunto FD001 usando RUL-LAXCAT.

Los resultados muestran una relación directa entre el número de sensores y el ATT, como era de esperar. Sin embargo, cabría esperar también que, a mayor número de sensores, menor RMSE. Esto se cumple en general, pero el RMSE al simular con 9 sensores es menor que el de la simulación con 14. Gracias a las explicaciones obtenidas al simular con todos los sensores, y poder hacer simulaciones con los sensores más relevantes, podemos obtener resultados similares o incluso mejores al simular con menos sensores.

La tabla también muestra como el menor RMSE se obtiene al simular con todos los sensores. Sin embargo, al simular solo con 9, obtenemos un decremento de RMSE de tan solo 10.3%, con un ATT del 48.5%. Con el objetivo de tener un tiempo de entrenamiento razonable, se decide hacer las siguientes simulaciones con los 9 mejores sensores.

#### Simulación Hiperparámetros “Batch Size” y “Learning Rate”

Con el objetivo de mejorar la precisión del modelo, se hacen distintas simulaciones cambiando los valores de los siguientes hiperparámetros [89]:

- **“Batch size”**: Se refiere al número de ejemplos de entrenamiento que se utilizan en una sola iteración de entrenamiento antes de que se actualicen los parámetros del modelo. En otras palabras, es el número de muestras que la red neuronal procesa antes de realizar una actualización en los pesos a través del algoritmo de optimización.
- **“Learning Rate”**: Determina el tamaño de los pasos que el algoritmo de optimización da al ajustar los pesos de la red durante el proceso de entrenamiento. En otras palabras, controla cuánto se modifican los pesos de la red en respuesta al gradiente del error en cada iteración de entrenamiento.

Después de hacer varias simulaciones cambiando los valores de ambos hiperparámetros (se usan valores típicos), se obtienen los siguientes resultados:

|               | Batch Size   |         |       |         |       |              |
|---------------|--------------|---------|-------|---------|-------|--------------|
|               | 32           |         | 64    |         | 128   |              |
| Learning Rate | RMSE         | ATT (s) | RMSE  | ATT (s) | RMSE  | ATT (s)      |
| 0.01          | 29.75        | 32.34   | 37.13 | 29.50   | 32.77 | 27.80        |
| 0.001         | <b>15.86</b> | 30.63   | 16.38 | 26.66   | 16.23 | 26.16        |
| 0.0001        | 18.82        | 30.58   | 22.02 | 26.94   | 25.67 | <b>25.73</b> |

Tabla 6. 4. Simulaciones con distintos valores de “batch size” y “learning rate” del conjunto de datos FD001 usando RUL-LAXCAT.

Observamos en la Tabla 6.4 que el mejor valor de RMSE se obtuvo con un “batch size” de 0.001 y un “learning rate” de 32. Por otro lado, el mejor ATT se obtuvo con un “batch size” de 0.0001 y un “learning rate” de 128.

Dado que, por bastante diferencia, el mejor RMSE se obtiene con un “learning rate” de 0.001, se determina que este será su valor para el resto de las simulaciones.

Por otro lado, se observa que cuanto mayor “batch size”, menor ATT. Sin embargo, al ser el tiempo de entrenamiento razonable en todos los casos, se opta por minimizar el RMSE. Por lo tanto, se decide que se usará de ahora en adelante un “batch size” de 32.

De ahora en adelante, para el resto de los conjuntos de datos, se usarán también estos valores de “batch size” y “learning rate”.

### Resultados Finales

Después de determinar la simulación que mejor equilibrio entre RMSE y ATT guarda, se presentan los resultados de la simulación con los mejores 9 sensores y valores de “batch size” y “learning rate” de 32 y 0.001 respectivamente.

- **ATT:** 30.63 segundos.
- **RMSE:** toma un valor de 15.86. Además, como se observa en la Figura 6.3, tiene una distribución sesgada a la derecha, lo que sugiere que la mayoría de los errores son relativamente pequeños, pero hay algunos errores significativamente mayores que aumentan la media y sesgan la distribución.

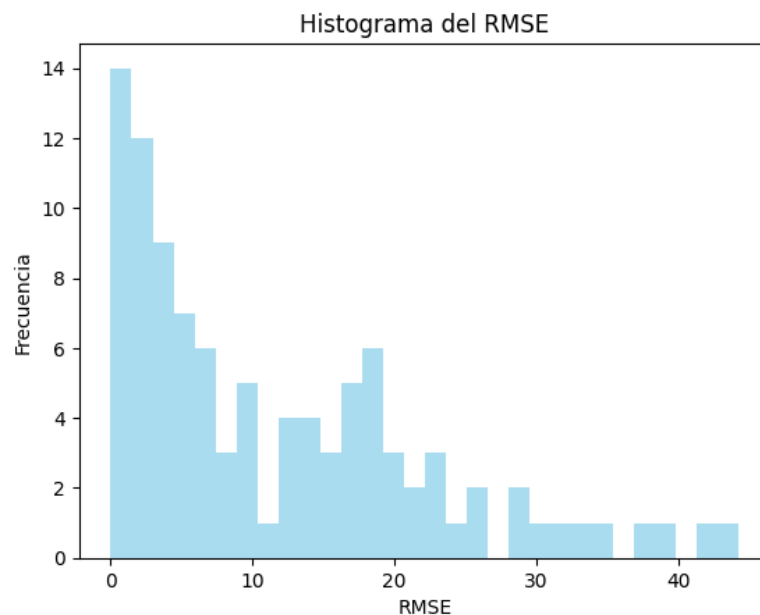


Figura 6. 3. Histograma del RMSE del conjunto de entrenamiento de FD001 usando RUL-LAXCAT.

- **Atenciones variables:** se muestran en la Figura 6.4.

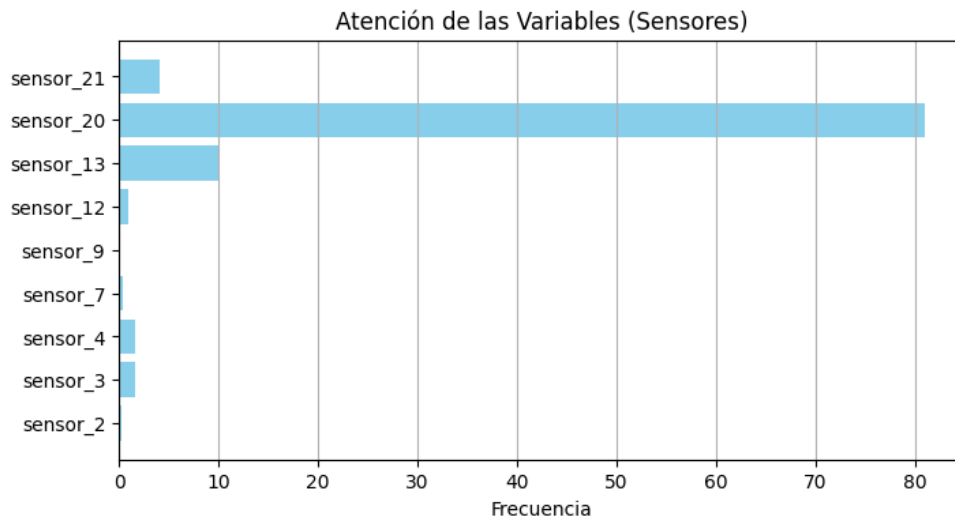


Figura 6. 4. Histograma de las atenciones de las variables de FD001 usando RUL-LAXCAT y simulado con 9 sensores.

Se observa en el mapa de calor que los sensores con más atención son el 20, 21 y 13. Pese a no estar entre los 5 mejores sensores, el 21 y el 13 ganan atención al simular con los mejores 9.

- **Atenciones cruzadas:** se muestran en la Figura 6.5.

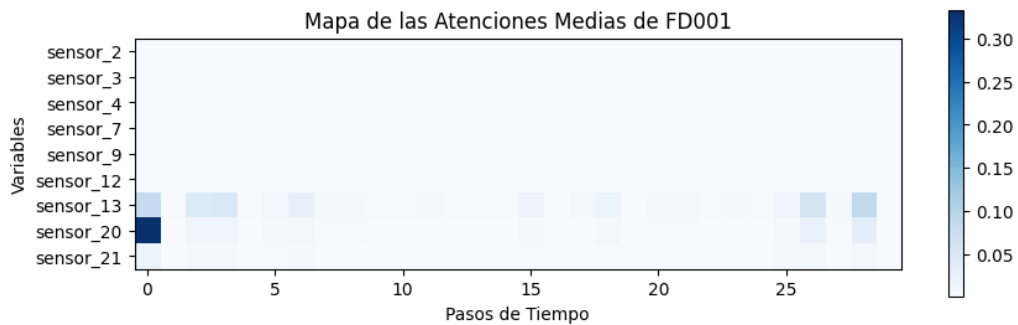


Figura 6. 5. Mapa de calor de las atenciones cruzadas de FD001 usando RUL-LAXCAT y simulado con 9 sensores.

Este mapa de calor muestra las atenciones cruzadas de cada variable y los instantes temporales. El sensor 20 en el instante 0 se lleva casi toda la atención.

- **RUL real vs. estimada:** esta gráfica se obtiene para el conjunto de datos de validación, como se explicó anteriormente. En el caso de FD001 hay 20 trayectorias de validación, que se muestran en la Figura 6.6. En azul, se muestra la degradación real de RUL, en naranja las predicciones.

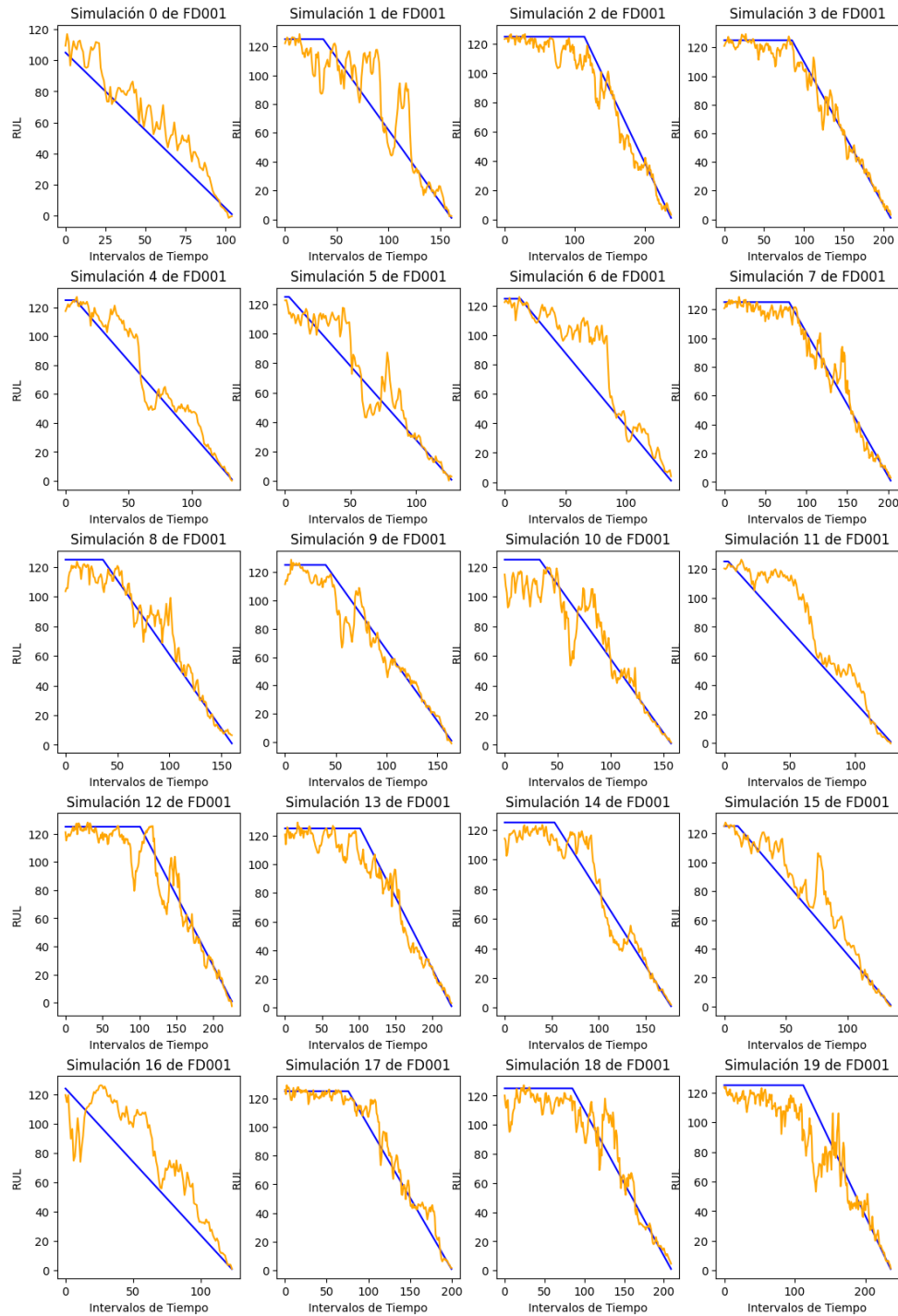


Figura 6. 6. RUL real vs. estimada de todas las trayectorias de validación de FD001 usando RUL-LAXCAT.

Como puede observarse en la Figura 6.6 hay trayectorias con una estimación más precisa (simulaciones 3, 7 y 14), pero en general todas las predicciones se acercan a los datos reales.

### 6.3.2. FD002

En primer lugar, se simuló el conjunto de datos FD002 con todos los sensores. Sin embargo, el modelo no mostró ningún aprendizaje, ya que las predicciones no se aproximaban a los datos reales. Por lo tanto, se optó por simular utilizando los mismos sensores que en RUL-RVE (véase la Sección 6.4): sensor 3, sensor 4, sensor 7, sensor 11 y sensor 12. De igual forma, el modelo no demostró aprendizaje. Entonces, se probaron distintas combinaciones de valores para "batch size" (32, 64 y 128) y "learning rate" (0.1, 0.01 y 0.001), pero sin obtener resultados significativos. En la Figura 6.7 se muestran dos ejemplos de los resultados obtenidos para diferentes hiper-parámetros.

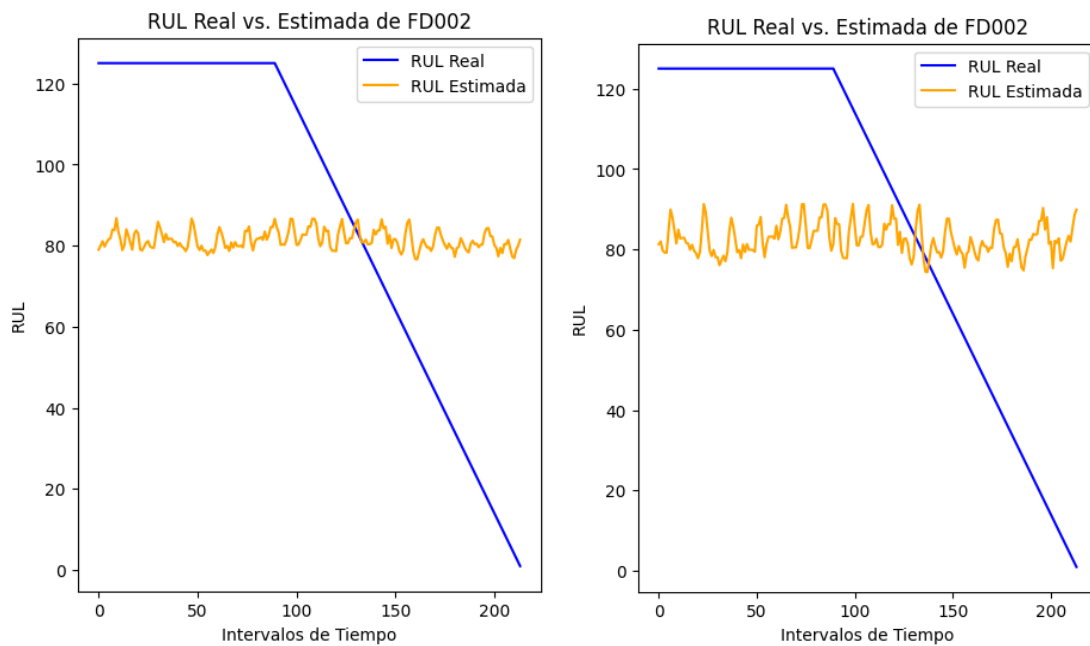


Figura 6. 7. Simulación 12 para "batch size" 32 y "learning rate" 0.001 (izquierda), y para "batch size" 64 y "learning rate" 0.01.

### 6.3.3. FD003

Al igual que se hizo con el conjunto de datos FD001, en primer lugar se simula el conjunto de datos FD003 con todos los sensores para determinar cuáles son los más relevantes en base a las explicaciones. Los resultados son los siguientes:

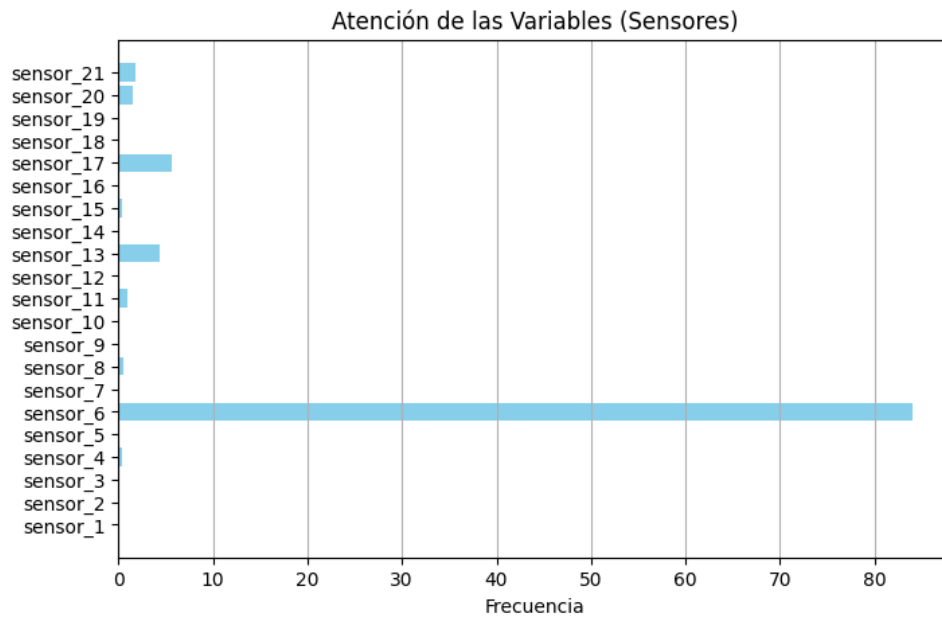


Figura 6. 8. Histograma de las atenciones de las variables de FD003 usando RUL-LAXCAT y simulado con todos los sensores.

En la Figura 6.8 se observa que los 9 sensores más relevantes son: sensor 4, sensor 6, sensor 8, sensor 11, sensor 13, sensor 15, sensor 17, sensor 20, sensor 21. Se procede entonces a simular con estos datos y se obtienen los siguientes resultados:

- **ATT:** 38.71 segundos frente a los 79.91 de simular con todos los sensores.
- **RMSE:** toma un valor de 16.82 frente a los 13.97 de simular con todos los sensores. Además, como se observa en la Figura 6.9, al igual que en el conjunto FD001, tiene una distribución sesgada a la derecha.

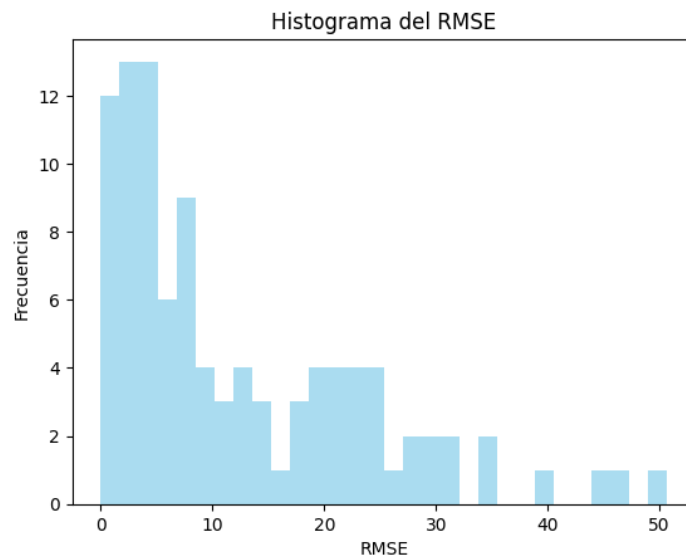


Figura 6. 9. Histograma del RMSE del conjunto de entrenamiento de FD003 usando RUL-LAXCAT.

- **Atenciones variables:** se muestran en la Figura 6.10.

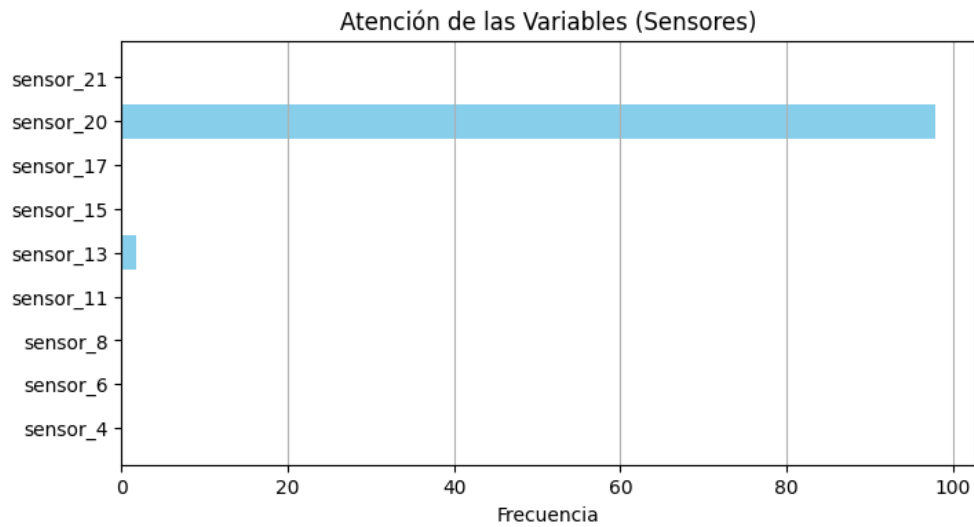


Figura 6. 10. Histograma de las atenciones de las variables de FD003 usando RUL-LAXCAT y simulado con 9 sensores.

Se observa en el mapa de calor que el sensor con más atención son el 20 y 13.

- **Atenciones cruzadas:** se muestran en la Figura 6.11.

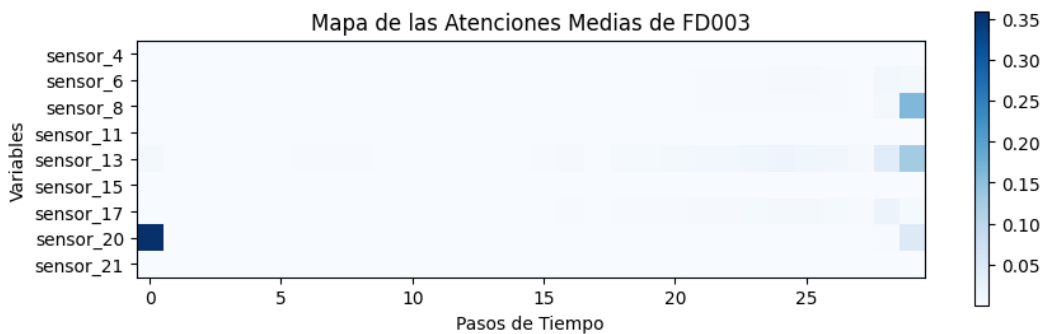


Figura 6. 11. Mapa de calor de las atenciones cruzadas de FD003 usando RUL-LAXCAT y simulado con 9 sensores.

El sensor 20 en el instante 0 se lleva casi toda la atención en este caso, aunque también hay un foco de atención importante en el instante 30 para los sensores 8 y 13.

- **RUL real vs. estimada:** En el caso de FD003, al igual que en FD001, hay 20 trayectorias de validación, mostradas en la Figura D.1 del Apéndice D. A continuación, en la Figura 6.12 se muestran las trayectorias 12 y 14.

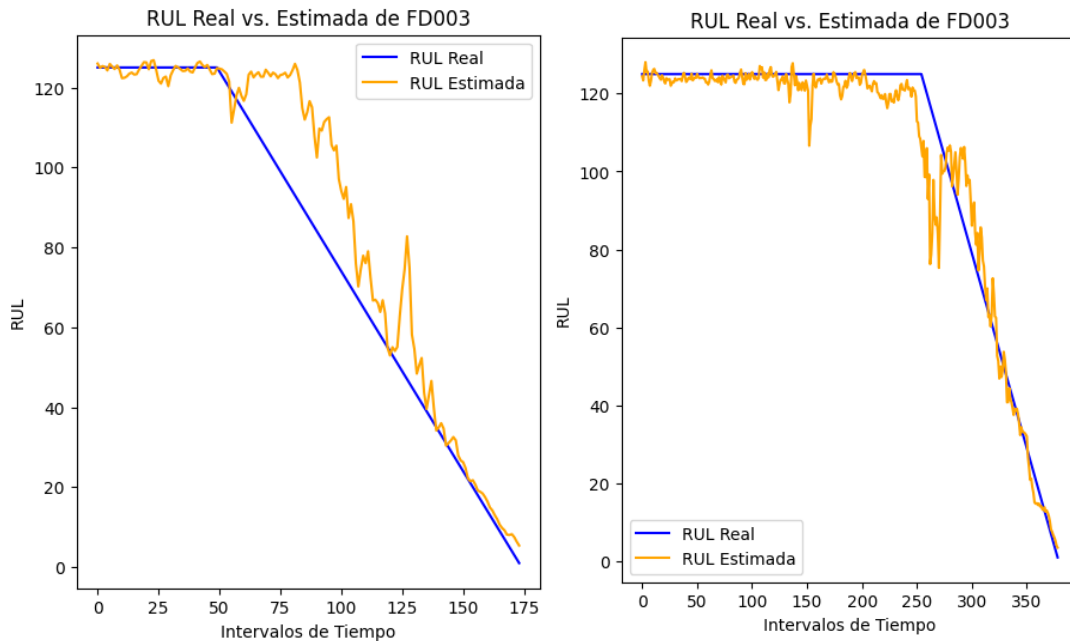


Figura 6. 12. RUL real vs. estimada de las trayectorias 12 y 14 de validación de FD003 usando RUL-LAXCAT.

#### 6.3.4. FD004

En el caso del conjunto de datos FD004, ocurre igual que con FD002: el modelo no da buenos resultados. En la Figura 6.13 se muestran 2 ejemplos de estos resultados.

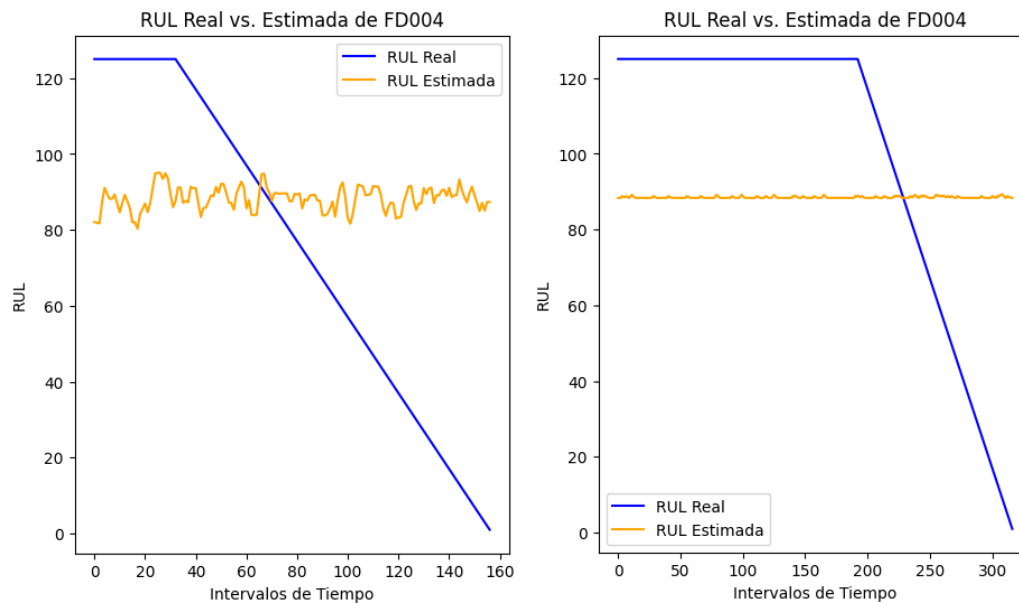


Figura 6. 13. Simulación del conjunto de datos FD004 con 5 sensores (izquierda) y con todos los sensores (derecha).

## 6.4. Simulaciones en RUL-RVE

Las simulaciones del modelo RUL-RVE se llevan a cabo con el código proporcionado por los propios creadores en su repositorio de GitHub [86].

Las simulaciones se hacen con los mismos 5 sensores para todos los conjuntos de datos: sensor 3, sensor 4, sensor 7, sensor 11 y sensor 12.

### 6.3.1. FD001

Al simular el conjunto de datos FD001 con los sensores anteriormente mencionados, se obtienen los siguientes resultados:

- **ATT:** 43.48 segundos.
- **RMSE:** 30.10 para las trayectorias de prueba.
- **Espacio latente:** En la Figura 6.14 se muestra el espacio latente generado por las trayectorias de entrenamiento (izquierda) y prueba (derecha).

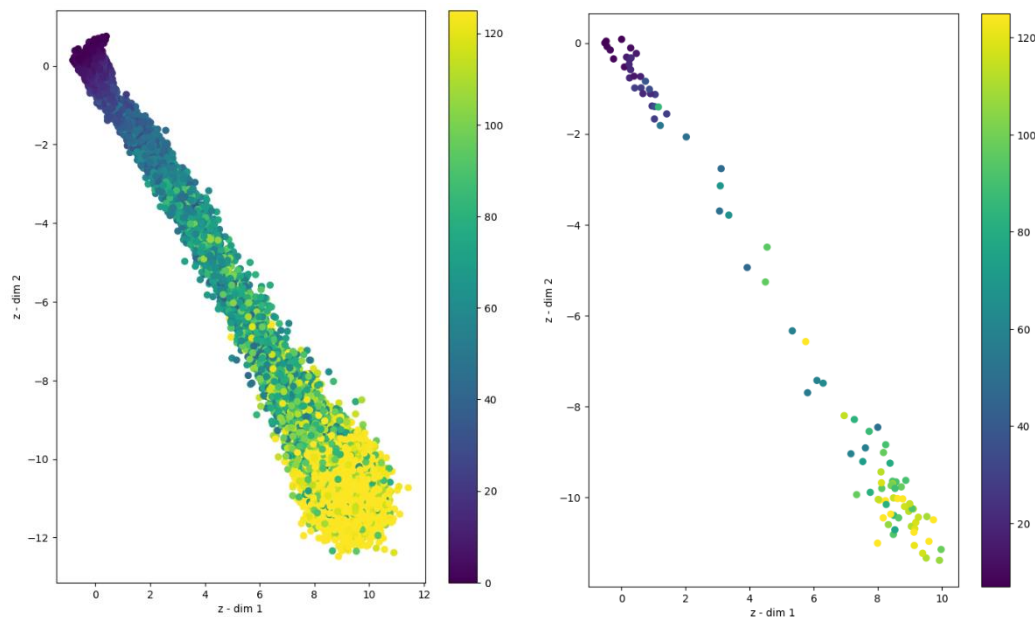


Figura 6. 14. Espacio latente de los conjuntos de entrenamiento y prueba de FD001 usando RUL-RVE.

La imagen del espacio latente sugiere una progresión lineal de la degradación del sistema, donde los datos muestran un desgaste gradual y continuo. Los puntos morados representan estados con alta vida útil remanente, mientras que los puntos amarillos indican una degradación avanzada. Esta visualización muestra que la degradación es uniforme y predecible a lo largo del tiempo. Ambas representaciones siguen la misma degradación porque son simulaciones de un mismo sistema.

- **RUL real vs. estimada:** se muestra en la Figura 6.15.

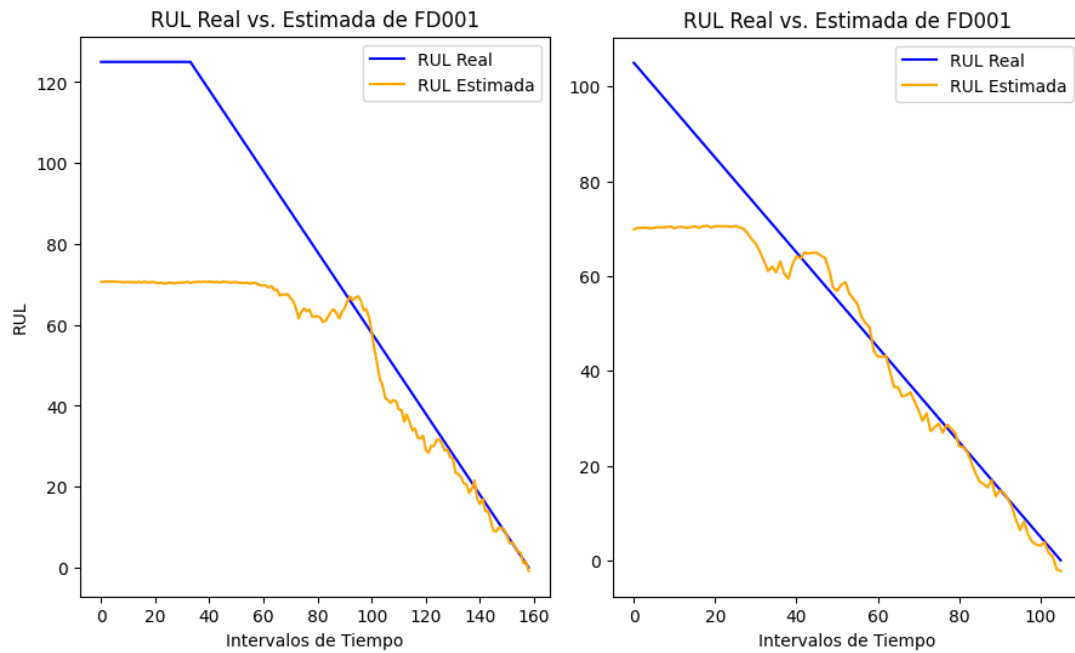


Figura 6. 15. Trayectorias de validación 10 y 20 de FD001 usando RUL-RVE.

Se observa que para ambas trayectorias existe algún tipo de limitación en las predicciones cerca del valor 70, lo que explica un RMSE tan alto de 30.10.

### 6.3.2. FD002

Se simula el conjunto de datos FD002 con los 5 sensores anteriormente mencionados, y se obtienen los siguientes resultados:

- **ATT:** 70.93 segundos.
- **RMSE:** 15.14 para las trayectorias de prueba.

- **Espacio latente:** En la Figura 6.16 se muestra el espacio latente generado por las trayectorias de entrenamiento (izquierda) y prueba (derecha).

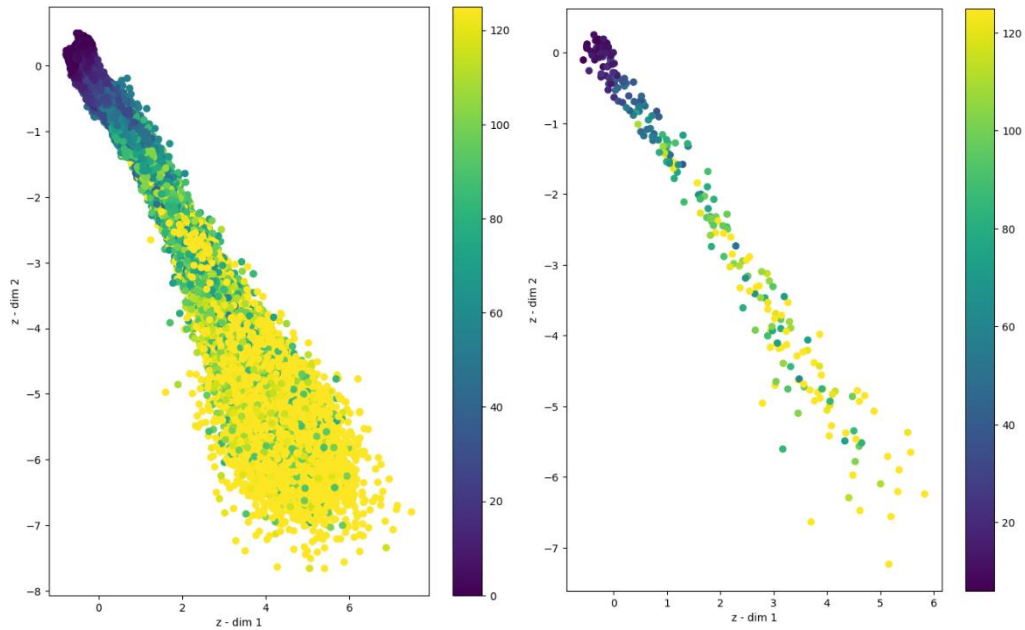


Figura 6. 16. Espacio latente de los conjuntos de entrenamiento y prueba de FD002 usando RUL-RVE.

La imagen del espacio latente muestra una degradación acelerada del sistema, donde los datos sugieren un desgaste que se intensifica rápidamente. Esto sugiere que el sistema experimenta un deterioro significativo en un corto periodo de tiempo.

- **RUL real vs. estimada:** se muestra en la Figura 6.17.

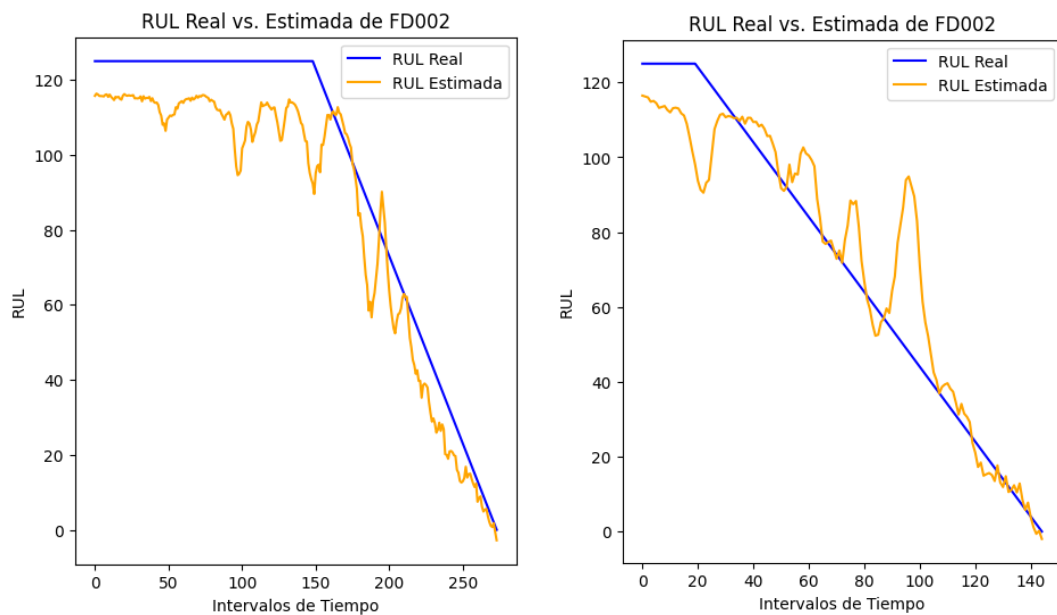


Figura 6. 17. Trayectorias de validación 2 y 47 de FD002 usando RUL-RVE.

En este caso, se observa que no existe ninguna limitación en las predicciones como ocurría con el conjunto FD001. Por eso mismo, en este caso el RMSE tienen un valor de 15.14, aunque se pueden apreciar mayores fluctuaciones en la RUL estimada.

### 6.3.3. FD003

Se simula el conjunto de datos FD003 con los 5 sensores mencionados, y se obtienen los siguientes resultados:

- **ATT:** 30.05 segundos.
- **RMSE:** 23.87 para las trayectorias de prueba.
- **Espacio latente:** En la Figura 6.18 se muestra el espacio latente generado por las trayectorias de entrenamiento (izquierda) y prueba (derecha).

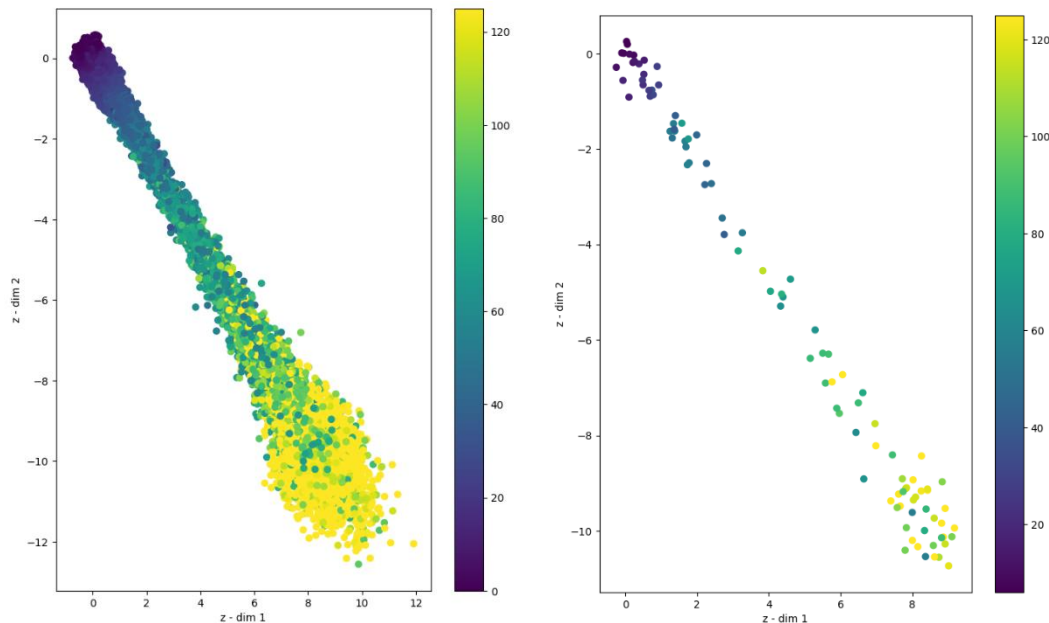


Figura 6. 18. Espacio latente de los conjuntos de entrenamiento y prueba de FD003 usando RUL-RVE.

Al igual que en el conjunto FD001, la imagen del espacio latente sugiere una degradación uniforme y gradual del sistema.

- **RUL real vs. estimada:** se muestra en la Figura 6.19.

Observando las gráficas, se aprecia como, al igual que en el conjunto FD001, existe algún tipo de sesgo cerca del valor 80. Es por eso por lo que se obtiene un RMSE alto de 23.87.

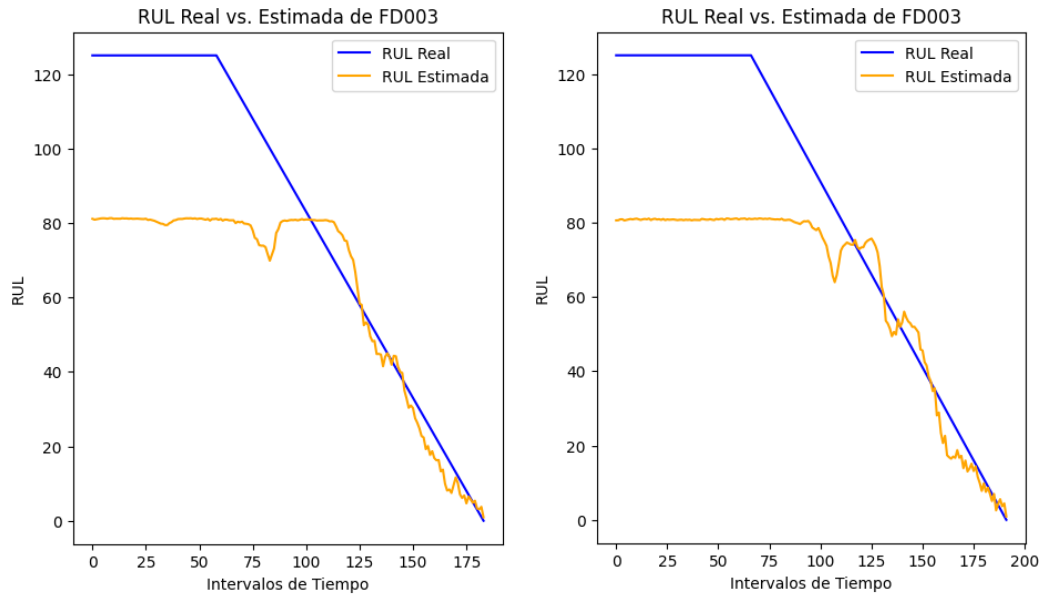


Figura 6. 19. Trayectorias de validación 2 y 17 de FD003 usando RUL-RVE.

#### 6.3.4. FD004

Se simula el conjunto de datos FD004 con los 5 sensores mencionados, y se obtienen los siguientes resultados:

- **ATT:** 83.70 segundos.
- **RMSE:** 17.47 para las trayectorias de prueba.
- **Espacio latente:** En la Figura 6.20 se muestra el espacio latente generado por las trayectorias de entrenamiento (izquierda) y prueba (derecha).

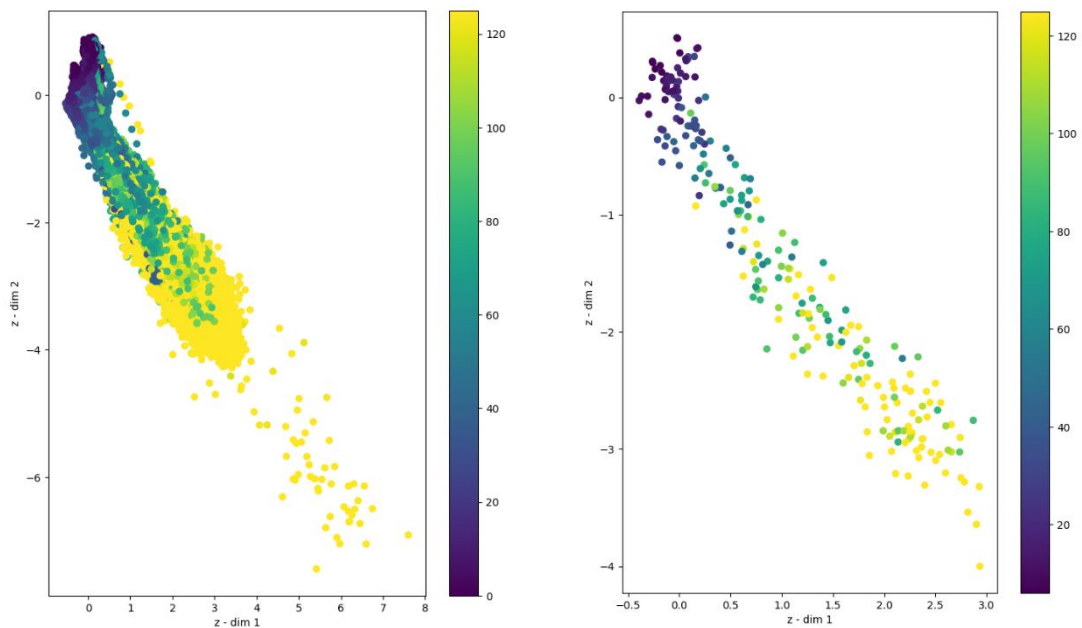


Figura 6. 20. Espacio latente de los conjuntos de entrenamiento y prueba de FD004 usando RUL-RVE.

La imagen del espacio latente muestra una degradación del sistema que se acelera y se dispersa hacia etapas avanzadas de desgaste. Esto sugiere que el sistema experimenta un deterioro acelerado y menos uniforme a medida que se aproxima al final de su vida útil.

- **RUL real vs. estimada:** se muestra en la Figura 6.21.

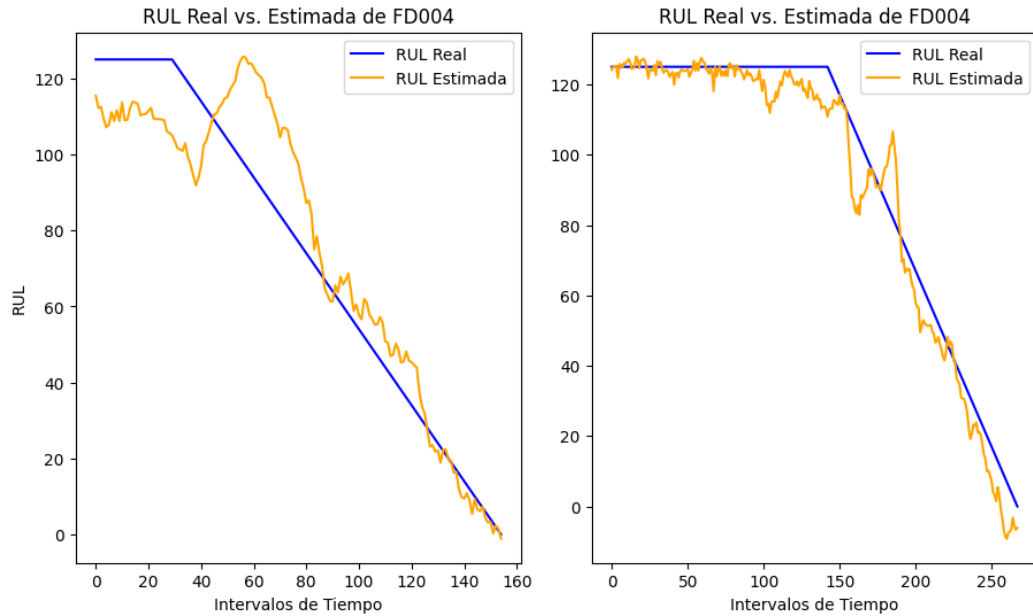


Figura 6. 21. Trayectorias de validación 24 y 39 de FD004 usando RUL-RVE.

En este caso ocurre lo mismo que en FD002, no hay ningún tipo de sesgo. Por lo tanto, la precisión es bastante buena, con un RMSE de 17.47.

## 6.5. Comparación de los Modelos

### 6.5.1. Precisión y Tiempo de Entrenamiento

Como se ha explicado anteriormente, para medir la precisión de los modelos se calcula el valor RMSE de las predicciones. En la Tabla 6.5 podemos ver una comparación de ambos modelos para los distintos sub-conjuntos de datos, teniendo en cuenta el RMSE y el ATT.

| Modelo     | Sub-conjunto de Datos |              |              |              |              |              |              |              |
|------------|-----------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|            | FD001                 |              | FD002        |              | FD003        |              | FD004        |              |
|            | RMSE                  | ATT          | RMSE         | ATT          | RMSE         | ATT          | RMSE         | ATT          |
| RUL-RVE    | 30.10                 | 43.48        | <b>15.14</b> | <b>70.93</b> | 23.87        | <b>30.05</b> | <b>17.47</b> | <b>83.70</b> |
| RUL-LAXCAT | <b>15.86</b>          | <b>30.63</b> | N.D.         | N.D.         | <b>16.82</b> | 38.71        | N.D.         | N.D.         |

Tabla 6. 5. Comparación de las simulaciones de RUL-LAXCAT y RUL-RVE.

RUL-LAXCAT obtiene mejores resultados en los conjuntos de datos FD001 y FD003, debido a las limitaciones que ambos conjuntos presentan al simular con RUL-RVE, como hemos

visto en las gráficas. Sin embargo, para los conjuntos FD002 y FD004, RUL-LAXCAT no proporciona ningún resultado, mientras que RUL-RVE ofrece muy buenas prestaciones.

En conclusión, se puede afirmar que RUL-LAXCAT no funciona en condiciones operativas variables, como ocurre en FD002 y FD004, pero ofrece buenas prestaciones en condiciones operativas constantes, como en FD001 y FD003.

### **6.5.2. Calidad de las Explicaciones**

Las explicaciones proporcionadas por los modelos RUL-RVE y RUL-LAXCAT son complementarias y no se pueden comparar directamente, ya que cada modelo utiliza diferentes metodologías para proporcionar información sobre las predicciones de RUL. Ambos enfoques ofrecen beneficios únicos, que se adaptan a distintas necesidades y perspectivas en el análisis de datos y el mantenimiento predictivo.

El modelo RUL-RVE se destaca por su capacidad de proyectar datos en un espacio latente continuo y bien regularizado, lo que facilita la visualización intuitiva de la degradación del sistema y mejora la confianza en las predicciones, aunque la necesidad de conocimientos especializados para interpretar las visualizaciones pueden ser una desventaja.

Por otro lado, RUL-LAXCAT utiliza técnicas de atención a variables y temporal, permitiendo enfocarse en la información más relevante y proporcionando explicaciones claras sobre los sensores más relevantes. Sin embargo, la falta de resultados para algunos subconjuntos de datos puede limitar su robustez y aplicabilidad en comparación con RUL-RVE.

## **6.6. Discusión**

Los resultados presentados en este capítulo destacan varios puntos clave en el análisis y la comparación de los modelos RUL-LAXCAT y RUL-RVE. Una de las conclusiones más relevantes es la diferencia en el rendimiento de estos modelos según el tamaño y las condiciones operativas de los sub-conjuntos de datos. Mientras que RUL-LAXCAT mostró un rendimiento superior en conjuntos de datos más pequeños y con menos variables, RUL-RVE demostró una mayor eficacia en conjuntos más grandes y con condiciones operativas más complejas. Esto sugiere que la elección del modelo puede depender significativamente del contexto y del tipo de datos disponibles, aunque son necesarias más simulaciones para verificarlo.

Otra observación importante es la relación entre el número de sensores utilizados y la precisión de las predicciones. Aunque en general un mayor número de sensores resulta en un menor RMSE, los resultados indican que un enfoque más selectivo con un número óptimo de sensores puede ofrecer una precisión comparable o incluso mejor, con tiempos de entrenamiento significativamente reducidos. Esta eficiencia en el uso de sensores no solo optimiza el rendimiento del modelo, sino que también facilita una implementación más práctica y menos costosa en aplicaciones reales.

## **Capítulo 7: Conclusiones y Líneas Futuras**

### **7.1. Introducción**

El presente capítulo tiene como objetivo sintetizar las conclusiones más relevantes obtenidas a lo largo del desarrollo del proyecto y proponer posibles líneas de trabajo futuras. A lo largo de los capítulos anteriores, se han abordado diversos aspectos relacionados con la estimación de la vida útil remanente (RUL) mediante modelos intrínsecamente interpretables, así como la implementación y comparación de los algoritmos RUL-RVE y RUL-LAXCAT.

### **7.2. Conclusiones**

El proyecto ha permitido obtener varias conclusiones significativas sobre el desempeño y aplicabilidad de los modelos desarrollados:

#### **1. Desempeño de los Modelos:**

- Los modelos RUL-RVE y RUL-LAXCAT mostraron rendimientos diferenciados dependiendo del tamaño y las condiciones operativas de los conjuntos de datos. En condiciones operativas constantes (FD001 y FD003), RUL-LAXCAT mostró un mejor desempeño con menores valores de RMSE. Sin embargo, en condiciones operativas variables (FD002 y FD004), RUL-RVE fue el único que ofreció resultados fiables.
- El análisis comparativo demostró que mientras RUL-LAXCAT es más adecuado para escenarios donde las condiciones operativas son estables, RUL-RVE ofrece mayor robustez en entornos dinámicos y variables.

#### **2. Calidad de las Explicaciones:**

- Ambos modelos proporcionaron explicaciones útiles, pero de naturaleza complementaria. RUL-RVE permite una visualización continua de la degradación del sistema en un espacio latente, mejorando la confianza en las predicciones. Esto se logra mediante la representación de los estados de degradación como puntos en un espacio bidimensional, lo que facilita la comprensión de la progresión del deterioro del sistema.
- Por su parte, RUL-LAXCAT emplea técnicas de atención a variables y temporal para identificar los sensores más relevantes. Esto permite destacar cuáles son las características más críticas en cada momento del ciclo de vida del componente, proporcionando una explicación detallada y comprensible de las decisiones del modelo. Sin embargo, su robustez se ve limitada en algunos subconjuntos de datos, especialmente aquellos con variabilidad operacional.

**3. Eficiencia:**

- La optimización en el número de sensores demostró ser clave para reducir los tiempos de entrenamiento. Un enfoque selectivo en el uso de sensores no solo optimiza el rendimiento del modelo, sino que también facilita una implementación más práctica y menos costosa en aplicaciones reales.
- La reducción del número de sensores no afecta significativamente a la precisión del modelo, pero sí impacta positivamente en la reducción de costes operativos y de implementación.

**4. Impacto en la Toma de Decisiones:**

- La capacidad de los modelos para proporcionar explicaciones claras y comprensibles es crucial para la toma de decisiones en entornos industriales. Los ingenieros y responsables de mantenimiento pueden utilizar estas explicaciones para justificar intervenciones y optimizar la planificación de mantenimiento, lo que a su vez reduce los tiempos de inactividad y mejora la eficiencia operativa.
- La confianza en los modelos se incrementa al poder entender las razones detrás de las predicciones. Esto es especialmente importante en sectores críticos donde la seguridad y la precisión son primordiales.

**5. Limitaciones Identificadas:**

- Aunque los modelos han mostrado resultados prometedores, también se han identificado limitaciones, especialmente en cuanto a la generalización en condiciones operativas altamente variables. Esto resalta la necesidad de continuar investigando y mejorando estos modelos para ampliar su aplicabilidad y robustez.

### **7.3. Líneas Futuras**

Las siguientes líneas de trabajo se presentan como posibles continuaciones y mejoras al trabajo desarrollado en este proyecto:

**1. Ampliación del Conjunto de Datos:**

- **Diversificación de Datos:** Realizar más simulaciones con conjuntos de datos más variados y en diferentes condiciones operativas para evaluar la robustez y generalización de los modelos. Esto incluye la recopilación y utilización de datos de distintos tipos de maquinaria y en diferentes sectores industriales.

2. **Mejora de la Interpretabilidad:**

- **Nuevas Técnicas de Explicabilidad:** Desarrollar nuevas metodologías que integren técnicas de explicabilidad tanto *ante-hoc* como *post-hoc* para mejorar la comprensión y confianza en los modelos predictivos.
- **Visualización Avanzada:** Mejorar las herramientas de visualización de las explicaciones para hacerlas más intuitivas y accesibles a los usuarios no técnicos. Esto podría incluir el desarrollo de “dashboards” interactivos y gráficos dinámicos que faciliten la interpretación de los resultados.

3. **Exploración de Nuevas Arquitecturas:**

- **Redes Prototípicas:** Investigar nuevas arquitecturas de redes neuronales que puedan ofrecer un mejor balance entre precisión y explicabilidad, como redes prototípicas o enfoques híbridos que combinen distintas metodologías de machine learning.

4. **Optimización de Algoritmos:**

- **Algoritmos de Optimización Avanzada:** Investigar y aplicar técnicas de optimización más avanzadas que puedan mejorar aún más el rendimiento de RUL-LAXCAT, así como su aplicabilidad en tiempo real.

5. **Integración en Entornos Industriales:**

- **Validación en la Industria:** Probar los modelos en entornos industriales reales para validar su aplicabilidad y eficacia en situaciones prácticas, además de realizar ajustes necesarios basados en el “feedback” de estas implementaciones.
- **Desarrollo de Interfaces de Usuario:** Crear interfaces de usuario intuitivas y eficientes que permitan a los ingenieros y técnicos interactuar fácilmente con los modelos y entender sus predicciones y explicaciones.

## 7.4. Discusión

El desarrollo de modelos intrínsecamente interpretables para la estimación de vida útil remanente ha demostrado ser una herramienta valiosa para la gestión de mantenimiento predictivo. La capacidad de estos modelos para proporcionar predicciones precisas y explicaciones claras facilita la toma de decisiones informadas y mejora la confianza en las soluciones implementadas. Sin embargo, es crucial continuar explorando y perfeccionando estas técnicas para adaptarlas a la variedad de escenarios y necesidades industriales.

El trabajo realizado en este proyecto sienta las bases para futuras investigaciones y desarrollos en el campo de la inteligencia artificial explicable aplicada al mantenimiento predictivo.



## Apéndice A: Presupuesto

En este apéndice se ofrecerá una estimación de los costes que conlleva la realización de este proyecto. Se diferencian 2 grandes grupos de costes: medios materiales y recursos humanos.

### A.1. Coste de Materiales

A continuación, en la Tabla A.1, se muestran los costes del material utilizado durante el desarrollo del proyecto, ascendiendo el total a **1.742€**.

| Concepto                                  | Total (€)    |
|-------------------------------------------|--------------|
| Ordenador MSI GF63 Thin 10SCXR            | 949          |
| Pantalla TTL 12475PXQU                    | 250          |
| Teclado HP                                | 30           |
| Ratón Logitech                            | 25           |
| Herramientas de desarrollo: Visual Studio | 0            |
| Herramientas de desarrollo: Python        | 0            |
| Sistema operativo Windows 11 Pro          | 259          |
| Microsoft Office 365                      | 69           |
| Base de datos NASA Turbofan Jet Engine    | 0            |
| Impresiones                               | 160          |
| <b>Total</b>                              | <b>1.742</b> |

Tabla A. 1. Coste de materiales.

### A.2. Coste de Personal

Según indican algunas fuentes [90, 91], el salario medio de un ingeniero recién graduado en España es de 25.000€ anuales a jornada completa. Teniendo en cuenta que hay 1820 horas laborales en un año, un recién graduado en Ingeniería y Sistemas de Datos puede cobrar en torno a 13,5€ la hora. En la tabla A.2 se puede ver a cuanto ascendería el coste total del recién graduado teniendo en cuenta que ha trabajado 320 horas.

Además, para el desarrollo de este proyecto ha sido necesaria la colaboración de un tutor y un director de PFG. Se estima que ambos profesionales cobrarán un 7,5% cada uno de la suma del coste de materiales y la mano de obra. En la Tabla A.2 están representados ambos sueldos, ascendiendo el total del coste de personal a **4.968€**.

| Concepto           | Cantidad | Coste unitario (€) | Total (€)    |
|--------------------|----------|--------------------|--------------|
| Mano de obra       | 320      | 13,5               | 4.320        |
| Coste de dirección | 2        | 324                | 648          |
| <b>Total</b>       |          |                    | <b>4.968</b> |

Tabla A. 2. Coste de Personal.

### A.3. Coste Total

Teniendo en cuenta el coste del material y el coste del personal, según podemos ver en la Tabla A.3, el coste total del proyecto asciende a **6.710€**.

| Categoría            | Total (€)    |
|----------------------|--------------|
| Recursos Humanos     | 4.968        |
| Coste de Materiales  | 1.742        |
| <b>Total General</b> | <b>6.710</b> |

*Tabla A. 3. Coste total del Proyecto.*

## **Apéndice B: Impacto del Proyecto**

### ***B.1. Introducción***

El desarrollo de modelos intrínsecamente interpretables para la estimación de vida útil remanente tiene implicaciones significativas en múltiples áreas. En este apéndice, se abordarán los impactos sociales, de salud y seguridad, económicos e industriales relacionados con el proyecto. Además, se explorará la contribución a los Objetivos de Desarrollo Sostenible (ODS) 3, 9 y 12 establecidos por las Naciones Unidas [92].

### ***B.2. Impacto Social***

El uso de modelos interpretables mejora la confiabilidad y transparencia en sectores como la medicina, la ingeniería y la manufactura. La interpretabilidad permite a los usuarios finales comprender las predicciones y tomar decisiones informadas, aumentando la confianza en las tecnologías basadas en inteligencia artificial.

En medicina, estos modelos pueden predecir el desgaste de equipos médicos, mejorando la gestión de recursos y asegurando la continuidad del tratamiento. Por ejemplo, saber cuándo es probable que un marcapasos necesite ser reemplazado puede prevenir emergencias médicas, evitar cirugías innecesarias y garantizar la seguridad del paciente. En ingeniería, la precisión en la predicción de fallos en componentes críticos previene accidentes y garantiza la seguridad. En manufactura, optimiza el uso de recursos y reduce el desperdicio, contribuyendo a la eficiencia operativa y la sostenibilidad ambiental.

La transparencia de los modelos interpretables promueve un uso responsable de la inteligencia artificial, asegurando que las decisiones se basen en un entendimiento claro de las predicciones y sus implicaciones. Esto es crucial para fomentar una cultura de responsabilidad y ética en el uso de tecnologías avanzadas, especialmente en áreas donde las decisiones pueden tener un impacto significativo en la vida de las personas.

### ***B.3. Impacto en la Salud y la Seguridad***

En sectores como la aviación y la energía, la capacidad de prever fallos en componentes críticos puede evitar accidentes y salvar vidas. La interpretabilidad de los modelos permite una mejor planificación y mitigación de riesgos, asegurando que los operadores puedan tomar medidas preventivas antes de que ocurran fallos graves.

La precisión en la estimación de vida útil permite programar mantenimientos de manera proactiva, evitando fallos inesperados. La identificación temprana de señales de desgaste en equipos industriales reduce el riesgo de accidentes laborales, protege a los trabajadores y asegura la continuidad operativa. Por ejemplo, prever el desgaste de una pieza en una línea de producción puede evitar accidentes laborales.

### **B.4. Impacto Económico**

La implementación de estos modelos permite evitar pérdidas de tiempo por inactividad y reparaciones inesperadas, optimizando los ciclos de vida de los productos y reduciendo costes operativos. Esto es especialmente relevante en industrias donde el tiempo de inactividad puede significar pérdidas financieras significativas.

La capacidad de prever con precisión la vida útil de componentes permite una mejor gestión de inventarios, optimizando el uso de recursos y mejorando la eficiencia operativa. Esto también contribuye a la sostenibilidad al reducir el desperdicio de materiales y la necesidad de producir componentes de repuesto innecesarios.

La transparencia en los modelos aumenta la confianza de los inversores y “stakeholders”, fortaleciendo la estabilidad económica de las empresas y fomentando la inversión en tecnologías avanzadas. La capacidad de demostrar cómo y por qué se toman decisiones basadas en modelos predictivos puede atraer inversiones y fortalecer la reputación de la empresa en el mercado.

### **B.5. Impacto Tecnológico e Industrial**

Los modelos interpretables impulsan la innovación en inteligencia artificial, promoviendo el desarrollo de técnicas y herramientas que buscan tanto la precisión como la comprensión de las predicciones. Esto fomenta la creación de tecnologías más avanzadas y accesibles, que pueden ser adoptadas en una gran variedad de industrias.

La adopción de modelos avanzados en la producción y el mantenimiento mejora la eficiencia y seguridad de los procesos industriales, beneficiando a una amplia gama de sectores.

### **B.6. Contribución a los Objetivos de Desarrollo Sostenible (ODS)**



*Figura B. 1. ODS 3: Salud y Bienestar, ODS 9: Industria, Innovación e Infraestructura, ODS 12: Producción y Consumo Responsables [92].*

#### **ODS 3: Salud y Bienestar**

Los modelos interpretables mejoran la seguridad y confiabilidad de los sistemas médicos, contribuyendo a la salud y bienestar general de la población. La capacidad de prever el

desgaste de equipos médicos y dispositivos implantables asegura que los pacientes reciban un cuidado continuo y seguro, reduciendo el riesgo de emergencias médicas y mejorando la calidad de vida.

#### **ODS 9: Industria, Innovación e Infraestructura**

Los modelos interpretables fomentan la innovación tecnológica y mejora de la infraestructura industrial mediante la adopción de métodos de predicción avanzados y confiables. La implementación de estas tecnologías puede llevar a procesos de producción más eficientes, seguros y sostenibles, impulsando el crecimiento económico y la competitividad de las industrias.

#### **ODS 12: Producción y Consumo Responsables**

Predecir la vida útil de los productos reduce el desperdicio, promoviendo un consumo y producción más sostenibles. La capacidad de gestionar de manera eficiente los recursos y predecir el desgaste de los componentes contribuye a la sostenibilidad ambiental, reduciendo la huella de carbono y el impacto negativo en el medio ambiente.

### ***B.7. Discusión***

El desarrollo de modelos intrínsecamente interpretables para la estimación de vida útil remanente tiene un impacto positivo y multifacético en la sociedad, la economía, la salud, la seguridad, y la tecnología. Desde mejorar la confianza y la transparencia en diversas industrias hasta promover un uso responsable y ético de la inteligencia artificial, estos modelos tienen el potencial de transformar la manera en que se toman decisiones críticas, con grandes beneficios para la sociedad.



## Apéndice C: Manual de Usuario

### C.1. RUL-RVE

El código del algoritmo RUL-RVE está explicado por los propios creadores en su repositorio de GitHub [86].

También, existe un archivo README.md donde explica los distintos archivos que existen.

### C.2. RUL-LAXCAT

El modelo RUL-LAXCAT tiene los siguientes archivos:

- **model.py:** Este archivo contiene la definición del modelo. En él, se definen tres clases: `input_attention_layer`, `temporal_attention_layer`, `LaxCat`.
- **training.py:** Este archivo se encarga de la preparación de datos, la creación y el entrenamiento del modelo.
  - **Carga de datos:** Solicita al usuario que seleccione un dataset y llama a la función `create_samples` para preparar los datos en conjuntos de entrenamiento, validación y prueba.
  - **Definición del modelo:** Importa la clase `LaxCat` desde `model.py` y la configura con los parámetros adecuados. Define el optimizador (Adam) y la función de pérdida (MSELoss).
  - **Entrenamiento del modelo:** Llama a la función `train` para entrenar el modelo con los datos de entrenamiento y validación.
  - **Guardado del modelo:** Guarda los pesos del modelo entrenado en la carpeta `models`.
- **predictions.ipynb:** Un cuaderno Jupyter que se utiliza para cargar el modelo entrenado y hacer predicciones.
  - **Carga de datos y preparación del modelo:** Carga el dataset seleccionado y prepara los datos en conjuntos de entrenamiento, validación y prueba. Define los parámetros del modelo y lo configura utilizando la clase `LaxCat` de `model.py`.
  - **Carga de pesos del modelo:** Carga los pesos del modelo entrenado previamente guardados.
  - **Evaluación del modelo:** Llama a la función `eval_model` para evaluar el modelo y obtener las atenciones.

- **Visualización de resultados:** Utiliza funciones como `show attentions` y `show_RUL_graph` para visualizar las atenciones y las predicciones del modelo.
- **utils.py:** Contiene funciones auxiliares.
  - **create\_samples:** Prepara los datos en conjuntos de entrenamiento, validación y prueba.
  - **train:** Entrena el modelo utilizando los datos de entrada y parámetros definidos.
  - **eval\_model:** Evalúa el modelo utilizando los datos de prueba y calcula las atenciones.
  - **show attentions:** Visualiza las atenciones de las variables y temporales.
  - **show\_RUL\_graph:** Muestra gráficos de RUL real y estimada.

Además, también tiene las siguientes carpetas:

- **models:** Carpeta para guardar los modelos entrenados. Los pesos del modelo se guardan aquí después del entrenamiento (`models/modelo_{fd}.h5`).
- **results:** Carpeta para guardar los resultados obtenidos después de las pruebas. Incluye gráficos y métricas de evaluación generadas durante la ejecución de `predictions.ipynb`.

## Apéndice D: Resultados Adicionales

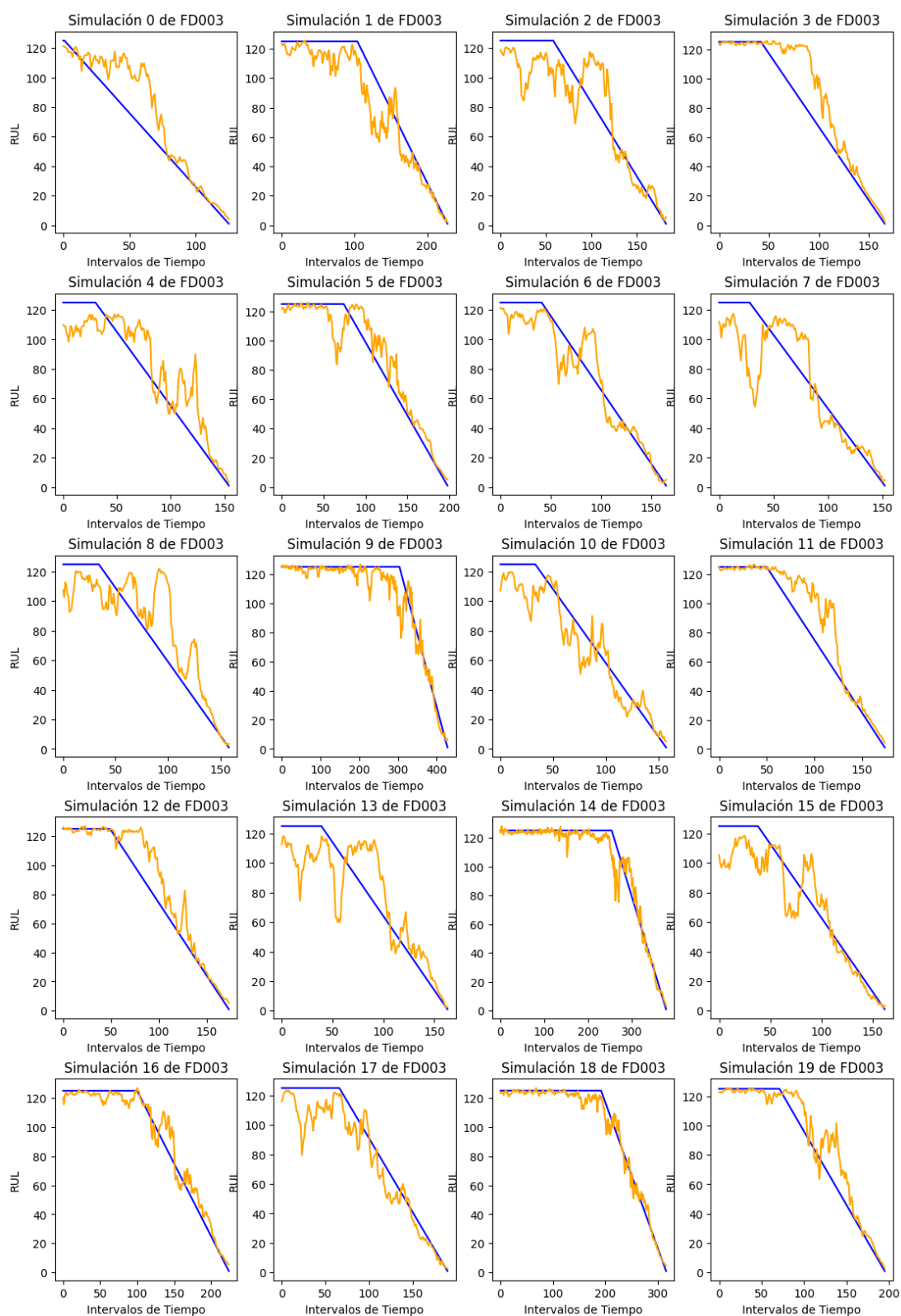


Figura D. 1. Simulaciones de validación de FD003 usando RUL-LAXCAT para los 9 mejores sensores.



## Referencias

- [1] Nahuel Costa, Luciano Sánchez, "RUL-RVE: Interpretable assessment of Remaining Useful Life", *Software Impacts*, vol. 13, 2022. Available: <https://www.sciencedirect.com/science/article/pii/S2665963822000537>.
- [2] Cummins, A. Sommers, S. B. Ramezani, S. Mittal, J. Jabour, M. Seale y S. Rahimi, "Explainable Predictive Maintenance: A Survey of Current Methods, Challenges and Opportunities," *arXiv:2401.07871*, 2024. Available: [\[2401.07871\] Explainable Predictive Maintenance: A Survey of Current Methods, Challenges and Opportunities \(arxiv.org\)](#).
- [3] T. Rojat, R. Puget, D. Filliat, J. Del Ser, R. Gelin y N. Díaz-Rodríguez, "Explainable Artificial Intelligence (XAI) on TimeSeries Data: A Survey" *arXiv:2104.00950*, 2021. Available: [\[2104.00950\] Explainable Artificial Intelligence \(XAI\) on TimeSeries Data: A Survey \(arxiv.org\)](#).
- [4] TY. Hsieh, S. Wang, Y. Sun y V. Honavar, "Explainable Multivariate Time Series Classification: A Deep Neural Network Which Learns To Attend To Important Variables As Well As Informative Time Intervals," *arXiv*, 2020. Available: [\[2011.11631\] Explainable Multivariate Time Series Classification: A Deep Neural Network Which Learns To Attend To Important Variables As Well As Informative Time Intervals \(arxiv.org\)](#).
- [5] X. Zhang, Y. Gao, J. Lin, and C.-T. Lu, "TapNet: Multivariate Time Series Classification with Attentional Prototypical Network", *AAAI*, vol. 34, no. 04, pp. 6845-6852, Apr. 2020. Available: [TapNet: Multivariate Time Series Classification with Attentional Prototypical Network | Proceedings of the AAAI Conference on Artificial Intelligence](#).
- [6] Molnar, C. "Interpretable Machine Learning: A Guide for Making Black Box Models Explainable". 2.<sup>a</sup> ed, 2022. Available: [3.5 Properties of Explanations | Interpretable Machine Learning \(christophm.github.io\)](#)
- [7] A. Moreno, E. Armengol, J. Béjar Alonso, L. A. Belanche Muñoz, C. U. Cortés García, R. Gavalda Mestre, J. M. Gimeno, M. Martín Muñoz, and M. Sánchez-Marré, "Aprendizaje automático". *Edicions UPC*, 1994. Disponible: <https://upcommons.upc.edu/bitstream/handle/2099.3/36157/9788483019962.pdf?sequence=1&isAllowed=y>
- [8] Fernando Sancho Caparrini, "Aprendizaje Supervisado y No Supervisado". Departamento de Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla, 2020. Disponible: [https://www.cs.us.es/~fsancho/Blog/posts/Aprendizaje\\_Supervisado\\_No\\_Superviado.md.html#](https://www.cs.us.es/~fsancho/Blog/posts/Aprendizaje_Supervisado_No_Superviado.md.html#)
- [9] "Aprendizaje semisupervisado". Wikipedia. Disponible: [https://es.wikipedia.org/wiki/Aprendizaje\\_semisupervisado](https://es.wikipedia.org/wiki/Aprendizaje_semisupervisado)
- [10] "Aprendizaje por refuerzo". Wikipedia. Disponible: [https://es.wikipedia.org/wiki/Aprendizaje\\_por\\_refuerzo](https://es.wikipedia.org/wiki/Aprendizaje_por_refuerzo)

- [11] Bishop, Christopher M., Nasrabadi, Nasser M., "Pattern recognition and machine learning". New York: Springer, 2006. Disponible: <https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>
- [12] "Aprendizaje Automático Supervisado", 2021. Master Data Science online, Universidad Camilo José Cela. Disponible: <https://masterdatascience.online/aprendizaje-automatico-supervisado/>
- [13] Sandoval Serrano and Lillian Judith, "Algoritmos de aprendizaje automático para análisis y predicción de datos". Revista Tecnológica, no 11, pp. 36.40, 2018. Disponible: <http://redicces.org.sv/jspui/handle/10972/3626>
- [14] James, Gareth, "An introduction to statistical learning". New York: Springer, 2013. Disponible: <http://www.ssc.upenn.edu/~fdiebold/NoHesitations/BookAdvanced.pdf>
- [15] Sereno, Mauro. "A Bayesian approach to linear regression in astronomy." Monthly Notices of the Royal Astronomical Society, Volume 455, Issue 2, pp. 2149-2162, 2016. Disponible: <https://academic.oup.com/mnras/article-abstract/455/2/2149/1111686>
- [16] Lucian Bușe, Mirela Ganea, Daniel Cîrciumaru, "Using linear regression in the analysis of financial-economic performances". Annals of University of Craiova- Economic Sciences Series, vol. 2, no 38, p. 32-43, 2010. Disponible: [http://feaa.central.ucv.ro/annals/v2\\_2010/0038v2-002.pdf](http://feaa.central.ucv.ro/annals/v2_2010/0038v2-002.pdf)
- [17] "Regresión lineal". Wikipedia. Disponible: [https://es.wikipedia.org/wiki/Regresi%C3%B3n\\_lineal](https://es.wikipedia.org/wiki/Regresi%C3%B3n_lineal)
- [18] Nelson Obregón-Neira, "Estructura de un árbol de decisión". ResearchGate, 2012. Disponible: [https://www.researchgate.net/figure/Figura-18-Estructura-de-un-arbol-de-decision\\_fig6\\_49175438](https://www.researchgate.net/figure/Figura-18-Estructura-de-un-arbol-de-decision_fig6_49175438)
- [19] Will Koehrsen, "Random Forest Simple Explanation". Medium, Dic 27, 2017. Disponible: <https://williamkoehrsen.medium.com/random-forest-simple-explanation-377895a60d2d>
- [20] Podgorelec, Vili, "Decision trees: an overview and their use in medicine". Journal of medical systems 26, 445–463, 2002. Disponible: [https://idp.springer.com/authorize/casa?redirect\\_uri=https://link.springer.com/article/10.1023/A:1016409317640&casa\\_token=KBhMzQBd7rcAAAAA:3FnFeekrJUDAgtgJlfy482UoOXltpln\\_TUSKmetodwyUcFRDzgRmzVAtqnVw3fAnmGE0zegQPlbzA6x8lOk](https://idp.springer.com/authorize/casa?redirect_uri=https://link.springer.com/article/10.1023/A:1016409317640&casa_token=KBhMzQBd7rcAAAAA:3FnFeekrJUDAgtgJlfy482UoOXltpln_TUSKmetodwyUcFRDzgRmzVAtqnVw3fAnmGE0zegQPlbzA6x8lOk)
- [21] Karim, Masud, Rahman, Rashedur M., "Decision tree and naive bayes algorithm for classification and generation of actionable knowledge for direct marketing". Journal of Software Engineering and Applications, Vol.6, No.4, 2013. Disponible: [https://www.scirp.org/html/6-9301587\\_30463.htm](https://www.scirp.org/html/6-9301587_30463.htm)
- [22] César Chique Rodríguez, "Máquina de Soporte Vectorial (SVM)". Medium, Sep 3, 2022. Disponible: <https://medium.com/@csarchiquerodriguez/maquina-de-soporte-vectorial-svm-92e9f1b1b1ac>

- [23] Xing, Wenchao, Yilin Bei, "Medical health big data classification based on KNN classification algorithm." IEEE Access, vol. 8, pp. 28808-28819, 2020. Disponible: <https://ieeexplore.ieee.org/abstract/document/8911389/>
- [24] Sachinoni, "K Nearest Neighbours — Introduction to Machine Learning Algorithms", Medium, Ene 11, 2023. Disponible: <https://medium.com/@sachinoni600517/k-nearest-neighbours-introduction-to-machine-learning-algorithms-9dbc9d9fb3b2>
- [25] Rish, Irina, "An empirical study of the naive Bayes classifier". IJCAI 2001 Workshop on Empirical Methods in Artificial Intelligence, pp. 41-46, 2001. Disponible: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=2825733f97124013e8841b3f8a0f5bd4ee4af88a>
- [26] Ian Goodfellow, Yoshua Bengio, Aaron Courville, "Deep Learning". The MIT Press, 2016. Disponible: <https://books.google.es/books?hl=es&lr=&id=omivDQAAQBAJ&oi=fnd&pg=PR5&dq=deep+learning&ots=MON1dplDTX&sig=qauTqzQC4CknwTg7SoXPzYf-Joo#v=onepage&q&f=false>
- [27] "¿Qué es Deep Learning y qué es una red neuronal?". Datademia. Disponible: <https://datademia.es/blog/que-es-deep-learning-y-que-es-una-red-neuronal>
- [28] M. Gheisari, G. Wang and M. Z. A. Bhuiyan, "A Survey on Deep Learning in Big Data", 22017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC), pp. 173-180, 2017. Disponible: <https://ieeexplore.ieee.org/document/8005992>
- [29] Jordi Torres, "Redes Neuronales Recurrentes". torres.ai, 2019. Disponible: <https://torres.ai/redes-neuronales-recurrentes/>
- [30] Chaudhary, Janhavi R., Patel, Ankit C., "Bilingual machine translation using RNN based deep learning". Int J Sci Res Sci Eng Technol, vol. 4, no 4, p. 1480-1484, 2018. Disponible: [4621-libre.pdf \(d1wqtxts1xzle7.cloudfront.net\)](https://www.researchgate.net/publication/328111111-Bilingual-machine-translation-using-RNN-based-deep-learning)
- [31] Biswas, Saroj Kr, "Weather prediction by recurrent neural network dynamics". International Journal of Intelligent Engineering Informatics, vol. 2, no 2-3, p. 166-180, 2014. Disponible: <https://www.inderscienceonline.com/doi/abs/10.1504/IJIEI.2014.066208>
- [32] Sonata, I., "Autonomous car using CNN deep learning algorithm". Journal of Physics: Conference Series. IOP Publishing, Vol. 1869, No. 1, 2021. Disponible: <https://iopscience.iop.org/article/10.1088/1742-6596/1869/1/012071/meta>
- [33] Ikromovich, Hamidov Oybek, Mamatkulovich, Babakulov Bekzod, "Facial recognition using transfer learning in the deep CNN". Open Access Repository, vol. 4, no 3, p. 502-507, 2023. Disponible: <https://oarepo.org/index.php/oa/article/view/2264>
- [34] "How CNNs Work". MathWorks. Disponible: <https://www.mathworks.com/discovery/convolutional-neural-network.html>

- [35] Md Zahangir Alom, Tarek M. Taha, Christopher Yakopcic, Stefan Westberg, Paheding Sidike, Mst Shamima Nasrin, Brian C Van Esesn, Abdul A S. Awwal, Vijayan K. Asari, "The history began from alexnet: A comprehensive survey on deep learning approaches". arXiv preprint arXiv:1803.01164, 2018. Disponible: <https://arxiv.org/abs/1803.01164>
- [36] G. Cohen, S. Afshar, J. Tapson and A. van Schaik, "EMNIST: Extending MNIST to handwritten letters". 2017 International Joint Conference on Neural Networks (IJCNN), Anchorage, AK, USA, 2017, pp. 2921-2926. Disponible: [https://ieeexplore.ieee.org/abstract/document/7966217?casa\\_token=aF10eWc0f\\_4AAA AA:JD7lseR6s\\_iYiWzP2TIQkZsrr-6iQU8iviZoABGv2TRd1MxXYQ\\_2ph5KceZ6l4tGfUavFQXzF0k](https://ieeexplore.ieee.org/abstract/document/7966217?casa_token=aF10eWc0f_4AAA AA:JD7lseR6s_iYiWzP2TIQkZsrr-6iQU8iviZoABGv2TRd1MxXYQ_2ph5KceZ6l4tGfUavFQXzF0k)
- [37] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta and A. A. Bharath, "Generative Adversarial Networks: An Overview". IEEE Signal Processing Magazine, vol. 35, no. 1, pp. 53-65, Enero 2018. Disponible: [https://ieeexplore.ieee.org/abstract/document/8253599?casa\\_token=WiscF2rLL3cAAAAA:k37TKxggCdfqlZylcRj5vp1Ka99tiqLypt9c83Yi4lCloALFbqgxsUyVkaF\\_nHxLeuKm2Hk7RCk](https://ieeexplore.ieee.org/abstract/document/8253599?casa_token=WiscF2rLL3cAAAAA:k37TKxggCdfqlZylcRj5vp1Ka99tiqLypt9c83Yi4lCloALFbqgxsUyVkaF_nHxLeuKm2Hk7RCk)
- [38] Thalles Silva, "A Short Introduction to Generative Adversarial Networks". Thalles' blog, 2017. Disponible: <https://sthalles.github.io/intro-to-gans/>
- [39] Walter Hugo Lopez Pinaya, Sandra Vieira, Rafael Garcia-Dias, Andrea Mechelli, "Machine Learning". Academic Press, Chapter 11 - Autoencoders, Pages 193-208, 2020. Disponible: <https://doi.org/10.1016/B978-0-12-815739-8.00011-0>
- [40] Yun-Gyung Cheong, "Analysis of Autoencoders for Network Intrusion Detection". ResearchGate, 2021. Disponible: [https://www.researchgate.net/figure/An-example-autoencoder-model-architecture-with-symmetrical-encoder-and-decoder-networks\\_fig2\\_352703131](https://www.researchgate.net/figure/An-example-autoencoder-model-architecture-with-symmetrical-encoder-and-decoder-networks_fig2_352703131)
- [41] Zeng, Kuo-Hao, Shoeybi, Mohammad, Liu, Ming-Yu, "Style example-guided text generation using generative adversarial transformers". arXiv preprint arXiv:2003.00674, 2020. Disponible: <https://arxiv.org/abs/2003.00674>
- [42] Gavita Regunath, "Understanding The Difference Between AI, ML, And DL: Using An Incredibly Simple Example". Advancing Analytics, 2021. Disponible: <https://www.advancinganalytics.co.uk/blog/2021/12/15/understanding-the-difference-between-ai-ml-and-dl-using-an-incredibly-simple-example>
- [43] P. P. Shinde and S. Shah, "A Review of Machine Learning and Deep Learning Applications," 2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA), p. 1-6, Pune, India, 2018. Disponible: <https://ieeexplore.ieee.org/abstract/document/8697857>
- [44] Daisy Tsang, "White Box vs. Black Box Algorithms in Machine Learning". ActiveState, 2023. Disponible: <https://www.activestate.com/blog/white-box-vs-black-box-algorithms-in-machine-learning/>
- [45] O. Loyola-González, "Black-Box vs. White-Box: Understanding Their Advantages and Weaknesses From a Practical Point of View," in IEEE Access, vol. 7, pp.

- 154096-154113, 2019. Disponible:  
<https://ieeexplore.ieee.org/abstract/document/8882211>
- [46] Confalonieri, Roberto, "What makes a good explanation? Cognitive dimensions of explaining intelligent machines". CogSci, pp. 25-26, 2019. Disponible:  
[confalonieri\\_et\\_al\\_cogsci\\_19-libre.pdf\(d1wqtxts1xzle7.cloudfront.net\)](https://confalonieri_et_al_cogsci_19-libre.pdf(d1wqtxts1xzle7.cloudfront.net))
- [47] Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin, ""Why should I trust you?" Explaining the predictions of any classifier." Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp. 1135-1144, 2016. Disponible:  
<https://dl.acm.org/doi/abs/10.1145/2939672.2939778>
- [48] "Descifrando los diagnósticos de la Inteligencia Artificial". Universidad Complutense de Madrid, 2020. Disponible: <https://masterisd.es/descifrando-los-diagnosticos-de-la-inteligencia-artificial/>
- [49] Thomas McGrath, Andrei Kapishnikov, Nenad Tomašev, Adam Pearce, Martin Wattenberg, Demis Hassabis, Been Kim, Ulrich Paquet, Vladimir Kramnik, "Acquisition of chess knowledge in AlphaZero." Proceedings of the National Academy of Sciences, vol. 119, no 47, 2022. Disponible:  
<https://www.pnas.org/doi/abs/10.1073/pnas.2206625119>
- [50] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bénéttot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila, Francisco Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI". Information Fusion, vol. 58, pp. 82-115, 2020. Disponible:  
<https://doi.org/10.1016/j.inffus.2019.12.012>
- [51] Mydatamood, "¿Qué es XAI?: Explainable Artificial Intelligence (y por qué es importante para los consumidores)". Medium, Jul 23, 2020. Disponible:  
<https://medium.com/mydatamood/qu%C3%A9-es-xai-e7548fbd9edb>
- [52] Zhaoyang Niu, Guoqiang Zhong, Hui Yu, "A review on the attention mechanism of deep learning", Neurocomputing, Vol. 452, pp. 48-62, 2021. Disponible:  
<https://doi.org/10.1016/j.neucom.2021.03.091>
- [53] Bordt, S., Finck, M., Raidl, E., Von Luxburg "Post-hoc explanations fail to achieve their purpose in adversarial contexts". Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency, pp. 891-905, 2022. Disponible:  
<https://dl.acm.org/doi/abs/10.1145/3531146.3533153>
- [54] Marco Tulio Ribeiro, Sameer Singh, Carlos Guestrin, "Local Interpretable Model-Agnostic Explanations (LIME): An Introduction". arXiv preprint arXiv:1606.05386, 2016. Disponible: <https://www.oreilly.com/content/introduction-to-local-interpretable-model-agnostic-explanations-lime/>
- [55] Scott Lundberg, "Welcome to the SHAP documentation". SHAP, 2018. Disponible:  
<https://shap.readthedocs.io/en/latest/>
- [56] M. B. Muhammad and M. Yeasin, "Eigen-CAM: Class Activation Map using Principal Components," 2020 International Joint Conference on Neural Networks (IJCNN),

- Glasgow, UK, 2020, pp. 1-7. Disponible:  
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9206626>
- [57] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, Dhruv Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization". International Conference on Computer Vision (ICCV), 2017, pp. 618-626. Disponible:  
[https://openaccess.thecvf.com/content\\_iccv\\_2017/html/Selvaraju\\_Grad-CAM\\_Visual\\_Explanations\\_ICCV\\_2017\\_paper.html](https://openaccess.thecvf.com/content_iccv_2017/html/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.html)
- [58] A. Chattopadhyay, A. Sarkar, P. Howlader and V. N. Balasubramanian, "Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks," 2018 IEEE Winter Conference on Applications of Computer Vision (WACV), Lake Tahoe, NV, USA, 2018, pp. 839-847. Disponible:  
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8354201>
- [59] Ioannis Mollas, Nick Bassiliades, Ioannis Vlahavas, Grigorios Tsoumakas, "LionForests: Local Interpretation of Random Forests". arXiv preprint arXiv:1911.08780, 2019. Disponible: <https://arxiv.org/pdf/1911.08780>
- [60] Oana-Maria Camburu, Eleonora Giunchiglia, Jakob Foerster, Thomas Lukasiewicz, Phil Blunsom, "Can I trust the explainer? Verifying post-hoc explanatory methods". arXiv, 2019. Disponible: <https://arxiv.org/abs/1910.02065>
- [61] "Intrinsic vs. post hoc interpretability". ResearchGate. Disponible:  
[https://www.researchgate.net/figure/Intrinsic-vs-post-hoc-interpretability\\_fig5\\_352986044](https://www.researchgate.net/figure/Intrinsic-vs-post-hoc-interpretability_fig5_352986044)
- [62] S. B. Ramezani, L. Cummins, B. Killen, R. Carley, A. Amirlatifi, S. Rahimi, M. Seale, and L. Bian, "Scalability, explainability and performance of data driven algorithms in predicting the remaining useful life: A comprehensive review," IEEE Access, 2023. Disponible:  
<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10103701>
- [63] H. Choi, D. Kim, J. Kim, J. Kim, and P. Kang, "Explainable anomaly detection framework for predictive maintenance in manufacturing systems." Applied Soft Computing, 2022. Disponible:  
<https://www.sciencedirect.com/science/article/abs/pii/S1568494622004069>
- [64] Maria Eddarhri, Jihad Adib, Mustapha Hain, Abdelaziz Marzak, "Towards predictive maintenance: the case of the aeronautical industry". Procedia Computer Science, 2022. Disponible:  
<https://www.sciencedirect.com/science/article/pii/S1877050922007207>
- [65] Won Shin, Jeongyun Han, Wonjong Rhee, "AI-assistance for predictive maintenance of renewable energy systems". Energy, 2021. Disponible:  
<https://www.sciencedirect.com/science/article/pii/S0360544221000244>
- [66] S. Vollert, M. Atzmueller and A. Theissler, "Interpretable Machine Learning: A brief survey from the predictive maintenance perspective," 2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), Vasteras,

- Sweden, 2021. Disponible: [Interpretable Machine Learning: A brief survey from the predictive maintenance perspective | IEEE Conference Publication | IEEE Xplore](#)
- [67] Xu M, Baraldi P, Lu X, Cannarile F, Zio E., “Anomaly detection for industrial systems using generative adversarial networks”. Proceedings of the 4th International Conference on System Reliability and Safety, 2019. Disponible: [https://www.researchgate.net/profile/Francesco-Cannarile/publication/337534778\\_Anomaly\\_Detection\\_for\\_Industrial\\_Systems\\_using\\_Generative\\_Adversarial\\_Networks/links/5ddd0fbd4585159aa448f03d/Anomaly-Detection-for-Industrial-Systems-using-Generative-Adversarial-Networks.pdf](https://www.researchgate.net/profile/Francesco-Cannarile/publication/337534778_Anomaly_Detection_for_Industrial_Systems_using_Generative_Adversarial_Networks/links/5ddd0fbd4585159aa448f03d/Anomaly-Detection-for-Industrial-Systems-using-Generative-Adversarial-Networks.pdf)
- [68] Baraldi P, Di Maio F, Rigamonti M, Zio E, Seraoui R, “Unsupervised clustering of vibration signals for identifying anomalous conditions in a nuclear turbine”. J Intell Fuzzy Syst, 2015. Disponible: <https://doi.org/10.3233/IFS-141459>
- [69] Liu H, Zhou J, Xu Y, Zheng Y, Peng X, Jiang W, “Unsupervised fault diagnosis of rolling bearings using a deep neural network based on generative adversarial networks”. Neurocomputing, 2018. Disponible: <https://doi.org/10.1016/j.neucom.2018.07.034>
- [70] Xu M, Baraldi P, Zio E., “Fault diagnostics by conceptors-aided clustering”. Proceedings of the 30th European safety and reliability conference, ESREL 2020 and 15th probabilistic safety assessment and management conference, PSAM, 2020. Disponible: <https://re.public.polimi.it/handle/11311/1181257>
- [71] Xiao-Sheng Si, Wenbin Wang, Chang-Hua Hu, Dong-Hua Zhou, “Remaining useful life estimation – A review on the statistical data driven approaches”. European Journal of Operational Research, Volume 213, Issue 1, 2011. Disponible: <https://doi.org/10.1016/j.ejor.2010.11.018>
- [72] Zio, Enrico. "Prognostics and Health Management (PHM): Where are we and where do we (need to) go in theory and practice." Reliability Engineering & System Safety 218, 2022. Disponible: [https://www.sciencedirect.com/science/article/pii/S0951832021006153?casa\\_token=dhbVnyUGM3IAAAAA:laecwWUAqdwWftEek-Tcjp0pe\\_FxPNnQ4bbWXH\\_2jpLzY3\\_XtiBp1eFYor\\_olodNiNUCjcsIWvaZ](https://www.sciencedirect.com/science/article/pii/S0951832021006153?casa_token=dhbVnyUGM3IAAAAA:laecwWUAqdwWftEek-Tcjp0pe_FxPNnQ4bbWXH_2jpLzY3_XtiBp1eFYor_olodNiNUCjcsIWvaZ)
- [73] Xu M, Baraldi P, Al-Dahidi S, Zio E, “Fault prognostics by an ensemble of Echo State Networks in presence of event based measurements.” Eng Appl Artif Intell, 2020. Disponible: <https://doi.org/10.1016/j.engappai.2019.103346>
- [74] J. Jakubowski, P. Stanisz, S. Bobek, G.J. Nalepa, “Anomaly detection in asset degradation process using variational autoencoder and explanations”. Sensors, 2021. Disponible: <https://www.mdpi.com/1424-8220/22/1/291>
- [75] A. Lourenço, M. Fernandes, A. Canito, A. Almeida, and G. Marreiros, “Using an explainable machine learning approach to minimize opportunistic maintenance interventions”. International Conference on Practical Applications of Agents and Multi-Agent Systems, Springer, 2022. Disponible: [https://link.springer.com/chapter/10.1007/978-3-031-18697-4\\_4](https://link.springer.com/chapter/10.1007/978-3-031-18697-4_4)

- [76] M. Gashi, B. Mutlu, and S. Thalmann, "Impact of interdependencies: Multicomponent system perspective toward predictive maintenance based on machine learning and XAI." *Applied Sciences*, vol. 13, no. 5, 2023. Disponible: <https://www.mdpi.com/2076-3417/13/5/3088>
- [77] G. Protopapadakis, A. I. Kalfas, "Explainable and interpretable AI Assisted remaining useful life estimation for aeroengines". *ASME Turbo Expo 2022: Turbomachinery Technical Conference and Exposition*, 2022. Disponible: <https://asmedigitalcollection.asme.org/GT/proceedings-abstract/GT2022/85987/1148644>
- [78] B2, "NASA Turbofan Jet Engine Data Set". Kaggle, 2019. Disponible: [NASA Turbofan Jet Engine Data Set \(kaggle.com\)](https://www.kaggle.com/datasets/nasa/turbofan-jet-engine-data-set)
- [79] J.-H. Han, S.-U. Park, and S.-K. Hong, "A study on the effectiveness of current data in motor mechanical fault diagnosis using XAI". *Journal of Electrical Engineering & Technology*, 2022. Disponible: [https://idp.springer.com/authorize/casa?redirect\\_uri=https://link.springer.com/article/10.1007/s42835-022-01207-y&casa\\_token=7COMxFtt\\_FIAAAAA:uy69j1zwITcztciT2NTj3R20HtfsICWoZrtLbOBW0dQhrxikRUnDp2f\\_OkFQ1-aM65Nz7C-55-A0QGb3d2U](https://idp.springer.com/authorize/casa?redirect_uri=https://link.springer.com/article/10.1007/s42835-022-01207-y&casa_token=7COMxFtt_FIAAAAA:uy69j1zwITcztciT2NTj3R20HtfsICWoZrtLbOBW0dQhrxikRUnDp2f_OkFQ1-aM65Nz7C-55-A0QGb3d2U)
- [80] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition". *Proceedings of the IEEE*, 1998. Disponible: [https://ieeexplore.ieee.org/abstract/document/726791/?casa\\_token=gl4M4mVbhKAAAAA:YRkxsPw6R7DvRtZrGCRWYCxKDFsoLjS5ZGHR\\_SYStPNDWAOVyXpGhmLqRYsBWU-a6b5C8t5da9va](https://ieeexplore.ieee.org/abstract/document/726791/?casa_token=gl4M4mVbhKAAAAA:YRkxsPw6R7DvRtZrGCRWYCxKDFsoLjS5ZGHR_SYStPNDWAOVyXpGhmLqRYsBWU-a6b5C8t5da9va)
- [81] J. Jakubowski, P. Stanisz, S. Bobek, and G. J. Nalepa, "Roll wear prediction in strip cold rolling with physics-informed autoencoder and counterfactual explanations". *2022 IEEE 9th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE, 2022. Disponible: [https://ieeexplore.ieee.org/abstract/document/10032357/?casa\\_token=9muN3Mi7OqYAAAAA:s5FEnpLILRDW0inPao-2AwAhdRW1nBTH5kZwTCpJxfDeYEGVOqftaUXEQiotbTLgGS56f8q2QZgA](https://ieeexplore.ieee.org/abstract/document/10032357/?casa_token=9muN3Mi7OqYAAAAA:s5FEnpLILRDW0inPao-2AwAhdRW1nBTH5kZwTCpJxfDeYEGVOqftaUXEQiotbTLgGS56f8q2QZgA)
- [82] Jakub Jakubowski, Przemysław Stanisz, Szymon Bobek, Grzegorz J. Nalepa, "Roll wear prediction in strip cold rolling with physics-informed autoencoder and counterfactual explanations". *2022 IEEE 9th International Conference on Data Science and Advanced Analytics (DSAA)* (pp. 1-10), IEEE, 2022. Disponible: [Roll Wear Prediction in Strip Cold Rolling with Physics-Informed Autoencoder and Counterfactual Explanations | IEEE Conference Publication | IEEE Xplore](https://ieeexplore.ieee.org/abstract/document/10032357)
- [83] O. Mey and D. Neufeld, "Explainable ai algorithms for vibration data-based fault detection: Use case-adapted methods and critical evaluation". *Sensors*, 2022. Disponible: <https://www.mdpi.com/1424-8220/22/23/9037>
- [84] N. Mylonas, I. Mollas, N. Bassiliades, and G. Tsoumakas, "Local multilabel explanations for random forest". *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2022. Disponible: [https://link.springer.com/chapter/10.1007/978-3-031-23618-1\\_25](https://link.springer.com/chapter/10.1007/978-3-031-23618-1_25)

- 
- [85] David M. Blei, Alp Kucukelbir, Jon D. McAuliffe, "Variational inference: A review for statisticians". Journal of the American statistical Association, vol. 112, no 518, p. 859-877, 2017. Disponible: <https://www.tandfonline.com/doi/full/10.1080/01621459.2017.1285773?scroll=top&needAccess=true>
- [86] Nahuel Costa Cortez, "RUL-RVE". GitHub, 2021. Disponible: [GitHub - SoftwareImpacts/SIMPAC-2022-58](#)
- [87] Jason Brownlee, "Deep learning with Python: develop deep learning models on Theano and TensorFlow using Keras". Machine Learning Mastery, 2016. Disponible: [https://books.google.es/books?hl=es&lr=&id=K-ipDwAAQBAJ&oi=fnd&pg=PP1&dq=tensorflow+and+keras&ots=orRn0lYnys&sig=NNwKx1OPpDMnOgLJKye0Ho\\_KYw#v=onepage&q=tensorflow%20and%20keras&f=false](https://books.google.es/books?hl=es&lr=&id=K-ipDwAAQBAJ&oi=fnd&pg=PP1&dq=tensorflow+and+keras&ots=orRn0lYnys&sig=NNwKx1OPpDMnOgLJKye0Ho_KYw#v=onepage&q=tensorflow%20and%20keras&f=false)
- [88] Sagar Imambi, Kolla Bhanu Prakash, G. R. Kanagachidambaresan, "Introduction to pytorch". Deep learning with python: learn best practices of deep learning models with PyTorch, pág. 27-91, 2021. Disponible: [https://link.springer.com/chapter/10.1007/978-3-030-57077-4\\_10](https://link.springer.com/chapter/10.1007/978-3-030-57077-4_10)
- [89] Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, Quoc V. Le, "Don't decay the learning rate, increase the batch size." *arXiv preprint arXiv:1711.00489*, 2017. Disponible: <https://arxiv.org/pdf/1711.00489>
- [90] "¿Cuánto cobra un ingeniero recién graduado?". UFV, 2023. Disponible: <https://www.ufv.es/cuanto-cobra-un-ingeniero-recien-graduado-preguntas-grados/>
- [91] "¿Cuánto gana Ingeniero de datos en España?". Jooble. Disponible: <https://es.jooble.org/salary/ingeniero-de-datos>
- [92] "Objetivos Desarrollo Sostenible". Naciones Unidas, 2015. Disponible: <https://www.un.org/sustainabledevelopment/es/objetivos-de-desarrollo-sostenible/>